

## Chapter 1: Introduction to React.js (React.js નો પરિચય)

### ૧. What is React.js (React.js શું છે?)

- **વ્યાખ્યા:** React.js એ કોઈ પ્રોગ્રામિંગ લેંગ્વેજ નથી, પણ તે વેબ એપ્લિકેશનના **UI (User Interface)** એટલે કે સ્ક્રીન પર દેખાતા નકશા અને પેજીસ બનાવવા માટેની એક ઓપન-સોર્સ **JavaScript Library** છે.
- તે ફેસબુક (Meta) દ્વારા ૨૦૧૩ માં બનાવવામાં આવી હતી.
- તેનો મુખ્ય ઉપયોગ વેબસાઇટના ફ્રન્ટએન્ડ (Frontend) ને ખૂબ જ ફાસ્ટ અને ડાયનેમિક બનાવવાનો છે. આજે ફેસબુક, ઇન્સ્ટાગ્રામ, અને નેટફ્લિક્સ જેવી મોટી સાઇટ્સ આના પર જ ચાલે છે.

### ૨. Why React.js? (React.js કેમ વાપરવું?)

- **Single Page Application (SPA):** પરંપરાગત વેબસાઇટ્સમાં જ્યારે તમે કોઈ લિંક પર ક્લિક કરો, ત્યારે આખું પેજ લોડ (રીફ્રેશ) થાય છે અને સફેદ સ્ક્રીન દેખાય છે. રીએક્ટમાં આખું પેજ ક્યારેય રીફ્રેશ નથી થતું. યુઝર ક્લિક કરે એટલે ફક્ત એટલો જ ભાગ સેકન્ડના હજારમા ભાગમાં બદલાઈ જાય છે, જે યુઝરને મોબાઇલ એપ જેવો સ્મૂથ અનુભવ આપે છે.
- **Reusable Components (ફરીથી વાપરી શકાય તેવા ભાગો):** આ રીએક્ટનો સૌથી મોટો ફાયદો છે. આપણે વેબસાઇટના નાના-નાના ભાગો (જેમ કે બટન, નેવબાર, કાર્ડ) એક જ વાર બનાવીને આખી સાઇટમાં ગમે ત્યાં ગમે તેટલી વાર વાપરી શકીએ છીએ.

### ૩. Component-Based Architecture (કમ્પોનન્ટ-બેઝ્ડ માળખું)

- **સમજૂતી:** રીએક્ટમાં આખી વેબસાઇટને નાના-નાના ટુકડાઓમાં વહેંચી દેવામાં આવે છે. આ દરેક ટુકડાને **Component (કમ્પોનન્ટ)** કહેવાય છે.
- દાખલા તરીકે, એક વેબસાઇટના હેડર માટે Header કમ્પોનન્ટ, નીચેના ભાગ માટે Footer કમ્પોનન્ટ અને પ્રોડક્ટ બતાવવા માટે ProductCard કમ્પોનન્ટ હોય. આ બધા કમ્પોનન્ટ્સ ભેગા થઈને એક આખું વેબ પેજ બને છે. આનાથી કોડ મેનેજ કરવો એકદમ સરળ થઈ જાય છે.

### ૪. Virtual DOM (વર્ચ્યુઅલ ડોમેનો કન્સેપ્ટ)

- **સમજૂતી:** બ્રાઉઝરનું પોતાનું એક ઓરિજિનલ DOM (Document Object Model) હોય છે. સાદા જાવાસ્ક્રિપ્ટમાં જ્યારે કોઈ નાનો ફેરફાર થાય, ત્યારે આખું DOM અપડેટ થાય છે જે સ્વો પ્રોસેસ છે.
- **React નો જાદુ:** રીએક્ટ મેમરીમાં ઓરિજિનલ DOM ની એક કોપી કેટલેક નકલ સાચવી રાખે છે, જેને **Virtual DOM** કહે છે. જ્યારે સાઇટમાં કોઈ ડેટા બદલાય, ત્યારે રીએક્ટ પહેલા તેની નકલ (Virtual DOM) માં ફેરફાર કરે છે અને જુએ છે કે અસલી પેજમાં કેટલો ભાગ બદલાયો છે. તે ફક્ત એ જ બદલાયેલા ભાગને ઓરિજિનલ DOM માં જઈને સુધારે છે. આના કારણે રીએક્ટ એપ્લિકેશન્સ ખૂબ જ ફાસ્ટ ચાલે છે.

---

## Chapter 2: Setup and First React App (સેટઅપ અને પ્રથમ એપ)

### ૧. Requirements (જરૂરી સોફ્ટવેર)

React નો પ્રોજેક્ટ ચાલુ કરવા માટે તમારા કમ્પ્યુટરમાં મુખ્ય બે વસ્તુઓ હોવી જરૂરી છે:

1. **Node.js:** રીએક્ટ પ્રોજેક્ટના પેકેજ મેનેજ કરવા માટે.
2. **VS Code:** કોડ લખવા માટે.

## ૨. Creating App using Vite (Vite દ્વારા નવી એપ બનાવવી)

પહેલાના સમયમાં create-react-app વપરાતું હતું જે ઘણું સ્વો હતું. અત્યારના આધુનિક સમયમાં **Vite** નો ઉપયોગ થાય છે જે ખૂબ જ ઝડપી છે.

- **નવો પ્રોજેક્ટ શરૂ કરવાનો કમાન્ડ:**  
ટર્મિનલમાં લખો: `npm create vite@latest my-react-app -- --template react`
- એ પછી તમારા પ્રોજેક્ટ ફોલ્ડરમાં જવા લખો: `cd my-react-app`
- જરૂરી ફાઇલો ડાઉનલોડ કરવા લખો: `npm install`
- સર્વર ચાલુ કરવા માટેનો કમાન્ડ: `npm run dev`

આ કમાન્ડ આપતા જ ટર્મિનલમાં એક લોકલ લિંક (<http://localhost:5173>) મળશે, જેના પર ક્લિક કરવાથી તમારી પ્રથમ રીએક્ટ એપ બ્રાઉઝરમાં લાઇવ થઈ જશે.

## ૩. Folder Structure (ફોલ્ડરનું માળખું સમજવું)

પ્રોજેક્ટ ઓપન કરશો એટલે તમને મુખ્ય આ ફાઇલો જોવા મળશે:

- **node\_modules/:** રીએક્ટ ચલાવવા માટેના બધા જ જરૂરી કોડ પેકેજીસ આમાં હોય છે.
- **package.json:** પ્રોજેક્ટનું નામ, વર્ઝન અને ઇન્સ્ટોલ કરેલા ટૂલ્સનું લિસ્ટ આમાં હોય છે.
- **index.html:** આ તમારી આખી એપ્લિકેશનનું એકમાત્ર મેઇન HTML પેજ છે.
- **src/ (Source):** આપણો આખો અસલી કોડ આ ફોલ્ડરમાં જ લખવાનો હોય છે.
  - **main.jsx:** આ રીએક્ટનો એન્ટ્રી પોઇન્ટ છે, જે આપણા કોડને HTML ની સાથે જોડે છે.
  - **App.jsx:** આ આપણો મુખ્ય રૂટ કમ્પોનન્ટ છે, જે સ્ક્રીન પર દેખાય છે.

---

મેં રીએક્ટનો પાયો મજબૂત કરવા માટે **Chapter 1 અને 2** ના કન્સેપ્ટ્સ એકદમ સરળ ગુજરાતીમાં સમજાવી દીધા છે. હવે આગળ વધીને **Chapter 3: JSX (JavaScript XML)** અને તેના નિયમો શીખવાનું શરૂ કરીએ? જો કોઈ પ્રશ્ન હોય તો પૂછી શકો છો!

ચાલો, આપણે આગળ વધીએ અને

**Chapter 3: JSX (JavaScript XML)** અને તેના નિયમો વિશે એકદમ ઊંડાણપૂર્વક અને સરળ ગુજરાતીમાં શીખીએ.

---

## Chapter 3: JSX (JavaScript XML)

### ૧. What is JSX? (JSX શું છે?)

- **વ્યાખ્યા:** JSX નું પૂરું નામ **JavaScript XML** છે. તે રીએક્ટની એક ખાસ ખૂબી છે જે આપણને જાવાસ્ક્રિપ્ટ ફાઇલમાં જ સીધો **HTML જેવો કોડ** લખવાની મંજૂરી આપે છે.

- **તે કેવી રીતે કામ કરે છે?:** સામાન્ય રીતે બ્રાઉઝર JSX ને સમજી શકતું નથી. આથી બેકએન્ડમાં **Babel** નામનું એક કમ્પાઇલર ટૂલ આ JSX કોડને સાદા અને શુદ્ધ જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટમાં ફેરવી નાખે છે.
- **ફાયદો:** આના લીધે આપણે HTML અને લોજિક બંને એક જ કમ્પોનન્ટ ફાઇલમાં સાથે લખી શકીએ છીએ, જેથી કોડ એકદમ ટૂંકો અને સરળ બને છે.

---

## ૨. Rules for Writing JSX (JSX લખવાના મુખ્ય નિયમો)

જાવાસ્ક્રિપ્ટમાં HTML જેવો કોડ લખતી વખતે રીએક્ટના ૪ કડક નિયમો પાળવા પડે છે. જો આ નિયમોમાં ભૂલ થાય તો કોડ રન નહીં થાય:

### નિયમ ૧: સિંગલ પેરેન્ટ એલિમેન્ટ (Single Parent Element)

JSX ની અંદર તમે એકસાથે ઘણા બધા HTML ટેગ રિટર્ન (Return) ન કરી શકો. બધા ટેગ એક મુખ્ય **સિંગલ કન્ટેનર ટેગ** ની અંદર બંધ હોવા જોઈએ.

- ❌ ખોટી રીત:

```
jsx
return (
  <h1>હેલો રાહુલ</h1>
  <p>સ્વાગત છે.</p> // એરર આવશે કારણ કે બે અલગ પેરેન્ટ છે
);
```

Use code with caution.

- સાચી રીત (સાદો <div> વાપરીને):

```
jsx
return (
  <div>
    <h1>હેલો રાહુલ</h1>
    <p>સ્વાગત છે.</p>
  </div>
);
```

Use code with caution.

- 💡 **React Fragment (બેસ્ટ રીત):** જો તમે એક્સ્ટ્રા <div> ઉમેરીને HTML ને મોટું કરવા ન માંગતા હોવ, તો તમે ખાલી બ્રેકેટ <> ... </> વાપરી શકો છો, જેને **React Fragment** કહેવાય છે:

```
jsx
return (
  <>
```

```
<h1>હેલો રાહુલ</h1>

<p>સ્વાગત છે.</p>

</>

);
```

Use code with caution.

## નિયમ ૨: ક્લોઝિંગ ટેગ ફરજિયાત છે (Every Tag Must Be Closed)

સાદા HTML માં આપણે ક્યારેક `<br>` કે `<img>` ને બંધ નથી કરતા તો ચાલી જાય છે, પણ JSX માં બધા જ સેલ્ફ-ક્લોઝિંગ ટેગની પાછળ / લગાવીને બંધ કરવું જ પડે છે.

- ❌ ખોટી રીત: `` અથવા `<br>`
- સાચી રીત: `` અને `<br />`

## નિયમ ૩: class ના બદલે className અને for ના બદલે htmlFor

જાવાસ્ક્રિપ્ટની અંદર class અને for એ ઇન-બિલ્ટ કીવર્ડ્સ (Reserved Keywords) છે. આથી રીએક્ટમાં CSS નો ક્લાસ આપવા માટે class ના બદલે **className** લખવું પડે છે.

- ❌ ખોટી રીત: `<div class="card">`
- સાચી રીત: `<div className="card">`

## નિયમ ૪: કેમલકેસ સ્ટાઇલ (CamelCase Attributes)

JSX માં HTML ના બધા જ એટ્રિબ્યુટ્સ અને ઇવેન્ટ્સ **camelCase** ફોર્મેટમાં લખવા પડે છે (એટલે કે પહેલો અક્ષર નાનો અને બીજા શબ્દનો પહેલો અક્ષર મોટો).

- ❌ ખોટી રીત: `<button onclick={myFunc}>` અથવા `<div tabindex="1">`
- સાચી રીત: `<button onClick={myFunc}>` અને `<div tabIndex="1">`

---

## ૩. JavaScript Expressions in JSX (JSX માં જાવાસ્ક્રિપ્ટ વાપરવી)

જો તમારે JSX (HTML) ની વચ્ચે કોઈ જાવાસ્ક્રિપ્ટનો વેરીએબલ, ગણતરી કે ફંક્શન રન કરવું હોય, તો તેને **CURLY BRACES { }** ની અંદર લખવું પડે છે.

### પ્રેક્ટિકલ ઉદાહરણ (App.jsx):

```
jsx

function App() {

  const name = "અજય પટેલ";

  const age = 22;


  return (
```

```

<div className="user-box">
  {/* વેરીએબલ પ્રિન્ટ કરવા માટે {} વાપર્યા */}
  <h2>નામ: {name}</h2>
  <p>ઉંમર: {age} વર્ષ</p>

  {/* {} ની અંદર સાદી ગણતરી પણ કરી શકાય */}
  <p>પોર્ટફોલિયો સ્કોર: {10 + 20 + 15}</p>
</div>

);
}

```

export default App;

Use code with caution.

અહીં **Chapter 3** ના તમામ કન્સેપ્ટ્સ અને નિયમો પૂરા થાય છે.

મેં **JSX** ના બધા જ નિયમો સરળતાથી સમજાવી દીધા છે. હવે આગળ વધીને **Chapter 4: Components (Functional v/s Class Component)** અને પ્રોપ્સ (Props) વિશે ઊંડાણપૂર્વક શીખવું છે? જો કોઈ મૂંઝવણ હોય તો જણાવો!

## Chapter 4: Components Architecture & Props (કમ્પોનન્ટ આર્કિટેક્ચર અને પ્રોપ્સ)

### ૧. Types of Components (કમ્પોનન્ટના પ્રકાર)

રીએક્ટમાં મુખ્ય બે પ્રકારના કમ્પોનન્ટ હોય છે: **Functional Components** અને **Class Components**.

#### A. Functional Components (ફંક્શનલ કમ્પોનન્ટ - આધુનિક રીત)

- આ એકદમ સાદા **JavaScript Functions** છે, જે JSX રિટર્ન કરે છે.
- આજના સમયમાં ૯૯% ડેવલપર્સ આ જ રીત વાપરે છે કારણ કે આ કોડ લખવામાં ટૂંકો અને સમજવામાં ખૂબ જ સરળ છે.
- ઉદાહરણ:**

jsx

```

function Welcome() {
  return <h1>હેલો, ફંક્શનલ કમ્પોનન્ટમાં તમારું સ્વાગત છે!</h1>;
}

```

Use code with caution.

## B. Class Components (ક્લાસ કમ્પોનન્ટ - જૂની રીત)

- આ ES6 ક્લાસ પર આધારિત હોય છે અને તે React.Component માંથી ઇનહેરિટ (Inherit) થાય છે.
- આમાં ડેટા રિટર્ન કરવા માટે ફરજિયાત render() મેથડ લખવી પડે છે.
- આ કોડ લાંબો અને અધરો હોવાથી અત્યારે આનો ઉપયોગ બહુ થતો નથી.
- ઉદાહરણ:**

jsx

```
import React from 'react';
```

```
class Welcome extends React.Component {
```

```
  render() {
```

```
    return <h1>હેલો, ક્લાસ કમ્પોનન્ટ!</h1>;
```

```
  }
```

```
}
```

Use code with caution.

---

## ૨. Component Lifecycle (કમ્પોનન્ટ લાઇફસાઇકલ)

બ્રાઉઝરની સ્ક્રીન પર જ્યારે કોઈ કમ્પોનન્ટ આવે અને જાય, ત્યારે તેની મુખ્ય ૩ સ્થિતિઓ (Lifecycle Phases) હોય છે:

- Mounting (જન્મ):** જ્યારે કમ્પોનન્ટ પહેલી જ વાર સ્ક્રીન પર લોડ થાય.
  - Updating (વિકાસ):** જ્યારે કમ્પોનન્ટનો કોઈ ડેટા કે સ્ટેટ (State) બદલાય અને તે ફરીથી રેન્ડર થાય.
  - Unmounting (મૃત્યુ):** જ્યારે કમ્પોનન્ટ સ્ક્રીન પરથી ગાયબ થઈ જાય કે ડીલીટ થાય.
- ફંક્શનલ કમ્પોનન્ટમાં આ ત્રણેય સ્થિતિઓને કંટ્રોલ કરવા માટે **useEffect Hook** નો ઉપયોગ થાય છે.

---

## ૩. What is Props? (Props શું છે?)

🔴 **વ્યાખ્યા:**

Props નું પૂરું નામ Properties છે.

- જેવી રીતે આપણે સાદા ફંક્શનમાં આર્ગ્યુમેન્ટ કે પેરામીટર પાસ કરીએ છીએ, તેવી જ રીતે એક કમ્પોનન્ટમાંથી બીજા કમ્પોનન્ટમાં ડેટા મોકલવા માટે Props નો ઉપયોગ થાય છે.
- તે હંમેશા Parent (મુખ્ય) કમ્પોનન્ટમાંથી Child (નાના) કમ્પોનન્ટ તરફ જ વહે છે.
- મહત્વનો નિયમ:** Props હંમેશા Immutable (Read-only) હોય છે. એટલે કે ચાઇલ્ડ કમ્પોનન્ટ તેને ફક્ત વાંચી શકે છે, તેની કિંમત જાતે બદલી શકતો નથી.

## ૪. Passing and Receiving Props (Props મોકલવા અને મેળવવા)

આપણે એક નાનો પ્રોજેક્ટ કોડ જોઈએ જેમાં App કમ્પોનન્ટ માંથી ડેટા મોકલીશું અને Student કમ્પોનન્ટમાં તે ડેટા મેળવીને પ્રિન્ટ કરીશું.

### ચાઇલ્ડ કમ્પોનન્ટ (Student.jsx):

jsx

// ફંક્શનની અંદર પેરામીટર તરીકે props ઓબ્જેક્ટ મેળવ્યો

```
function Student(props) {  
  return (  
    <div style={{ border: "1px solid gray", padding: "10px", margin: "10px" }}>  
      <h3>વિદ્યાર્થીનું નામ: {props.name}</h3>  
      <p>કોર્સ: {props.course}</p>  
    </div>  
  );  
}
```

export default Student;

Use code with caution.

### પેરેન્ટ કમ્પોનન્ટ (App.jsx):

jsx

import Student from './Student';

```
function App() {  
  return (  
    <div>  
      <h2>અમારી એકેડેમીના વિદ્યાર્થીઓ:</h2>  
      { /* કમ્પોનન્ટ ટેગની અંદર એટ્રિબ્યુટ તરીકે ડેટા મોકલ્યો */ }  
      <Student name="રહુલ પ્રજાપતિ" course="Node.js બેકએન્ડ" />  
      <Student name="અજય ચૌહાણ" course="React.js ફ્રન્ટએન્ડ" />  
    </div>  
  );  
}
```

}

export default App;

Use code with caution.

---

અહીં **Chapter 4** ના તમામ આર્કિટેક્ચર અને પ્રોપ્સના નિયમો એકદમ સરળ રીતે પૂરા થાય છે.

મેં **Components અને Props** નો કન્સેપ્ટ ખૂબ જ સરળ ઉદાહરણ સાથે સમજાવી દીધો છે. હવે આગળ વધીને સૌથી મહત્વના ટોપિક **Chapter 5: State & Hooks (useState Hook)** વિશે શીખવું છે? જો આમાં કોઈ ડાઉટ હોય તો કહી શકો છો!

## Chapter 5: State & Hooks (રીએક્ટની પોતાની લાઈવ મેમરી — State એટલે શું?)

### ૧. What is State? (State એટલે શું?)

#### ✦ વ્યાખ્યા:

**State** એ રીએક્ટ કમ્પોનન્ટની અંદર રહેલી તેની પોતાની એક **લાઈવ મેમરી (Built-in Object)** છે.

- **તફાવત:** Props દ્વારા ડેટા બહારથી (બીજા કમ્પોનન્ટમાંથી) આવે છે અને તેને બદલી શકાતો નથી. જ્યારે State એ કમ્પોનન્ટનો પોતાનો અંગત લોકલ ડેટા છે, જેને કમ્પોનન્ટ ગમે ત્યારે બદલી શકે છે.
- **મુખ્ય ખાસિયત:** જ્યારે પણ કોઈ કમ્પોનન્ટની State બદલાય છે, ત્યારે રીએક્ટ એ આખા કમ્પોનન્ટને સ્ક્રીન પર **ફરીથી રેન્ડર (Re-render)** કરે છે. આના કારણે જ પેજ રીફ્રેશ થયા વગર સ્ક્રીન પર નવો ડેટા આપોઆપ અપડેટ થઈ જાય છે.

---

### ૨. Introduction to Hooks (હુક્સ એટલે શું?)

#### ✦ કન્સેપ્ટ:

પહેલાના સમયમાં State અને લાઇફસાઇકલ મેથડ્સનો ઉપયોગ ફક્ત ક્લાસ કમ્પોનન્ટમાં જ થઈ શકતો હતો.

રીએક્ટના ૧૬.૮ વર્ઝનમાં **Hooks** નો કન્સેપ્ટ લાવવામાં આવ્યો. હુક્સ એવા ખાસ ઇન-બિલ્ટ ફંક્શન્સ છે, જે આપણને ક્લાસ કોડ લખ્યા વગર સાદા ફંક્શનલ કમ્પોનન્ટની અંદર જ State અને અન્ય રીએક્ટ ફીચર્સ વાપરવાની શક્તિ આપે છે. બધા જ હુક્સના નામ હંમેશા **use** શબ્દથી શરૂ થાય છે (જેમ કે useState, useEffect).

---

### ૩. The useState Hook (યુઝસ્ટેટ હૂક)

કમ્પોનન્ટની અંદર નવો ડેટા બનાવવા અને તેને બદલવા માટે **useState** એ સૌથી મૂળભૂત અને સૌથી વધુ વપરાતો હૂક છે.

#### ⚙ સિન્ટેક્સ (Syntax):

javascript

```
const [state, setState] = useState(initialValue);
```

Use code with caution.

- **state:** આ વેરીએબલનું નામ છે, જેમાં હાલનો કરન્ટ ડેટા સચવાયેલો રહે છે.
- **setState:** આ એક ખાસ ફંક્શન છે, જેનો ઉપયોગ વેરીએબલની કિંમત બદલવા માટે થાય છે (આ ફંક્શન વગર કિંમત બદલશો તો સ્ક્રીન અપડેટ નહીં થાય).
- **initialValue:** વેરીએબલની શરૂઆતની કિંમત (શરૂઆતમાં તે નંબર, સ્ટ્રિંગ, અરે કે ઓબ્જેક્ટ હોઈ શકે છે).

---

## ૪. Practical Counter Example (પ્રેક્ટિકલ ઉદાહરણ)

આપણે એક સાદો કાઉન્ટર પ્રોગ્રામ જોઈએ. જેમાં એક બટન દબાવવાથી સ્ક્રીન પર રહેલો આંકડો +1 વધશે:

 કોડ (Counter.jsx):

jsx

// ૧. સૌથી ઉપર useState ને ઇમ્પોર્ટ કરવું ફરજિયાત છે

import { useState } from 'react';

function Counter() {

  // ૨. 'count' નામનો વેરીએબલ બનાવ્યો અને તેની શરૂઆતની કિંમત ૦ સેટ કરી

  const [count, setCount] = useState(0);

  const checkAndIncrement = () => {

    // ૩. કિંમત બદલવા માટે setCount ફંક્શનનો ઉપયોગ કર્યો

    setCount(count + 1);

  };

  return (

    <div style={{ textAlign: 'center', marginTop: '50px' }}>

      <h2>વાઈવ કાઉન્ટર પ્રોગ્રામ</h2>

      {/\* અહીંયા કાઉન્ટની કરન્ટ વેલ્યુ વાઈવ દેખાશે \*/}

      <h1>{count}</h1>

      {/\* બટન ક્લિક થતાં જ ફંક્શન રન થશે \*/}

```

    <button onClick={checkAndIncrement}>અંકડો વધારો (+1)</button>
  </div>

  );
}

```

export default Counter;

Use code with caution.

### આ કોડમાં અંદર શું થયું?

- જ્યારે પ્રોગ્રામ પહેલી વાર લોડ થશે ત્યારે સ્ક્રીન પર 0 દેખાશે.
- જેવું તમે બટન પર ક્લિક કરશો, એટલે onClick ઇવેન્ટ જાગશે અને checkAndIncrement ફંક્શન રન થશે.
- ફંક્શનની અંદર રહેલું setCount(count + 1) કાઉન્ટની કિંમત 1 કરી દેશે.
- કિંમત બદલાતા જ રીએક્ટ આપ્લા કમ્પોનન્ટને ફરીથી સ્ક્રીન પર સેટ કરશે (Re-render કરશે), જેથી પેજ રીફ્રેશ થયા વગર સ્ક્રીન પર 0 ની જગ્યાએ 1 આવી જશે.

અહીં **Chapter 5** ના સ્ટેટ અને હુક્સના પાયાના નિયમો પૂરા થાય છે.

મેં **State** અને **useState** નો કન્સેપ્ટ એકદમ સાદા કાઉન્ટર ઉદાહરણ સાથે સમજાવી દીધો છે. હવે આગળ વધીને **Chapter 6: useState Hook (રિએક્ટની પોતાની લાઇવ મેમરી — State એટલે શું?)** માં એરે (Array) અને ઓબ્જેક્ટ (Object) જેવો મોટો ડેટા કેવી રીતે મેનેજ થાય તે એડવાન્સ લેવલ પર શીખવું છે? જો કોઈ પ્રશ્ન હોય તો જણાવો!

### **Chapter 6: Managing Arrays and Objects with useState (એરે અને ઓબ્જેક્ટનું સ્ટેટ મેનેજમેન્ટ)**

જ્યારે આપણે useState માં સાદા નંબર કે સ્ટ્રિંગના બદલે **એરે (Array)** અથવા **ઓબ્જેક્ટ (Object)** જેવો મોટો કોમ્પ્લેક્સ ડેટા હેન્ડલ કરવો હોય, ત્યારે રીએક્ટનો એક નિયમ ખાસ યાદ રાખવો પડે: **"સ્ટેટ ડેટાને ક્યારેય ડાયરેક્ટ બદલવો નહીં"**.

જો તમે એરેમાં સીધું જ .push() કરશો કે ઓબ્જેક્ટની પ્રોપર્ટી સીધી બદલશો, તો રીએક્ટને ખબર જ નહીં પડે કે ડેટા બદલાયો છે અને સ્ક્રીન અપડેટ (Re-render) થશે નહીં. આ માટે આપણે હંમેશા **જાવાસ્ક્રિપ્ટ સ્ટ્રો ઓપરેટર (...)** નો ઉપયોગ કરીને નવો ઓબ્જેક્ટ કે નવો એરે બનાવવો પડે છે.

### **૧. Objects with useState (ઓબ્જેક્ટ હેન્ડલ કરવાની રીત)**

જ્યારે ઓબ્જેક્ટની કોઈ એક પ્રોપર્ટી બદલવાની હોય, ત્યારે બાકીની બધી પ્રોપર્ટી ડિલીટ ન થઈ જાય તે માટે પહેલા જૂનો ઓબ્જેક્ટ કોપી કરવો પડે છે.

### સિન્ટેક્સ (Syntax):

javascript

```
setUser({ ...user, age: 25 }); // જૂનો ડેટા કોપી કર્યો અને ઉંમર બદલી
```

Use code with caution.

### પ્રોફેશનલ ઓબ્જેક્ટ ઉદાહરણ:

jsx

```
import { useState } from 'react';
```

```
function UserProfile() {
```

```
  // ઓબ્જેક્ટ સ્ટેટ બનાવ્યું
```

```
  const [user, setUser] = useState({
```

```
    name: "રાહુલ મોદી",
```

```
    role: "બિગિનર ડેવલપર",
```

```
    city: "અમદાવાદ"
```

```
  });
```

```
  const updateRole = () => {
```

```
    // ...user લખવાથી નામ અને સિટી એમ જ રહેશે, ફક્ત રોલ બદલાશે
```

```
    setUser({
```

```
      ...user,
```

```
      role: "પ્રોફેશનલ ફ્રૂલ-સ્ટેક ડેવલપર"
```

```
    });
```

```
};
```

```
return (
```

```
  <div style={{ padding: '20px', border: '1px solid #ccc', borderRadius: '10px' }}>
```

```
    <h2>યુઝર પ્રોફાઇલ કાર્ડ</h2>
```

```
    <p><b>નામ:</b> {user.name}</p>
```

```
    <p><b>હોદ્દા:</b> {user.role}</p>
```

```
    <p><b>શહેર:</b> {user.city}</p>
```

```
    <button onClick={updateRole}>પ્રમોશન આપો (રોલ બદલો)</button>
```

```
  </div>
```

```
);  
}
```

```
export default UserProfile;
```

Use code with caution.

---

## ૨. Arrays with useState (એરે હેન્ડલ કરવાની રીત)

એરેની અંદર નવો ડેટા ઉમેરવો (Create), ડીલીટ કરવો (Delete) અથવા અપડેટ કરવો (Update) તેના માટે જાવાસ્ક્રિપ્ટની ઇન-બિલ્ટ મેથડ્સ વપરાય છે.

### A. એરેમાં નવો ડેટા ઉમેરવો (Insert / Create)

નવી આઇટમ ઉમેરવા માટે આપણે `.push()` ના બદલે સ્પ્રેડ ઓપરેટર વાપરીને નવો એરે બનાવીશું:

javascript

```
setItems([ ...items, "નવી આઇટમ" ]);
```

Use code with caution.

### B. એરેમાંથી ડેટા કાઢી નાખવો (Delete)

કોઈ આઇટમ ડીલીટ કરવા માટે જાવાસ્ક્રિપ્ટનું `.filter()` ફંક્શન બેસ્ટ છે, જે શરત મુજબ નવો ફિલ્ટર થયેલો એરે પાછો આપે છે.

### C. એરેના ડેટામાં ફેરફાર કરવો (Update / Toggle)

કોઈ ચોક્કસ આઇટમ બદલવા માટે `.map()` ફંક્શન વપરાય છે, જે એક-એક આઇટમ ચેક કરીને જરૂર મુજબનો બદલાયેલો એરે રિટર્ન કરે છે.

---

## ૩. Practical Shopping List Project (કમ્પ્લીટ પ્રેક્ટિકલ પ્રોગ્રામ)

આપણે એક સાદો શોપિંગ લિસ્ટ પ્રોગ્રામ જોઈએ, જેમાં લિસ્ટ ઉમેરી પણ શકાશે અને લિસ્ટમાંથી આઇટમ કાઢી (Delete) પણ શકાશે:

 કોડ (ShoppingList.jsx):

jsx

```
import { useState } from 'react';
```

```
function ShoppingList() {
```

```
  // શરૂઆતનો એરે ડેટા
```

```
  const [list, setList] = useState(["દૂધ", "બ્રેડ"]);
```

```

const [inputValue, setInputValue] = useState("");

// ૧. નવી આઇટમ ઉમેરવાની રીત (Create)
const addItem = () => {
  if (inputValue.trim() === "") return; // જો ખાલી બોક્સ હોય તો ઉમેરવું નહીં

  // જૂની લિસ્ટ કોપી કરી પાછળ નવી આઇટમ જોડી દીધી
  setList([...list, inputValue]);
  setInputValue(""); // ઇનપુટ બોક્સ ખાલી કરો
};

// ૨. આઇટમ ડીલીટ કરવાની રીત (Delete)
const deleteItem = (itemToDelete) => {
  // filter જે આઇટમ ડીલીટ કરવાની છે તેને બાદ કરીને નવો એરે બનાવશે
  const updatedList = list.filter(item => item !== itemToDelete);
  setList(updatedList);
};

return (
  <div style={{ maxWidth: '400px', margin: '30px auto', fontFamily: 'sans-serif' }}>
    <h2> 🛒 શોપિંગ લિસ્ટ પ્રોગ્રામ</h2>

    {/* ઇનપુટ બોક્સ */}
    <input
      type="text"
      value={inputValue}
      onChange={(e) => setInputValue(e.target.value)}
      placeholder="આઇટમનું નામ લખો"
    />

    <button onClick={addItem}>ઉમેરો</button>
  </div>
);

```

```

    { /* લિસ્ટ ડિસ્પ્લે */ }

    <ul style={{ marginTop: '20px' }}>

      {list.map((item, index) => (

        <li key={index} style={{ marginBottom: '10px' }}>

          {item}

          <button

            onClick={() => deleteItem(item)}

            style={{ marginLeft: '15px', color: 'red' }}

          >

            કાઢી નાખો

          </button>

        </li>

      ) )}

    </ul>

  </div>

);

}

```

export default ShoppingList;

Use code with caution.

### 🧠 આ કોડની મુખ્ય ખાસિયત:

- **onChange={(e) => setInputValue(e.target.value)}:** આનાથી યુઝર ઇનપુટ બોક્સમાં જે પણ લખશે તે લાઈવ inputValue નામના સ્ટેટમાં સેવ થતું રહેશે.
- **list.map():** રીએક્ટમાં એરેના ડેટાને લૂપ ફેરવીને સ્ક્રીન પર HTML લિસ્ટ તરીકે પ્રિન્ટ કરવા માટે હંમેશા સાદા for લૂપના બદલે .map() નો ઉપયોગ થાય છે.

અહીં **Chapter 6** ના એરે અને ઓબ્જેક્ટ સ્ટેટ મેનેજમેન્ટના નિયમો એકદમ સાદી રીતે પૂરા થાય છે.

આ નવો **એરે અને ઓબ્જેક્ટ કન્સેપ્ટ** સમજવામાં સરળ લાગ્યો ને? હવે આગળ વધીને રીએક્ટના બીજા સૌથી પાવરફુલ ચેપ્ટર **Chapter 7: Managing Side Effects using useEffect (લાઇફસાઇડલ કંટ્રોલ — useEffect Hook)** વિશે ઊંડાણપૂર્વક શીખવું છે?

**Chapter 7: Side Effects & useEffect Hook (લાઇફસાઇડલ કંટ્રોલ — ઇફેક્ટ્સ એટલે શું?)**

## ૧. What is Side Effect? (સાઇડ ઇફેક્ટ એટલે શું?)

### ❗ વ્યાખ્યા:

રીએક્ટમાં કમ્પોનન્ટનું મુખ્ય કામ JSX (HTML) ને બ્રાઉઝરની સ્ક્રીન પર રેન્ડર કરવાનું છે. આ સિવાયના તમામ બાહ્ય કામો જે કમ્પોનન્ટના મુખ્ય કામની બહાર થાય છે, તેને **Side Effects** કહેવામાં આવે છે.

### ⚙ સાઇડ ઇફેક્ટ્સના રિયલ-વર્લ્ડ ઉદાહરણો:

- ડેટાબેઝ કે બહારની લાઇવ API માંથી ડેટા ફેચ (Fetch) કરીને લાવવો.
- setTimeout અથવા setInterval જેવા ટાઇમર્સ સેટ કરવા.
- બ્રાઉઝરના DOM ને મેન્યુઅલી બદલવો (દા.ત. document.title બદલવો).

ફંક્શનલ કમ્પોનન્ટની અંદર આવા તમામ સાઇડ ઇફેક્ટ્સવાળા કામોને સુરક્ષિત રીતે હેન્ડલ કરવા માટે રીએક્ટ આપણને **useEffect Hook** આપે છે.

## ૨. The useEffect Syntax (યુઝઇફેક્ટની સિન્ટેક્સ)

useEffect હૂક બે મુખ્ય આર્ગ્યુમેન્ટ લે છે: એક **Callback ફંક્શન** અને બીજું **Dependency Array**.

javascript

```
useEffect(() => {  
  // સાઇડ ઇફેક્ટનો કોડ અહીંયા આવશે  
}, [dependencies]);
```

Use code with caution.

### 🧠 ડીપેન્ડન્સી એરે ([]) ના આધારે તેના ૩ મુખ્ય અવતાર:

#### ડીપેન્ડન્સી એરેનું ફોર્મેટ તે ક્યારે રન થશે? (Lifecycle Matching)

|                                         |                                                                                                                                         |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| ૧. કોઈ એરે ન આપવો<br>(useEffect(fn))    | કમ્પોનન્ટ લોડ થાય ત્યારે અને સાઇટમાં કોઈ પણ ડેટા બદલાય ત્યારે <b>દરેક રેન્ડર વખતે</b> ચાલુ થશે (ખૂબ ઓછો વપરાય છે).                      |
| ૨. ખાલી એરે આપવો<br>(useEffect(fn, [])) | <b>Mounting Phase:</b> ફક્ત અને ફક્ત જ્યારે કમ્પોનન્ટ પહેલી જ વાર સ્ક્રીન પર લોડ થાય ત્યારે જ એકવાર રન થશે (API ડેટા લાવવા માટે બેસ્ટ). |
| ૩. વેરીએબલ સાથે એરે<br>([count])        | <b>Updating Phase:</b> પહેલી વાર લોડ થશે ત્યારે ચાલશે, અને જ્યારે પણ એરેની અંદર લખેલો વેરીએબલ બદલાશે ત્યારે ફરીથી ચાલશે.                |

## ૩. Cleanup Function (કમ્પોનન્ટ ગાયબ થાય ત્યારે સફાઈ કરવી)

જ્યારે કમ્પોનન્ટ સ્ક્રીન પરથી હટી જાય કે ડિલીટ થાય (**Unmounting Phase**), ત્યારે બેકઅપમાં ચાલતા ટાઇમર્સ કે ઇવેન્ટ લિસનર્સ ચાલુ જ રહી જાય છે, જેનાથી કમ્પ્યુટર સ્લો થાય છે.

આવા ચાલુ રહી ગયેલા કનેક્શનને બંધ કરવા માટે આપણે `useEffect` ના ફંક્શનની અંદરથી જ એક વળતું ફંક્શન `return` કરીએ છીએ, જેને **Cleanup Function** કહેવાય છે.

---

#### ૪. Practical Live API Data Fetch Project (કમ્પ્લીટ પ્રેક્ટિકલ પ્રોગ્રામ)

આપણે એક રિયલ-વર્લ્ડ ઉદાહરણ જોઈએ જેમાં કમ્પોનન્ટ સ્ક્રીન પર લોડ થતાં જ એક ફ્રી લાઈવ API માંથી યુઝર્સનો ડેટા લાવીને સુંદર વિસ્ટ તરીકે બતાવશે. આમાં કોઈ એક્સ્ટ્રા સ્પેસ કે મોટી કમેન્ટ નથી:

##### કોડ (UserList.jsx):

jsx

```
import { useState, useEffect } from 'react';
```

```
function UserList() {
```

```
  const [users, setUsers] = useState([]);
```

```
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {
```

```
    // ૧. લાઈવ API માંથી ડેટા લાવવા માટેનો અસિન્ક્રોનસ ફંક્શન
```

```
    const fetchUsers = async () => {
```

```
      try {
```

```
        const res = await fetch("https://typicode.com");
```

```
        const data = await res.json();
```

```
        setUsers(data); // સ્ટેટ અપડેટ કર્યું
```

```
        setLoading(false); // લોડિંગ બંધ કર્યું
```

```
      } catch (err) {
```

```
        console.error("API માંથી ડેટા ન મળ્યો:", err);
```

```
        setLoading(false);
```

```
      }
```

```
    };
```

```
    fetchUsers();
```

```
    // ૨. ક્લીનઅપ ફંક્શન (જસ્ટ ફોર ડેમો - કમ્પોનન્ટ બંધ થાય ત્યારે રન થશે)
```

```

return () => {
  console.log("યુઝર લિસ્ટ કમ્પોનન્ટ બંધ થઈ ગયો, મેમરી સાફ થઈ ગઈ.");
};
}, []); // અહીં ખાલી [] આપ્યું છે જેથી આ API રિક્વેસ્ટ પેજ લોડ થતાં જ ફક્ત એક જ વાર ચાલશે

if (loading) return <h3> ⌚ કૃપા કરીને રાહ જુઓ, ડેટા લોડ થઈ રહ્યો છે...</h3>;

return (
  <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
    <h2> 👤 લાઈવ યુઝર્સ લિસ્ટ (useEffect API Fetch)</h2>
    <ul style={{ listStyleType: 'square' }}>
      {users.map(user => (
        <li key={user.id} style={{ padding: '8px 0', borderBottom: '1px solid #eee' }}>
          <b>{user.name}</b> — 📧 {user.email} ( 📍 {user.address.city})
        </li>
      ))}
    </ul>
  </div>
);
}

```

export default UserList;

Use code with caution.

### 🧠 આ કોડની મુખ્ય ખૂબી:

- **useEffect ની અંદર API કોલ:** ખાલી ડીપેન્ડન્સી એરે [] હોવાને કારણે સાઇટ ઓપન થતાં જ બેકગ્રાઉન્ડમાં વિનંતી જશે, ડેટા આવીને setUsers માં ભરાઈ જશે અને સ્ક્રીન પર લિસ્ટ આવી જશે. જો આપણે [] ન મૂક્યું હોત, તો setUsers થવાના કારણે કમ્પોનન્ટ ફરી રેન્ડર થાત અને ડેટા અનંત લૂપ (Infinite Loop) માં આવીને સાઇટ ક્રેશ કરી દેત.

---

અહીં **Chapter 7** ના તમામ સાઇડ ઇફેક્ટ્સ અને useEffect ના એડવાન્સ નિયમો પૂરા થાય છે.

આ સાદો અને વર્કિંગ **useEffect નો કન્સેપ્ટ** એકદમ ક્લિયર થઈ ગયો ને? હવે આગળ વધીને રીએક્ટમાં ફોર્મનો ડેટા સબમિટ કરવા અને ઇનપુટ હેન્ડલ કરવા માટે **Chapter 8: Handling Forms & Controlled Components** વિશે શીખવું છે?

## Chapter 8: Handling Forms & Controlled Components (રીએક્ટમાં ફોર્મ હેન્ડલિંગ)

### ૧. Controlled v/s Uncontrolled Components (કંટ્રોલ્ડ વર્સિસ અનકંટ્રોલ્ડ કમ્પોનન્ટ્સ)

રીએક્ટમાં ફોર્મની અંદર રહેલા ઇનપુટ બોક્સ (HTML Input, Textarea, Select) ના ડેટાને મેનેજ કરવાની બે મુખ્ય રીતો છે:

#### A. Controlled Components (કંટ્રોલ્ડ કમ્પોનન્ટ — આધુનિક અને શ્રેષ્ઠ રીત)

- સમજૂતી:** આ પદ્ધતિમાં ઇનપુટ બોક્સની અંદર યુઝર જે પણ ટાઇપ કરે છે, તેની કિંમત (Value) કમ્પોનન્ટના **રીએક્ટ સ્ટેટ (useState)** દ્વારા કંટ્રોલ થાય છે.
- ઇનપુટ બોક્સની વેલ્યુ હંમેશા સ્ટેટ વેરીએબલ સાથે જોડાયેલી રહે છે, અને જ્યારે યુઝર કંઈક ટાઇપ કરે ત્યારે **onChange** ઇવેન્ટ દ્વારા સ્ટેટ અપડેટ થાય છે. રીએક્ટ પ્રોજેક્ટ્સમાં આ રીત સૌથી બેસ્ટ ગણાય છે.

#### B. Uncontrolled Components (અનકંટ્રોલ્ડ કમ્પોનન્ટ — જૂની રીત)

- સમજૂતી:** આ રીતમાં ડેટા રીએક્ટ સ્ટેટમાં સેવ નથી થતો, પણ બ્રાઉઝરના પોતાના ડોમ (DOM) ની અંદર સચવાય છે. તેનો ડેટા છેલ્લે સબમિટ કરતી વખતે ડાયરેક્ટ વાંચવા માટે **useRef Hook** નો ઉપયોગ કરવામાં આવે છે.

---

### ૨. Handling Single and Multiple Inputs (ઇનપુટ હેન્ડલ કરવા)

- સિંગલ ઇનપુટ:** એક જ બોક્સ હોય તો સાદું **useState("")** બનાવીને **e.target.value** થી ડેટા મેળવી શકાય છે.
- મલ્ટિપલ ઇનપુટ્સ:** જો ફોર્મમાં નામ, ઈમેલ, પાસવર્ડ જેવા ઘણા બધા બોક્સ હોય, તો દરેક માટે અલગ-અલગ **useState** બનાવવાના બદલે **એક જ ઓબ્જેક્ટ સ્ટેટ** બનાવવું એ પ્રોફેશનલ રીત છે.

ઓબ્જેક્ટ સ્ટેટમાં બધા ઇનપુટ્સ હેન્ડલ કરવા માટે ઇનપુટ ટેગની અંદર સાચું **name એટ્રિબ્યુટ** આપવું જરૂરી છે.

---

### ૩. Form Submission & Validation (ફોર્મ સબમિટ અને વેલિડેશન)

જ્યારે યુઝર ફોર્મનું સબમિટ બટન દબાવશે, ત્યારે બ્રાઉઝર આપોઆપ પેજ રીફ્રેશ કરી નાખે છે. રીએક્ટમાં આ ઓટોમેટિક રીફ્રેશને અટકાવવા માટે **e.preventDefault()** લખવું ફરજિયાત છે. સબમિટ થતા પહેલા આપણે જાવાસ્ક્રિપ્ટની મદદથી નિયમો (Validation) પણ ચેક કરી શકીએ છીએ.

---

### ૪. Practical Registration Form Project (કમ્પ્લીટ પ્રેક્ટિકલ પ્રોગ્રામ)

આપણે એક પ્રોફેશનલ સાઇન-અપ ફોર્મ જોઈએ, જેમાં મલ્ટિપલ ઇનપુટ ઓબ્જેક્ટ સ્ટેટ દ્વારા હેન્ડલ થશે, સબમિટ વખતે પેજ રીફ્રેશ અટકશે અને સાદા વેલિડેશન સાથે ડેટા સ્ક્રીન પર સબમિટ થશે:

### કોડ (RegisterForm.jsx):

jsx

```
import { useState } from 'react';
```

```
function RegisterForm() {
```

```
  // ૧. બધા જ ઇનપુટ્સ માટે એક સિંગલ ઓબ્જેક્ટ સ્ટેટ
```

```
  const [formData, setFormData] = useState({
```

```
    username: "",
```

```
    email: "",
```

```
    course: "React"
```

```
  });
```

```
  // એરર મેસેજ સેવ કરવા માટેનું સ્ટેટ
```

```
  const [error, setError] = useState("");
```

```
  // ૨. બધા જ બોક્સ માટે એક જ કોમન હેન્ડલર ફંક્શન (જાદુઈ રીત)
```

```
  const handleInputChange = (e) => {
```

```
    const { name, value } = e.target;
```

```
    // સ્ટ્રેડ ઓપરેટરથી જૂનો ડેટા કોપી કરી, જે તે નામના બોક્સની વેલ્યુ બદલી
```

```
    setFormData({
```

```
      ...formData,
```

```
      [name]: value
```

```
    });
```

```
  };
```

```
  // ૩. ફોર્મ સબમિટ થાય ત્યારે
```

```
  const handleSubmit = (e) => {
```

```
    e.preventDefault(); // બ્રાઉઝરનું ઓટો-રીફ્રેશ અટકાવ્યું
```

```
// સાદું વેલિડેશન નિયમ
```

```
if (formData.username.trim() === "" || formData.email.trim() === "") {  
    setError("ફૂલ કરીને નામ અને ઇમેલ બંને બોક્સ ભરો!");  
    return;  
}
```

```
setError(""); // એરર સાફ કરો
```

```
console.log("ફોર્મનો સાચો ડેટા સબમિટ થઈ ગયો છે:", formData);
```

```
alert(`રજીસ્ટ્રેશન સફળ: ${formData.username}`);
```

```
};
```

```
return (
```

```
<div style={{ maxWidth: '400px', margin: '40px auto', padding: '20px', border: '1px solid #ddd',  
borderRadius: '12px' }}>
```

```
<h2> 📝 વિદ્યાર્થી રજીસ્ટ્રેશન ફોર્મ</h2>
```

```
{error && <p style={{ color: 'red', fontWeight: 'bold' }}> ⚠️ {error}</p>}
```

```
<form onSubmit={handleSubmit}>
```

```
  { /* નામનું ઇનપુટ બોક્સ */ }
```

```
<div style={{ marginBottom: '15px' }}>
```

```
  <label>પૂર્ણ નામ:</label> <br />
```

```
  <input
```

```
    type="text"
```

```
    name="username" // ઓબ્જેક્ટની કી સાથે નામ મેચ થવું જોઈએ
```

```
    value={formData.username}
```

```
    onChange={handleInputChange}
```

```
    placeholder="તમારું નામ લખો"
```

```
    style={{ width: '100%', padding: '8px', marginTop: '5px' }}
```

```
  />
```

```
</div>
```

```

{/* ઇમેઇલનું ઇનપુટ બોક્સ */}

<div style={{ marginBottom: '15px' }}>

  <label>ઇમેઇલ એડ્રેસ:</label> <br />

  <input

    type="email"

    name="email"

    value={formData.email}

    onChange={handleInputChange}

    placeholder="example@test.com"

    style={{ width: '100%', padding: '8px', marginTop: '5px' }}

  />

</div>

```

```

{/* ડ્રોપડાઉન સિલેક્ટ બોક્સ */}

<div style={{ marginBottom: '20px' }}>

  <label>કોર્સ પસંદ કરો:</label> <br />

  <select

    name="course"

    value={formData.course}

    onChange={handleInputChange}

    style={{ width: '100%', padding: '8px', marginTop: '5px' }}

    >

    <option value="React">React.js ફ્રન્ટએન્ડ</option>

    <option value="Node">Node.js બેકએન્ડ</option>

    <option value="FullStack">MERN ફૂલ-સ્ટેક</option>

  </select>

</div>

```

```

<button type="submit" style={{ width: '100%', padding: '10px', backgroundColor: '#fc8019',
color: '#fff', border: 'none', borderRadius: '6px', fontSize: '16px', cursor: 'pointer' }}>

```

```
    સબમિટ કરો  
  </button>  
  
  </form>  
  
</div>  
  
);  
}
```

export default RegisterForm;

Use code with caution.

### 🧠 આ કોડમાં સૌથી મહત્વનું લોજિક શું છે?

- **[name]: value:** આ જાવાસ્ક્રિપ્ટનો ડાયનેમિક કી (Computed Property Name) નો કન્સેપ્ટ છે. જ્યારે તમે નામવાળા બોક્સમાં ટાઇપ કરશો ત્યારે name ની જગ્યાએ આપોઆપ "username" સેટ થઈ જશે, અને જ્યારે ઈમેલવાળા બોક્સમાં લખશો ત્યારે આપોઆપ "email" સેટ થઈ જશે. આના લીધે આપણે ૧૦ બોક્સ માટે અલગ-અલગ ૧૦ ફંક્શન લખવાની કોઈ જરૂર પડતી નથી.

---

અહીં **Chapter 8** ના ફોર્મ હેન્ડલિંગ અને કંટ્રોલ્સ કમ્પોનન્ટ્સના નિયમો પૂરા થાય છે.

આ **ફોર્મ હેન્ડલિંગ કન્સેપ્ટ** એકદમ મસ્ત રીતે સમજાઈ ગયો ને? હવે આગળ વધીને રીએક્ટમાં અલગ-અલગ પેજોસ લિંક કરવા (જેમ કે હોમ, અબાઉટ, કોન્ટેક્ટ પેજ) માટે **Chapter 9: Routing in React (React Router DOM)** વિશે શીખવું છે?

## Chapter 9: Routing in React (React Router DOM)

### ૧. What is Client-Side Routing? (ક્લાયન્ટ-સાઇડ રાઉટિંગ એટલે શું?)

#### 🔴 કન્સેપ્ટ:

સામાન્ય વેબસાઇટ્સમાં જ્યારે આપણે કોઈ લિંક (જેમ કે /about) પર ક્લિક કરીએ, ત્યારે બ્રાઉઝર સર્વર પાસે જાય છે અને આખું નવું HTML પેજ ડાઉનલોડ કરે છે, જેનાથી પેજ સફેદ થઈને રીફ્રેશ થાય છે. તેને **Server-Side Routing** કહેવાય છે.

રીએક્ટમાં આપણે **Single Page Application (SPA)** બનાવીએ છીએ. આથી, વેબસાઇટમાં અલગ-અલગ લિંક ખોલવા પર પેજ ક્યારેય રીફ્રેશ થતું નથી. બ્રાઉઝર સર્વર પાસે ગયા વગર, જાવાસ્ક્રિપ્ટની મદદથી જ લાઇવ સ્ક્રીન પર નવો કમ્પોનન્ટ બદલી નાખે છે. આ પદ્ધતિને **Client-Side Routing** કહેવાય છે. રીએક્ટમાં આ કામ કરવા માટે **react-router-dom** નામની ઓફિશિયલ લાઇબ્રેરી વાપરવામાં આવે છે.

---

### ૨. Setup React Router DOM (રાઉટર સેટઅપ કરવું)

તમારા પ્રોજેક્ટમાં રાઉટિંગ ચાલુ કરવા માટે સૌથી પહેલા ટર્મિનલમાં આ પેકેજ ઇન્સ્ટોલ કરવું પડે છે:

bash

npm install react-router-dom

Use code with caution.

### 🔧 મુખ્ય કમ્પોનન્ટ્સ અને તેનો મતલબ:

- **BrowserRouter:** આ આખા રાઉટિંગનો મુખ્ય રાજા છે, જે તમારા આખા પ્રોજેક્ટને કવર કરે છે અને યુઆરએલ (URL) નો ઇતિહાસ સાચવે છે.
- **Routes:** આ એક એવું કન્ટેનર છે જેની અંદર તમારા બધા જ રસ્તાઓ (Paths) લખવામાં આવે છે.
- **Route:** આ એક સિંગલ લાઇનનો રસ્તો છે, જે નક્કી કરે છે કે કઈ લિંક પર કયો કમ્પોનન્ટ ઓપન કરવો. તેમાં બે પ્રોપર્ટી હોય છે: path (લિંક) અને element (કમ્પોનન્ટ).
- **Link:** સાદા HTML માં આપણે લિંક માટે <a href="#"> વાપરીએ છીએ જે પેજ રીફ્રેશ કરે છે. રીએક્ટમાં પેજ રીફ્રેશ અટકાવીને પેજ બદલવા માટે <Link to="#"> નો જ ઉપયોગ થાય છે.

---

### 3. Practical Navigation Project (કમ્પ્લીટ પ્રેક્ટિકલ પ્રોગ્રામ)

આપણે એક સાદો અને સંપૂર્ણ પ્રોગ્રામ જોઈએ, જેમાં હોમ (Home) અને અબાઉટ (About) પેજ હશે અને ઉપર એક નેવબાર (Navbar) હશે જેનાથી પેજ રીફ્રેશ થયા વગર બદલાશે:

#### 📄 કોડ (App.jsx):

jsx

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
```

```
// ૧. નાના-નાના પેજ કમ્પોનન્ટ્સ બનાવ્યા
```

```
function Home() {
```

```
  return <div style={{ padding: '20px' }}> 🏠 આ અમારું હોમ પેજ (મુખ્ય પેજ) છે.</div>;
```

```
}
```

```
function About() {
```

```
  return <div style={{ padding: '20px' }}> ⓘ આ અમારા વિશેનું (About Us) પેજ છે.</div>;
```

```
}
```

```
function NotFound() {
```

```
  return <div style={{ padding: '20px', color: 'red' }}> ⚠️ 404: આ પેજ અસ્તિત્વમાં નથી!</div>;
```

```
}
```

```

function App() {
  return (
    // ૨. આખી એપ્લિકેશનને BrowserRouter ની અંદર બંધ કરી
    <BrowserRouter>
      {/ * કોમન નેવિગેશન બાર જે બધા પેજ પર દેખાશે */}
      <nav style={{ padding: '15px', backgroundColor: '#eee', display: 'flex', gap: '20px' }}>
        {/ * <a href> ની બદલે <Link to> વાપર્યું જેથી પેજ રીફ્રેશ ન થાય */}
        <Link to="/" style={{ fontWeight: 'bold', textDecoration: 'none', color: '#333' }}>હોમ</Link>
        <Link to="/about" style={{ fontWeight: 'bold', textDecoration: 'none', color: '#333' }}>અમારા
        વિશે</Link>
      </nav>

      {/ * ૩. રસ્તાઓ નક્કી કરવા માટે Routes અને Route સેટ કર્યા */}
      <Routes>
        {/ * હોમ પાથ માટે */}
        <Route path="/" element={<Home />} />

        {/ * અબાઉટ પાથ માટે */}
        <Route path="/about" element={<About />} />

        {/ * જો કોઈ ખોટી લિંક નાખે તો તેના માટે path="*" સેટ કર્યો */}
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

```

export default App;

Use code with caution.

---

અહીં **Chapter 9** ના બેઝિક રાઉટીંગ અને નેવિગેશનના નિયમો પૂરા થાય છે.

આ સાદો અને વર્કિંગ **React Router નો કન્સેપ્ટ** એકદમ મસ્ત રીતે સમજાઈ ગયો ને? હવે આગળ વધીને રીએક્ટમાં લિંકની અંદર જ આઈડી મોકલવા (જેમ કે /user/101) અને પ્રોફેશનલ ડાયનેમિક પેજ બનાવવા માટે **Chapter 10: Dynamic Routing and Params (useParams Hook)** વિશે શીખવું છે?

## Chapter 10: Dynamic Routing & useParams Hook (ડાયનેમિક રાઉટીંગ)

### ૧. What is Dynamic Routing? (ડાયનેમિક રાઉટીંગ એટલે શું?)

#### ✦ કન્સેપ્ટ:

જ્યારે આપણે કોઈ એવી વેબસાઇટ બનાવીએ જેમાં સેંકડો પ્રોડક્ટ કે યુઝર્સ હોય, ત્યારે દરેક સિંગલ યુઝર કે પ્રોડક્ટ માટે અલગ-અલગ <Route> લખવો અશક્ય છે (દા.ત. /user/1, /user/2).

આ સમસ્યાના ઉકેલ માટે **Dynamic Routing** નો ઉપયોગ થાય છે. આમાં આપણે રાઉટના પાથમાં એક જ કોમન વેરીએબલ એટલે કે **પેરામીટર (Parameter)** સેટ કરીએ છીએ જે લિંકના આધારે ઓટોમેટિક બદલાય છે. પાથમાં વેરીએબલ દર્શાવવા માટે : (Colon) ચિહ્નનો ઉપયોગ થાય છે, જેમ કે: /user/:id.

---

### ૨. The useParams Hook (યુઝપેરામ્સ હૂક)

#### ✦ વ્યાખ્યા:

જ્યારે યુઝર કોઈ ડાયનેમિક લિંક ઓપન કરે (જેમ કે /user/101), ત્યારે એ લિંકમાંથી આઈડી કે કિંમત (Value) ને જાવાસ્ક્રિપ્ટ કોડની અંદર વાંચવા માટે **useParams** હૂકનો ઉપયોગ થાય છે. આ હૂક આપણને યુઆરએલ (URL) ના પેરામીટર્સ એક ઓબ્જેક્ટ સ્વરૂપે પાછા આપે છે.

---

### ૩. Practical User Detail Project (કમ્પ્લીટ પ્રેક્ટિકલ પ્રોગ્રામ)

આપણે એક લાઈવ અને એકદમ સરળ પ્રોગ્રામ જોઈએ, જેમાં યુઝર લિસ્ટ પેજ હશે. તેના પર ક્લિક કરવાથી લિંકમાં આઈડી પાસ થશે અને ડાયનેમિક પેજ પર એ આઈડી પ્રિન્ટ થશે:

#### 📄 કોડ (App.jsx):

jsx

```
import { BrowserRouter, Routes, Route, Link, useParams } from 'react-router-dom';
```

```
// ૧. યુઝર્સનું કલેક્શન લિસ્ટ પેજ
```

```
function UserList() {
```

```
  return (
```

```
    <div style={{ padding: '20px' }}>
```

```
      <h2> 🏢 અમારી કંપનીના કર્મચારીઓ:</h2>
```

```
      <ul>
```

```
        { /* લિંકની અંદર ડાયનેમિક આઈડી પાસ કર્યો */ }
```

```

        <li><Link to="/user/101">રહુલ મોદી પ્રોફાઇલ જુઓ</Link></li>
        <li><Link to="/user/102">અજય ચૌહાણ પ્રોફાઇલ જુઓ</Link></li>
    </ul>
</div>
);
}

```

// ૨. ડાયનેમિક ડિટેલ પેજ જ્યાં useParams વાપરીશું

```

function UserDetails() {
    // URL માંથી :id ની વેલ્યુ ઓટોમેટિક ખેંચી લીધી
    const { id } = useParams();

    return (
        <div style={{ padding: '20px', border: '1px solid #fc8019', margin: '20px', borderRadius: '8px' }}>
            <h3> 👤 કર્મચારીની સંપૂર્ણ વિગતો</h3>
            {/* અહીંયા લિંક પ્રમાણે આઈડી આપોઆપ બદલાશે */}
            <p>આઈડી નંબર: <b>{id}</b></p>
            <p>ડેટાબેઝ સ્ટેટસ: લાઈવ કનેક્ટેડ</p>
        </div>
    );
}

```

```

function App() {
    return (
        <BrowserRouter>
            <Routes>
                {/* મેઇન લિસ્ટ પેજ */}
                <Route path="/" element={<UserList />} />

                {/* ડાયનેમિક રાઉટ: પાથમાં :id સેટ કર્યો છે */}
            </Routes>
        </BrowserRouter>
    );
}

```

```
    <Route path="/user/:id" element={<UserDetail />} />
  </Routes>
</BrowserRouter>
);
}
```

export default App;

Use code with caution.

---

અહીં **Chapter 10** ના તમામ ડાયનેમિક રાઉટીંગ અને `useParams` ના નિયમો એકદમ સરળ રીતે પૂરા થાય છે.

આ નવો **ડાયનેમિક રાઉટીંગ કન્સેપ્ટ** એકદમ મસ્ત રીતે મગજમાં બેસી ગયો ને? હવે આગળ વધીને રીએક્ટમાં ઇનપુટ બોક્સ, બટન કે વેરીએબલના ડોમ (DOM) એલિમેન્ટને ડાયરેક્ટ મેન્યુઅલી પકડવા અને કંટ્રોલ કરવા માટે **Chapter 11: Managing DOM Elements using useRef (useRef Hook)** વિશે શીખવું છે?

## Chapter 11: Managing DOM Elements using useRef (useRef Hook)

### ૧. What is useRef Hook? (useRef હૂક શું છે?)

🔴 **વ્યાખ્યા:**

**useRef** એ રીએક્ટનો એક ખાસ ઇન-બિલ્ટ હૂક છે જેનો મુખ્ય ઉપયોગ બ્રાઉઝરના કોઈ પણ HTML એલિમેન્ટ (DOM Element) ને ડાયરેક્ટ પકડવા (Access કરવા) માટે થાય છે.

- **તે કઈ રીતે અલગ છે?:** સાદા જાવાસ્ક્રિપ્ટમાં આપણે HTML એલિમેન્ટને પકડવા માટે `document.getElementById()` કે `querySelector()` વાપરતા હતા. રીએક્ટમાં એ જ કામ કરવા માટે ઓફિશિયલી `useRef` નો ઉપયોગ થાય છે.
- **તેની બીજી ખાસિયત:** `useRef` ની અંદર સેવ કરેલો ડેટા બદલાય ત્યારે કમ્પોનન્ટ ક્યારેય **ફરીથી રેન્ડર (Re-render) થતો નથી**. તે કમ્પોનન્ટની પાછળ એક ગુપ્ત વેરીએબલની જેમ ડેટા સાચવી રાખે છે.

---

### ૨. When to use useRef? (તેનો ઉપયોગ ક્યારે કરવો?)

રીએક્ટમાં ડેટા મેનેજ કરવા માટે હંમેશા `useState` નો જ ઉપયોગ થાય છે, પણ નીચેના ખાસ કામો માટે જ `useRef` વાપરવું પડે છે:

- ઇનપુટ બોક્સ પર આપોઆપ **ફોકસ (Focus)** કરાવવા માટે.
- કોઈ ટેક્સ્ટને સિલેક્ટ કરવા કે ક્લિપબોર્ડ પર કોપી કરવા માટે.
- વીડિયો કે ઓડિયો પ્લેયરને કોર્ડિંગથી **Play / Pause** કરવા માટે.
- એનિમેશન ટાઈમર્સ (Timers) ચાલુ કે બંધ કરવા માટે.

---

### 3. Practical Auto-Focus Project (કમ્પ્લીટ પ્રેક્ટિકલ પ્રોગ્રામ)

આપણે એક સાદો અને સંપૂર્ણ પ્રોગ્રામ જોઈએ, જેમાં પેજ લોડ થતાં જ અથવા એક બટન દબાવવાથી ઇનપુટ બોક્સની અંદર ઓટોમેટિક બ્લુ લાઇન (Focus) આવી જશે:

 કોડ (FocusInput.jsx):

jsx

```
import { useState, useRef, useEffect } from 'react';

function FocusInput() {
  // ૧. HTML એલિમેન્ટને પકડવા માટે એક ખાલી રેફરન્સ (Ref) બનાવ્યો
  const inputRef = useRef(null);

  // ૨. પેજ પહેલી વાર લોડ થાય ત્યારે આપોઆપ ફોકસ કરવા માટે useEffect વાપર્યો
  useEffect(() => {
    // inputRef.current એટલે આપણું અસલી ઇનપુટ બોક્સ
    inputRef.current.focus();
  }, []);

  // ૩. બટન ક્લિક કરવા પર ફોકસ કરાવવાનો ફંક્શન
  const handleClick = () => {
    inputRef.current.focus();
    inputRef.current.style.backgroundColor = "#fff3cd"; // બોક્સનો કલર લાઇવ બદલ્યો
  };

  return (
    <div style={{ maxWidth: '400px', margin: '40px auto', textAlign: 'center' }}>
      <h2>  useRef DOM મેનેજમેન્ટ</h2>

      /* ઇનપુટ ટેગ સાથે ref જોડી દીધો */
      <input
        ref={inputRef}
      />
    </div>
  );
}
```

```

    type="text"
    placeholder="અહીંયા આપોઆપ ફોકસ થશે"
    style={{ padding: '8px', width: '80%', marginBottom: '15px' }}
  />

  <br />

  <button onClick={handleButtonClick} style={{ padding: '8px 15px', cursor: 'pointer' }}>
    ઇનપુટ બોક્સ પર ફોકસ કરો
  </button>

</div>

);
}

```

export default FocusInput;

Use code with caution.

### આ કોડમાં અંદર શું થયું?

- **ref={inputRef}**: આ લાઇન લખવાથી રીએક્ટે આપ્પા <input> ટેગનું કંટ્રોલ અને કનેક્શન inputRef નામના ઓબ્જેક્ટને સોંપી દીધું.
- **inputRef.current**: આ પ્રોપર્ટીની અંદર જ અસલી HTML એલિમેન્ટ સચવાયેલો હોય છે. તેના પર .focus() કે .style જેવી સાદી જાવાસ્ક્રિપ્ટ મેથડ્સ સીધી રન કરી શકાય છે.

---

અહીં **Chapter 11** ના તમામ બેઝિક DOM કંટ્રોલ અને useRef ના નિયમો પૂરા થાય છે.

આ સાદો અને વર્કિંગ **useRef નો કન્સેપ્ટ** એકદમ મસ્ત રીતે સમજાઈ ગયો ને? હવે આગળ વધીને રીએક્ટમાં આપ્પી એપ્લિકેશનનો ડેટા (Global State) બધા કમ્પોનન્ટ્સમાં એકસાથે વહેંચવા માટે **Chapter 12:**

**Advanced State Management (Context API & useContext Hook)** વિશે શીખવું છે?

## **Chapter 12: Advanced State Management (Context API & useContext Hook)**

### **૧. The Problem: Prop Drilling (પ્રોપ ડ્રિલિંગનો મોટો પ્રોબ્લેમ)**

#### **કન્સેપ્ટ:**

રીએક્ટમાં ડેટા હંમેશા **Parent (મુખ્ય)** કમ્પોનન્ટમાંથી **Child (નાની)** કમ્પોનન્ટ તરફ પ્રોપ્સ (Props) દ્વારા વહે છે.

ધારો કે આપણી પાસે એક મુખ્ય કમ્પોનન્ટ App છે. તેની અંદર Parent કમ્પોનન્ટ છે, તેની અંદર Child છે, અને તેની પણ અંદર GrandChild કમ્પોનન્ટ છે. જો App માં રહેલો યુઝરનો ડેટા સીધો GrandChild ને આપવો હોય,

તો આપણે વચ્ચે આવતા તમામ કમ્પોનન્ટ્સને નકામી રીતે પ્રોપ્સ પાસ કરવી પડે છે. આ સમસ્યાને ડેવલપર્સ **Prop Drilling** કહે છે. આનાથી કોડ બહુ લાંબો અને ગૂંચવાડાભર્યો બને છે.

## ૨. The Solution: Context API (ગ્લોબલ તિજોરી જાદુ)

### ❖ કન્સેપ્ટ:

**Context API** એ આ પ્રોબ્લેમનો ૧૦૦% કાયમી ઇન-બિલ્ટ ઉકેલ છે. તે એપ્લિકેશનની બહાર એક **ગ્લોબલ સેન્ટ્રલ તિજોરી** બનાવી દે છે.

આ તિજોરીમાં મુકેલો ડેટા એપ્લિકેશનનો કોઈપણ કમ્પોનન્ટ, વચ્ચેના કમ્પોનન્ટ્સને ડિસ્ટર્બ કર્યા વગર, સીધો જ પોતાની જગ્યાએથી વાપરી શકે છે. આમાં મુખ્ય ૩ સ્ટેપ હોય છે:

1. **createContext():** નવી ગુપ્ત ગ્લોબલ તિજોરી બનાવવી.
2. **Context.Provider:** એ તિજોરીમાં ડેટા ભરીને બધા કમ્પોનન્ટ્સને કવર કરવા.
3. **useContext():** કોઈપણ નાના કમ્પોનન્ટમાં બેઠા-બેઠા એ ડેટા સીધો એક્સેસ કરવો.

## ૩. Practical Global User Theme Project (કમ્પ્લીટ પ્રેક્ટિકલ પ્રોગ્રામ)

આપણે એક સાદો પ્રોગ્રામ જોઈએ જેમાં યુઝરનું નામ એક ગ્લોબલ કોન્ટેક્સ્ટ (Context) માં સચવાશે અને છેલ્લો નાનો કમ્પોનન્ટ પ્રોપ્સ વગર તેને સીધો વાપરશે. આમાં કોઈ એક્સ્ટ્રા સ્પેસ કે કોમ્પ્લેક્સ કમેન્ટ નથી:

### 📁 કોડ (App.jsx):

jsx

```
import { useState, createContext, useContext } from 'react';
```

```
// સ્ટેપ ૧: એક નવી ગ્લોબલ તિજોરી (Context) બનાવી
```

```
const UserContext = createContext();
```

```
function App() {
```

```
  const [user, setUser] = useState("રાહુલ મોદી");
```

```
  return (
```

```
    // સ્ટેપ ૨: Provider ની મદદથી ડેટા વહેંચ્યો (આખી એપ્લિકેશનને કવર કરી)
```

```
    <UserContext.Provider value={user}>
```

```
      <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
```

```
        <h1>મેઇન કમ્પોનન્ટ (App)</h1>
```

```

    <p>અહીંયા યુઝર સ્ટેટ છે.</p>

    <hr />

    {/ * આપણે વચ્ચેના કમ્પોનન્ટને કોઈ પ્રોપ્સ નથી આપી રહ્યા */}

    <ProfilePage />

  </div>

</UserContext.Provider>

);
}

```

// આ વચ્ચેનો કમ્પોનન્ટ છે, જે પ્રોપ્સ લેતો કે આપતો નથી

```

function ProfilePage() {
  return (
    <div>
      <h2>વચ્ચેનો કમ્પોનન્ટ (ProfilePage)</h2>
      <hr />
      <UserCard />
    </div>
  );
}

```

// સ્ટેપ ૩: છેલ્લો નાનો ચાઇલ્ડ કમ્પોનન્ટ જ્યાં અસલી જાદુ થશે

```

function UserCard() {
  // useContext ટૂંક ની મદદથી સીધો જ ગ્લોબલ ડેટા ખેંચી લીધો
  const globalUser = useContext(UserContext);

  return (
    <div style={{ padding: '15px', background: '#fff3cd', borderRadius: '8px' }}>
      <h3>છેલ્લો ચાઇલ્ડ કમ્પોનન્ટ (UserCard)</h3>
      {/ * કોઈ પ્રોપ્સ ટ્રિલિંગ વગર ડેટા સીધો પ્રિન્ટ થયો */}
      <p>ગ્લોબલ તિજોરીમાંથી મળેલું નામ: <b>{globalUser}</b></p>
    </div>
  );
}

```

```
</div>

);

}
```

export default App;

Use code with caution.

---

અહીં **Chapter 12** ના એડવાન્સ સ્ટેટ મેનેજમેન્ટ અને useContext ના તમામ કન્સેપ્ટ્સ પૂરા થાય છે.

આ **Context API** અને **useContext** નો પ્રોપ ડ્રિલિંગ સોલ્યુશન કન્સેપ્ટ એકદમ મસ્ત રીતે મગજમાં બેસી ગયો ને? હવે આગળ વધીને રીએક્ટનો આખો કોર્સ અહીં પૂરો કરવો છે કે હજુ આગળ કોઈ નવો એડવાન્સ ટોપિક શીખવો છે?

આપણે અત્યાર સુધી શીખેલા **React.js** ના **તમામ ૧૨ ચેપ્ટર્સ અને તેના એડવાન્સ કન્સેપ્ટ્સની કમ્પ્લીટ રિવિઝન સમરી (Summary)** નીચે મુજબ છે, જે આખા કોર્સનું એક માળખું તમને હંમેશા મગજમાં ફિટ રાખવા માટે બેસ્ટ સાબિત થશે:

---

#### **React.js કમ્પ્લીટ કોર્સ સમરી (Summary Map)**

- **Chapter 1: Introduction:** React એ ફેસબુક દ્વારા બનેલી ફ્રન્ટએન્ડ **UI JavaScript Library** છે. તે **Single Page Application (SPA)** બનાવવા માટે વપરાય છે જેથી પેજ ક્યારેય રીફ્રેશ નથી થતું. તે **Virtual DOM** ના લીધે ડેટા બદલાતા જ ફક્ત તેટલો જ ભાગ સુધારે છે અને એપ્લિકેશનને સુપર ફાસ્ટ બનાવે છે.
- **Chapter 2: Setup:** આધુનિક સમયમાં **Vite (Vite)** ટૂલનો ઉપયોગ કરીને npm create vite@latest થી પ્રોજેક્ટ ઝડપથી સેટ થાય છે. src/ ફોલ્ડર આપણો અસલી કોડિંગ એરિયા છે અને main.jsx એન્ટ્રી પોઇન્ટ છે.
- **Chapter 3: JSX (JavaScript XML):** જાવાસ્ક્રિપ્ટ ફાઇલમાં જ સીધો HTML લખવાની રીત. તેનો નિયમ છે કે બધું **Single Parent Fragment (<...>)** માં બંધ હોવું જોઈએ, CSS માટે **className** વપરાય છે, અને જાવાસ્ક્રિપ્ટ વેરીએબલ વાપરવા **Curly Braces {}** નો ઉપયોગ થાય છે.
- **Chapter 4: Components & Props:** વેબસાઇટના નાના-નાના ટુકડા એટલે કમ્પોનન્ટ. આજના સમયમાં **Functional Components** જ વપરાય છે. પેરેન્ટમાંથી ચાઇલ્ડમાં ડેટા મોકલવા **Props** વપરાય છે જે Read-only (Immutable) હોય છે. કમ્પોનન્ટની મુખ્ય ૩ સ્થિતિઓ (Mounting, Updating, Unmounting) હોય છે.
- **Chapter 5: State & useState:** કમ્પોનન્ટની પોતાની અંગત લાઇવ મેમરી એટલે **State**. જ્યારે સ્ટેટ બદલાય ત્યારે રીએક્ટ કમ્પોનન્ટને ઓટોમેટિક **Re-render** કરે છે. આના માટે આપણે `const [count, setCount] = useState(0)` જેવો બેઝિક કાઉન્ટર ટૂક વાપરીએ છીએ.

- **Chapter 6: Array & Object State:** મોટો કોમ્પ્લેક્સ ડેટા હેન્ડલ કરતી વખતે સ્ટેટને ડાયરેક્ટ બદલી શકાતો નથી. આપણે જાવાસ્ક્રિપ્ટ **Spread Operator (...)** વાપરીને નવો ઓબ્જેક્ટ કે એરે કોપી કરવો પડે છે, અને લિસ્ટ બતાવવા સાદા લૂપના બદલે **.map()** નો ઉપયોગ થાય છે.
- **Chapter 7: useEffect Hook:** કમ્પોનન્ટની બહાર થતા સાઈડ ઇફેક્ટ્સ જેવા કે લાઈવ API માંથી ડેટા ફ્રેશ કરવો કે ટાઈમર્સ સેટ કરવા તે આનાથી હેન્ડલ થાય છે. ખાલી ડીપેન્ડન્સી એરે [] મુકવાથી API રિક્વેસ્ટ પેજ લોડ થતા જ ફક્ત એક જ વાર રન થાય છે.
- **Chapter 8: Form Handling:** ઇનપુટ બોક્સના ડેટાને રીએક્ટ સ્ટેટ દ્વારા કંટ્રોલ કરવા માટે **Controlled Components** રીત વપરાય છે. ડાયનેમિક કી **[name]: value** નો ઉપયોગ કરીને એક જ હેન્ડલર ફંક્શનથી ઘણા બધા ઇનપુટ્સ એકસાથે કંટ્રોલ કરી શકાય છે અને સબમિટ વખતે **e.preventDefault()** થી પેજ રીફ્રેશ અટકે છે.
- **Chapter 9: React Router:** પેજ રીફ્રેશ કર્યા વગર અલગ-અલગ પેજ્સ લિંક કરવા માટે **react-router-dom** પેકેજ વપરાય છે. આમાં **BrowserRouter, Routes,** અને **Route path="..."** સેટ કરીને નેવિગેશન માટે **<Link to="...">** નો જ ઉપયોગ થાય છે.
- **Chapter 10: Dynamic Routing:** સેંકડો પ્રોડક્ટ્સ કે યુઝર્સ માટે ડાયનેમિક પાથ બનાવવા માટે વેરીએબલની આગળ કોલન મુકાય છે (જેમ કે **/user/:id**). એ લિંકમાંથી લાઈવ આઈડી કોડની અંદર ખેંચવા માટે **useParams()** ફૂંકનો જાદુ વપરાય છે.
- **Chapter 11: useRef Hook:** બ્રાઉઝરના કોઈ પણ HTML એલિમેન્ટને (DOM) ડાયરેક્ટ કોડિંગથી પકડવા માટે આ ફૂંક વપરાય છે, જેમ કે ઇનપુટ બોક્સ પર ઓટો-ફોકસ કરાવવું કે વીડિયો પ્લેયરને પ્લે/પોઝ કરવું. આ ડેટા બદલાવાથી પેજ રી-રેન્ડર થતું નથી.
- **Chapter 12: useContext (Context API):** પ્રોપ્સને એક કમ્પોનન્ટમાંથી બીજા અને ત્રીજામાં નકામી પાસ કરવાની પ્રક્રિયા એટલે **Prop Drilling**. આના કાયમી ઉકેલ માટે આપણે Context API ની મદદથી એક ગ્લોબલ સેન્ટ્રલ ટિજોરી બનાવીએ છીએ અને **useContext** થી કોઈપણ ખૂણેથી ડેટા ડાયરેક્ટ એક્સેસ કરીએ છીએ.

---

તમે ખૂબ જ સરસ રીતે અને એકદમ ધ્યાન આપીને **Node.js, Express, MongoDB અને React.js (MERN Stack)** ના તમામ પાયાના અને એડવાન્સ કન્સેપ્ટ્સ કમ્પ્લીટ ગુજરાતીમાં શીખી લીધા છે!

## Stage One: INTRODUCTION TO JAVASCRIPT, BROWSERS AND DOM [1]

### ૧. What is Event driven programming [1]

- **સમજૂતી:** આ એક પ્રોગ્રામિંગ સ્ટાઇલ છે જેમાં કોડ પોતાની જાતે રન નથી થતો, પણ યુઝરની કોઈ એક્ટિવિટી કે ઘટના (Event) ના આધારે ચાલે છે.
- દાખલા તરીકે, વેબસાઇટ પર કોઈ બટન પર ક્લિક કરવું, માઉસ ફેરવવું, કે કીબોર્ડ પર કી દબાવવી— આ બધી ઇવેન્ટ્સ છે. જ્યારે આ ઇવેન્ટ બને ત્યારે જ તેની પાછળ લખેલો જાવાસ્ક્રિપ્ટ કોડ એક્ઝિક્યુટ થાય છે.

### ૨. What is Javascript [1]

- **સમજૂતી:** જાવાસ્ક્રિપ્ટ એ દુનિયાની સૌથી લોકપ્રિય પ્રોગ્રામિંગ લેંગ્વેજ છે જે વેબસાઇટને લાઇવ અને ડાયનેમિક (Interactivity) બનાવવાનું કામ કરે છે. HTML થી પેજનું માળખું બને છે, CSS થી કલર-ડિઝાઇન થાય છે અને JavaScript થી વેબસાઇટનું મગજ ચાલે છે.

### ૩. Introduction to DOM | How does the browser work [1]

- **DOM (Document Object Model):** જ્યારે બ્રાઉઝર કોઈ HTML પેજને લોડ કરે છે, ત્યારે તે અંદરથી આખા પેજનું એક ટ્રી જેવું માળખું (Tree Structure) બનાવે છે, જેને DOM કહેવાય છે.
- **બ્રાઉઝર કેવી રીતે કામ કરે છે?:** તમે લિંક ઓપન કરો એટલે બ્રાઉઝર સૌથી પહેલા HTML ફાઇલને ડાઉનલોડ કરીને તેને એક ઓબ્જેક્ટ ટ્રી (DOM) માં ફેરવે છે, જેથી જાવાસ્ક્રિપ્ટ તેની સાથે વાતચીત કરી શકે.

### ૪. Render-Tree construction, layout and paint in Browser [1]

બ્રાઉઝર જ્યારે સ્ક્રીન પર પેજ બતાવે છે, ત્યારે તે ૩ સ્ટેપમાંથી પસાર થાય છે:

- **Render-Tree Construction:** બ્રાઉઝર HTML ના DOM અને CSS ના નિયમો (CSSOM) બંનેને ભેગા કરીને એક 'Render Tree' બનાવે છે (જેમાં કઈ વસ્તુ સ્ક્રીન પર દેખાડવાની છે તેનું વિસ્તર ઢોચ છે).
- **Layout:** બ્રાઉઝર ગણતરી કરે છે કે સ્ક્રીન પર કયો બોક્સ કે ટેક્સ્ટ કઈ જગ્યાએ, કેટલા પિક્સલની સાઇઝમાં ગોઠવાશે.
- **Paint:** આ છેલ્લો સ્ટેપ છે, જેમાં બ્રાઉઝર સ્ક્રીન પર અસલી અક્ષરો, કલર, બોર્ડર અને ફોન્ટ દોરે છે (Render કરે છે).

### ૫. DOM traversal (most important part) [1]

- **સમજૂતી:** DOM Traversal એટલે જાવાસ્ક્રિપ્ટની મદદથી HTML ના માળખામાં ઉપર-નીચે કે આજુબાજુ મુસાફરી કરવી.
- દાખલા તરીકે, કોઈ ચોક્કસ વિસ્તર પરથી તેના પેરેન્ટ ટેગ (parentElement) પર જવું, તેના બાળકો (children) ને પકડવા, કે તેના ભાઈ-બહેન (nextSibling) ટેગ સુધી પહોંચીને તેનો કલર કે લખાણ બદલવું. રીએક્ટ સમજવા માટે આ પાઠો ખૂબ જ જરૂરી છે.

---

## Stage Two: INTRODUCTION TO REACTJS [1]

### ૧. What is react? | Why ReactJS? | Setup - React [1]

- **React શું છે?:** React એ વેબ એપ્લિકેશનના UI (User Interface) એટલે કે ફ્રન્ટએન્ડ પેજીસ બનાવવા માટેની એક ઓપન-સોર્સ JavaScript Library છે, જે ફેસબુક (Meta) દ્વારા બનાવવામાં આવી છે.
- **કેમ વાપરવું?:** તે વેબસાઇટને Single Page Application (SPA) બનાવે છે, જેથી યુઝર ક્લિક કરે ત્યારે આખું પેજ ક્યારેય રીફ્રેશ નથી થતું, ફક્ત ડેટા જ બદલાય છે.
- **Setup:** તેને લોકલ કમ્પ્યુટરમાં ચાલુ કરવા માટે Node.js ઇન્સ્ટોલ હોવું જરૂરી છે.

### ૨. Structure of ReactJS Projects [1]

- રીએક્ટ પ્રોજેક્ટમાં મુખ્ય ફોલ્ડર્સ હોય છે જેમ કે node\_modules (ડાઉનલોડ કરેલા ટૂલ્સ), public/ (સ્ટેટિક ફાઇલો) અને src/ (Source). આપણો અસલી કોડ હંમેશા src/ ફોલ્ડરની અંદર જ લખાય છે.

### 3. Webpack - the role of package managers or bundlers [1]

- **સમજૂતી:** જ્યારે આપણે મોટો પ્રોજેક્ટ બનાવીએ ત્યારે આપણી પાસે ૫૦ અલગ-અલગ જાવાસ્ક્રિપ્ટ અને CSS ફાઇલો હોય છે. **Webpack** એ એક બંડલર (Bundler) છે, જે આ બધી જ ફાઇલોને ભેગી કરીને એક નાની સિંગલ ફાઇલમાં પેક કરી દે છે, જેથી બ્રાઉઝર તેને ખૂબ જ ઝડપથી લોડ કરી શકે.

### ૪. JSX in detail | First React app [1]

- **JSX (JavaScript XML):** આ રીએક્ટની ખાસિયત છે જે આપણને જાવાસ્ક્રિપ્ટ ફાઇલમાં જ સીધો HTML જેવો કોડ લખવાની સુવિધા આપે છે. બ્રાઉઝર આને સમજી ન શકતું હોવાથી બેકએન્ડમાં Babel તેને સાદા જાવાસ્ક્રિપ્ટમાં કન્વર્ટ કરે છે.

---

## Stage Three: REACTJS COMPONENTS [1]

### ૧. Intro to React components | Thinking in React [1]

- **Components:** રીએક્ટમાં આખી વેબસાઇટને નાના-નાના ટુકડાઓમાં વહેંચવામાં આવે છે, જેને કમ્પોનન્ટ કહેવાય (જેમ કે Navbar, Sidebar, Footer).
- **Thinking in React:** પ્રોજેક્ટની ડિઝાઇન જોઈને મગજમાં એ નક્કી કરવું કે કયા ભાગને અલગ કમ્પોનન્ટ બનાવવો અને તેને કેવી રીતે આખી સાઇટમાં વાપરવો.

### ૨. Component lifecycle | Creating a React Component [1]

- **Component Lifecycle:** એક કમ્પોનન્ટ જ્યારે સ્ક્રીન પર આવે (Mount થાય), તેનો ડેટા બદલાય (Update થાય) અને તે સ્ક્રીન પરથી ગાયબ થાય (Unmount થાય), તેને તેની લાઇફસાઇકલ કહેવાય.
- **Creating:** તેને બનાવવા માટે આપણે એક સાદું જાવાસ્ક્રિપ્ટ ફંક્શન લખીને JSX રિટર્ન કરીએ છીએ.

### ૩. Rendering multiple components [1]

- આપણે એક કમ્પોનન્ટની અંદર બીજા ઘણા બધા કમ્પોનન્ટ્સ મૂકીને તેને સ્ક્રીન પર એકસાથે રેન્ડર (બતાવી) શકીએ છીએ.

### ૪. Nesting Components | How to split an App into components [1]

- **Nesting:** એક મોટા કમ્પોનન્ટની અંદર નાના કમ્પોનન્ટ મૂકવા (જેમ કે App ની અંદર Card મૂકવું).
- **Splitting:** મોટો કોડ એક જ ફાઇલમાં રાખવાના બદલે, દરેક કમ્પોનન્ટના નાના-નાના ટુકડા કરીને અલગ ફાઇલ બનાવવી.

### ૫. Comparing Stateless and Stateful Components [1]

- **Stateless Component (Functional):** જે કમ્પોનન્ટ પાસે પોતાનો કોઈ લાઈવ ડેટા (State) નથી હોતો, તે ફક્ત બહારથી મળેલા ડેટાને ડિસ્પ્લે કરે છે.
- **Stateful Component:** જે કમ્પોનન્ટ પાસે પોતાની મેમરી (State) હોય છે અને તે જરૂર મુજબ સ્ક્રીન પર ડેટા બદલી શકે છે.

### ૬. Understanding the Component Lifecycle [1]

- ક્લાસ કમ્પોનન્ટમાં `componentDidMount` અને ફંક્શનલ કમ્પોનન્ટમાં `useEffect` હૂક દ્વારા આ પ્રક્રિયા સમજાય છે કે ક્યારે કયો કોડ રન કરવો.

### 9. Converting Stateless to Stateful Components [1]

- જ્યારે કોઈ સાદા કમ્પોનન્ટમાં આપણે બટન ક્લિક કે યુઝર ડેટા બદલવાનો ફીચર ઉમેરવો હોય, ત્યારે તેની અંદર `useState` હૂક ઉમેરીને તેને સ્ટેટફુલ બનાવવામાં આવે છે.

### ૮. One Component Per File - how to break app in multiple files [1]

- પ્રોફેશનલ નિયમ છે કે એક ફાઇલમાં માત્ર એક જ મુખ્ય કમ્પોનન્ટ રાખવો. તેને નીચેથી `export default` કરવો અને બીજી ફાઇલમાં `import` કરીને વાપરવો.

---

## Stage Four: HOW REACTJS WORKS INTERNALLY [1]

### ૧. How React Updates the App & Component Tree [1]

- જ્યારે પણ કમ્પોનન્ટનો કોઈ ડેટા બદલાય, ત્યારે રીએક્ટ આખા પેજને બદલવાના બદલે ફક્ત એ જ ચોક્કસ કમ્પોનન્ટના ટ્રી (Component Tree) ને પકડે છે અને તેને અપડેટ કરે છે.

### ૨. Understanding React's DOM Updating Strategy [1]

- રીએક્ટ **Virtual DOM** ની સ્ટ્રેટેજી વાપરે છે. તે ઓરિજિનલ DOM ની ડુપ્લિકેટ કોપી મેમરીમાં રાખે છે. જૂના અને નવા ડેટા વચ્ચે શું તફાવત છે તે ચેક કરવા માટે તે **Diffing Algorithm** વાપરે છે, અને ફક્ત બદલાયેલો ભાગ જ સાચા બ્રાઉઝરમાં અપડેટ કરે છે.

### ૩. Understanding Higher Order Components [1]

- **Higher-Order Component (HOC):** આ એક એવું એડવાન્સ ફંક્શન છે જે આખા કમ્પોનન્ટને ઇનપુટ તરીકે લે છે, તેની અંદર કોઈ નવો ફીચર કે લોજિક ઉમેરે છે, અને એક નવો મોડિફાઇ થયેલો કમ્પોનન્ટ પાછો આપે છે (કોડ રીયુઝેબિલિટી માટે આ વપરાય છે).

---

મેં સિલેબસના **Stage 1 થી Stage 4** ના તમામ કન્સેપ્ટ્સ લાઇન બાય લાઇન ટૂંકી અને સાદી ભાષામાં સમજાવી દીધા છે. હવે આગળ વધીને **Stage 5 (Props, Events and States)** શીખવાનું શરૂ કરીએ? જો કોઈ ડાઉટ હોય તો કહી શકો છો!

ચાલો, હવે આપણે આગળ વધીએ અને **Stage Five (Props, Events and States)** તથા તેની પછીના સ્ટેજને એકદમ ઊંડાણપૂર્વક, લાઇન બાય લાઇન અને સરળ ગુજરાતીમાં સમજીએ.

---

## Stage Five: PROPS, EVENTS AND STATES

### ૧. Intro to props

- **Props (Properties):** આ રીએક્ટના એવા ઇનપુટ પેરામીટર્સ છે જેનો ઉપયોગ પેરેન્ટ કમ્પોનન્ટમાંથી ચાઇલ્ડ કમ્પોનન્ટમાં ડેટા મોકલવા માટે થાય છે.

### ૨. props in action | adding props to `ReactDOM.render()`

- **Props in Action:** કમ્પોનન્ટ ટેગની અંદર એટ્રિબ્યુટ તરીકે ડેટા લખવો (જેમ કે <User name="Rahul" />).
- **ReactDOM.render() માં Props:** જૂના રીએક્ટ વર્ઝનમાં આખી એપ્લિકેશનના મેઇન રૂટ પર ઝલોબલ ડેટા સેટ કરવા માટે ડાયરેક્ટ રેન્ડર ફંક્શનની અંદર પ્રોપ્સ પાસ કરવામાં આવતી હતી.

### ૩. Intro to events

- **Events:** વેબસાઇટ પર યુઝર દ્વારા થતી એક્ટિવિટીને ઇવેન્ટ કહેવાય છે. રીએક્ટમાં ઇવેન્ટ હેન્ડલિંગ સાદા HTML જેવું જ છે પરંતુ તેને **camelCase** માં લખવામાં આવે છે, જેમ કે onClick, onChange, onSubmit.

### ૪. Events in action

- જ્યારે યુઝર બટન દબાવે ત્યારે જાવાસ્ક્રિપ્ટનું ફંક્શન ટ્રિગર કરવું.
- **ઉદાહરણ:** <button onClick={handleClick}>ક્લિક કરો</button>

### ૫. What is state

- **State:** કમ્પોનન્ટની અંદર રહેલી તેની પોતાની એક **લાઇવ લોકલ મેમરી** છે. તેનો ઉપયોગ એવા ડેટાને સાચવવા માટે થાય છે જે ભવિષ્યમાં બદલાઈ શકે છે.

### ૬. Difference between props (immutable) and states

- **Props:** તે બહારથી (Parent માંથી) આવે છે અને તે **Immutable (Read-only)** છે, એટલે કે યાઇલ્ડ કમ્પોનન્ટ તેને જાતે બદલી શકતો નથી.
- **State:** તે કમ્પોનન્ટનો પોતાનો ડેટા છે જે **Mutable** છે, એટલે કે કમ્પોનન્ટ પોતાની મરજીથી ગમે ત્યારે તેને બદલી શકે છે.

### ૭. Adding state to the component

- ફંક્શનલ કમ્પોનન્ટમાં સ્ટેટ ઉમેરવા માટે **useState Hook** નો ઉપયોગ થાય છે (જેમ કે const [data, setData] = useState(initialValue)).

### ૮. Track changing data in your components

- જ્યારે યુઝર ઇનપુટ બોક્સમાં ટાઇપ કરે છે, ત્યારે onChange ઇવેન્ટ લાઇવ ડેટાને ટ્રેક કરે છે અને સ્ટેટ અપડેટિંગ ફંક્શન દ્વારા ડેટાને તુરંત મેમરીમાં અપડેટ કરે છે.

### ૯. Multiple child components

- એક સિંગલ પેરેન્ટ કમ્પોનન્ટની અંદર ઘણા બધા અલગ-અલગ યાઇલ્ડ કમ્પોનન્ટ્સ એકસાથે વાપરી શકાય છે અને દરેકને અલગ-અલગ પ્રોપ્સ મોકલી શકાય છે.

### ૧૦. Updating state and removing component

- **Updating State:** સ્ટેટ બદલાતા જ રીએક્ટ આપોઆપ સ્ક્રીનને અપડેટ (Re-render) કરે છે.
- **Removing Component:** જો કોઈ શરત ખોટી પડે (Conditional Rendering), તો રીએક્ટ એ કમ્પોનન્ટને DOM માંથી કાયમ માટે હટાવી (Unmount) દે છે.

### ૧૧. Creating App using states and props

- આ બંને કન્સેપ્ટ ભેગા કરીને એક ડાયનેમિક રીઅલ-ટાઇમ એપ્લિકેશન (જેમ કે શોપિંગ કાર્ટ કે ટુ-ડુ લિસ્ટ) બનાવવી.

(નોંધ: સિલેબસ ઇમેજમાં સ્ટેજ ૫ ના અમુક લિસ્ટ રીપીટ થયેલા છે, જેને આપણે એક જ વાર સમજી લીધા છે).

---

## Stage Six: STYLING REACT APP

### ૧. Setting Styles Dynamically

- રીએક્ટમાં આપણે જાવાસ્ક્રિપ્ટ વેરીએબલના આધારે વાઈવ CSS પ્રોપર્ટી બદલી શકાતી હોય તેને **Dynamic Styling** કહેવાય છે. જેમ કે જો કાઉન્ટ Negative હોય તો ટેક્સ્ટ કલર લાલ કરવો અને Positive હોય તો લીલો કરવો.

### ૨. Setting Class Names Dynamically

- કોઈ શરતના આધારે CSS નો ક્લાસ બદલવો.
- ઉદાહરણ:** <div className={isLoggedIn ? "active-box" : "guest-box"}>

### ૩. Using Radium for Media Queries

- Radium:** આ એક જૂની પ્રખ્યાત થર્ડ-પાર્ટી લાઇબ્રેરી છે જેનો ઉપયોગ ઇનવાઇઝન જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટ સ્ટાઇલમાં પણ મિડીયા ક્વેરી (@media) અને કસ્ટમ હોવર ઇફેક્ટ્સ (:hover) વાપરવા માટે થતો હતો.

### ૪. Enabling & Using CSS Modules

- CSS Modules:** આ એક ખૂબ જ પાવરફુલ પ્રોફેશનલ ફ્રીયર છે. આમાં .module.css ફાઇલ બનાવવામાં આવે છે. આનાથી એ ફાઇલના CSS ક્લાસ ફક્ત એ જ ચોક્કસ કમ્પોનન્ટ પૂરતા લોક (Scoped) થઈ જાય છે, જેથી એક કમ્પોનન્ટની CSS બીજા કમ્પોનન્ટ સાથે ક્લેશ અથડાતી (Clash) નથી.

### ૫. Outputting Lists with ngFor (Note: ngFor is Angular, React uses .map())

- સિલેબસમાં અહીંયા ભૂલથી એન્જ્યુલરનો ngFor કીર્ડ લખાયેલો છે. રીએક્ટની અંદર એરે કે ડેટાના લિસ્ટને સ્ક્રીન પર લૂપ ફેરવીને બતાવવા માટે **.map()** ફંક્શનનો ઉપયોગ થાય છે.

### ૬. Practicing Directives

- ડાયરેક્ટિવ્સ પણ એન્જ્યુલરનો કન્સેપ્ટ છે. રીએક્ટમાં તેના બદલે આપણે **Conditional Rendering** (જેમ કે {show && <Component />}) અને **List Rendering** ની પ્રેક્ટિસ કરીએ છીએ.

### ૭. Mobile Considerations

- વેબસાઇટ કમ્પ્યુટરની સાથે મોબાઇલ સ્ક્રીન પર પણ પ્રોપર અને રિસ્પોન્સિવ (Responsive) દેખાય તે માટે ફ્લેક્સબોક્સ, ગ્રીડ અને વ્યુપોર્ટ સેટિંગ્સનું ધ્યાન રાખવું.

---

## Stage Seven: WORKING WITH REACT-ROUTER

### ૧. Intro to React Router

- **React Router:** રીએક્ટ એપ્લિકેશનમાં પેજ રીફ્રેશ કર્યા વગર અલગ-અલગ લિંક્સ અને પેજીસ (જેમ કે હોમ, અબાઉટ) હેન્ડલ કરવા માટેની ઓફિશિયલ લાઇબ્રેરી.

## ૨. Server vs. Client Routing

- **Server Routing:** સાદી વેબસાઇટમાં લિંક પર ક્લિક કરવાથી બ્રાઉઝર સર્વર પર જાય છે અને આખું નવું પેજ રીફ્રેશ કરીને ડાઉનલોડ કરે છે.
- **Client Routing:** રીએક્ટમાં પેજ રીફ્રેશ થતું નથી. જાવાસ્ક્રિપ્ટ બેકએન્ડમાં લાઇવ સ્ક્રીન પર કમ્પોનન્ટ બદલી નાખે છે જેથી સાઇટ સુપર ફાસ્ટ ચાલે છે.

## ૩. Setting up a 404

- જો યુઝર વેબસાઇટ પર કોઈ એવી લિંક નાખે જે અસ્તિત્વમાં નથી, તો તેને **Page Not Found** બતાવવા માટે રાઉટના પાથમાં `path="*"`  સેટ કરવો.

## ૪. Linking Between Routes

- પેજ રીફ્રેશ અટકાવીને એક પેજ પરથી બીજા પેજ પર જવા માટે રીએક્ટમાં `<a>` ટેગના બદલે `<Link to="...">` કમ્પોનન્ટ વપરાય છે.

## ૫. Navigating Programmatically

- યુઝર કોઈ બટન દબાવે કે ફોર્મ સબમિટ કરે, ત્યારે આપોઆપ કોડિંગ દ્વારા તેને બીજા પેજ પર મોકલવા (Redirect કરવા) માટે **useNavigate Hook** નો ઉપયોગ થાય છે.

## ૬. Using Query Parameters

- URL ની પાછળ આવતા એક્સ્ટ્રા ડેટા (જેમ કે `/search?product=laptop`) ને કોડની અંદર એક્સેસ કરવા અને વાંચવા માટે **useSearchParams** નો ઉપયોગ કરવો.

## ૭. Redirecting and Wildcard Routes

- **Redirecting:** યુઝર ખોટો પાથ ઓપન કરે તો તેને ઓટોમેટિક સાચા હોમ પેજ પર મોકલી દેવો (`<Navigate to="/" />`).
- **Wildcard Routes:** ગમે તેવા અજાણ્યા પાથને કેચ કરવા માટે `*` રાઉટ સેટ કરવો.

---

## Stage Eight: HANDLING FORMS IN REACT APP

### ૧. What is web form

- **Web Form:** યુઝર પાસેથી નામ, ઈમેલ, પાસવર્ડ કે કમેન્ટ જેવો ઇનપુટ ડેટા લેવા માટેનું માળખું.

### ૨. Creating a Custom Dynamic Input Component

- એક એવો કોમન ઇનપુટ કમ્પોનન્ટ બનાવવો જે પ્રોપ્સના આધારે ટેક્સ્ટ બોક્સ, પાસવર્ડ બોક્સ કે ડ્રોપડાઉન બોક્સમાં આપોઆપ કન્વર્ટ થઈ શકે, જેથી વારંવાર કોડ ન લખવો પડે.

### ૩. Setting Up a JS Config for the Form

- ફોર્મના બધા જ ઇનપુટ્સનું લોજિક એક મોટો જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટ (Configuration Object) બનાવીને તેમાં સેવ કરવું.

#### ૪. Dynamically Create Inputs based on JS Config

- ઉપર બનાવેલા ઓબ્જેક્ટ કન્ફિગરેશન પર લૂપ (.map()) ફેરવીને બધા જ ઇનપુટ બોક્સ સ્ક્રીન પર ઓટોમેટિકલી જનરેટ કરવા.

#### ૫. Handling User Input

- યુઝર ટાઇપ કરે ત્યારે onChange ઇવેન્ટ લાઈવ e.target.value મેળવીને સ્ટેટમાં સેવ કરે છે.

#### ૬. Handling Form Submission

- ફોર્મ સબમિટ કરતી વખતે બ્રાઉઝરનું ઓટોમેટિક રીફ્રેશ અટકાવવા માટે e.preventDefault() રન કરવું.

#### ૭. Adding Custom Form Validation

- યુઝરે સાચો ઇમેલ નાખ્યો છે કે નહીં, પાસવર્ડ ૬ અક્ષરથી મોટો છે કે નહીં તે ચેક કરવા માટેના જાવાસ્ક્રિપ્ટ નિયમો (Validation Logic) ઉમેરવા.

#### ૮. Showing Error Messages

- જો વેલિડેશન ખોટું પડે, તો યુઝરને ઇનપુટ બોક્સની નીચે લાલ અક્ષરમાં એરર મેસેજ લાઈવ ડિસ્પ્લે કરવો.

---

અહીં **Stage 5 થી Stage 8** ના તમામ પ્રેક્ટિકલ અને ફોર્મ સેટઅપના કન્સેપ્ટ્સ પુરા થાય છે.

હવે આગળ વધીને આ સિલેબસના સૌથી એડવાન્સ ટોપિક **Stage 9, 10 અને 11 (Redux State Management)** માં એકદમ ડીપમાં જઈએ? જો આમાં કોઈ ડાઉટ હોય તો કહી શકો છો!

ચાલો, હવે આપણે આ સિલેબસના સૌથી મહત્વના અને એડવાન્સ ભાગ એટલે કે **Stage Nine, Stage Ten અને Stage Eleven (Redux અને Redux in Action)** માં એકદમ ઊંડાણપૂર્વક, લાઇન બાય લાઇન અને સરળ ગુજરાતીમાં સમજીએ.

---

### Stage Nine: REDUX (સ્ટેટ મેનેજમેન્ટનો કિંગ)

#### ૧. Understanding state management

- State Management એટલે શું?** અત્યાર સુધી આપણે useState થી જે સ્ટેટ બનાવતા હતા તે ફક્ત એ જ કમ્પોનન્ટ પૂરતું મર્યાદિત (Local) હતું. એપ્લિકેશન જ્યારે ખૂબ જ મોટી બને, ત્યારે બધા જ કમ્પોનન્ટ્સ વચ્ચે ગ્લોબલ ડેટાનું બેલેન્સ રાખવું અને તેને કન્ટ્રોલ કરવાની પદ્ધતિને સ્ટેટ મેનેજમેન્ટ કહેવાય છે.

#### ૨. Complexity of Managing State

- જ્યારે પ્રોજેક્ટ બહુ મોટો થાય ત્યારે Prop Drilling ના કારણે કોડ ગૂંચવાઈ જાય છે. કયા કમ્પોનન્ટમાંથી ડેટા બદલાયો અને ક્યાં અપડેટ થયો તે ટ્રેક કરવું અશક્ય બની જાય છે. કોડની આ જ જટિલતા (Complexity) ને દૂર કરવા માટે ગ્લોબલ સ્ટેટ ટૂલની જરૂર પડે છે.

#### ૩. Intro to redux

- **Redux શું છે?:** Redux એ જાવાસ્ક્રિપ્ટ એપ્લિકેશન્સ માટેનું એક ઓપન-સોર્સ **Global State Management Library** છે. તે આપી એપ્લિકેશનનો ડેટા એક જ સેન્ટ્રલ જગ્યાએ સ્ટોર કરવાની સુવિધા આપે છે.

#### ૪. Need for Redux

- રીએક્ટમાં ડેટા પાસ કરવાની જગ્યામાંથી મુક્તિ મેળવવા માટે આની જરૂર છે. આનાથી ગમે તેટલો મોટો પ્રોજેક્ટ હોય, ડેટા એક જ જગ્યાએથી કન્ટ્રોલ થતો હોવાથી બગસ આવવાની શક્યતા નહિવત થઈ જાય છે.

#### ૫. Understanding the Redux Flow

Redux નો ફ્લો એક વન-વે યક્ષ (Unidirectional Data Flow) છે, જે આ ૩ સ્ટેપમાં ચાલે છે:

[Component / UI] —► (Dispatches) —► [Action]



(Updates UI) ◀— [Store] ◀— (Returns) ◀— [Reducer]

#### ૬. What is an action

- **Action:** આ એક સાદો જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટ છે જે એ જણાવે છે કે એપ્લિકેશનમાં શું ફેરફાર કરવો છે. તેમાં એક type પ્રોપર્ટી હોવી ફરજિયાત છે. (દા.ત. { type: 'INCREMENT\_COUNT' } ).

#### ૭. What is store

- **Store:** આ આપી એપ્લિકેશનની એકમાત્ર મુખ્ય ગ્લોબલ તિજોરી (Central Database of Frontend) છે, જ્યાં આપો ગ્લોબલ ડેટા એકસાથે સેવ થયેલો રહે છે.

#### ૮. Understanding reducer

- **Reducer:** આ એક શુદ્ધ ફંક્શન (Pure Function) છે, જે હાલની State અને આવેલી Action ને ઇનપુટ તરીકે લે છે, અને તેના આધારે નવો ડેટા ગણતરી કરીને સ્ટોર (Store) ને પાછો આપે છે.

#### ૯. Understanding Provider

- **Provider:** આ રીએક્ટ-રેડક્સનું એક કન્ટેનર છે, જે આપણી મુખ્ય રીએક્ટ એપ્લિકેશનને ઉપરથી કવર કરે છે, જેથી રીએક્ટના બધા જ કમ્પોનન્ટ્સ આ રેડક્સ સ્ટોર (તિજોરી) સાથે સીધા કનેક્ટ થઈ શકે.

---

### Stage Ten: REDUX IN ACTION (પ્રેક્ટિકલ રેડક્સ)

#### ૧. Setting Up Reducer and Store

- કોડની અંદર ઇનિશિયલ ડેટા સેટ કરીને રીડ્યુસર ફંક્શન બનાવવું અને createStore(reducer) ની મદદથી અસલી ગ્લોબલ સ્ટોર ક્રિએટ કરવો.

#### ૨. Dispatching Actions

- કમ્પોનન્ટની અંદરથી સ્ટોરને એક્શન મોકલવા માટે **dispatch()** ફંક્શનનો ઉપયોગ કરવો. આ લાઇન રન થતાં જ રીડ્યુસર એક્ટિવ થઈ જશે.

### 3. Connecting React to Redux & Connecting the Store to React

- રીએક્ટ કમ્પોનન્ટ્સને રેડક્સના ડેટા સાથે લિંક કરવા. આના માટે જૂના સમયમાં `connect(mapStateToProps, mapDispatchToProps)` વાપરવામાં આવતું હતું. (જોકે અત્યારે આધુનિક રીએક્ટમાં આના બદલે **useSelector** અને **useDispatch** હૂક વપરાય છે).

### ૪. Dispatching Actions from within the Component

- બટનના `onClick` ઇવેન્ટની અંદર જ `dispatch({ type: 'ADD_ITEM', payload: data })` લખીને લાઈવ ડેટા સ્ટોર તરફ મોકલવો.

### ૫. Practice - Dispatching Actions

- અલગ-અલગ કસ્ટમ ઇવેન્ટ્સ અને પેલોડ (Payload - એક્સ્ટ્રા ડેટા) સાથે એક્શન મોકલવાની પ્રેક્ટિસ કરવી.

### ૬. Passing and Retrieving Data with Action

- એક્શન ઓબ્જેક્ટની અંદર `type` ની સાથે સાચો ડેટા મોકલવો (જેને `payload` કહેવાય) અને રીડ્યુસરની અંદર `action.payload` લખીને તે મેળવવો.

### ૭. Switch-Case in the Reducer

- રીડ્યુસર ફંક્શનની અંદર આવેલી એક્શનના `type` ને અલગ-અલગ કેસ સાથે સરખાવવા માટે આપણે મોટા `if-else` ના બદલે **`switch(action.type)`** સ્ટેટમેન્ટનો ઉપયોગ કરીએ છીએ, જે કોડને એકદમ ક્લીન રાખે છે.

### ૮. Updating State and arrays immutably

- રેડક્સનો સૌથી મોટો સખત નિયમ છે કે સ્ટેટનો ડેટા ક્યારેય ડાયરેક્ટ બદલવો નહીં. એરે કે ઓબ્જેક્ટ અપડેટ કરતી વખતે હંમેશા જાવાસ્ક્રિપ્ટ **Spread Operator (...)** વાપરીને નવી કોપી જ રીડ્યુસરમાંથી રિટર્ન કરવી જોઈએ.

## Stage Eleven: REDUX IN ACTION PROJECT USING REACT-REDUX

### ૧. Installing Redux and React Redux

- પ્રોજેક્ટ ફોલ્ડરમાં જઈને ટર્મિનલમાં આ બંને મોડ્યુલ્સ ઇન્સ્ટોલ કરવા:  
`npm install redux react-redux`

### ૨. Basic Redux Setup | Creating Ticketing App using React and Redux

- એક સાદો પ્રોજેક્ટ બનાવવો (જેમ કે ટિકિટિંગ એપ કે કાઉન્ટર એપ), જેમાં સ્ટોર, એક્શન અને રીડ્યુસર બધું જ પ્રોફેશનલી સેટ કરેલું હોય.

### 3. Deploying react app on server

- આપણો રીએક્ટ પ્રોજેક્ટ કમ્પીટ થઈ જાય એટલે npm run build કમાન્ડ રન કરીને તેનું પ્રોડક્શન ફોલ્ડર (dist) બનાવવું. ત્યારબાદ તેને **GitHub Pages**, **Netlify**, અથવા **Vercel** જેવા ફ્રી સર્વર પર લાઈવ ડેપ્લોય (Deploy) કરવું જેથી લિંક દુનિયામાં ગમે તેને મોકલી શકાય.

#### ૪. Wrap up let's test how much we learnt so far your feedback

- આખા કોર્સનું કમ્પીટ રિવિઝન કરવું, બિલ્ડ થયેલા પ્રોજેક્ટનું લાઈવ ટેસ્ટિંગ કરવું અને ભવિષ્યના ડેવલપમેન્ટ માટે ફીડબેક શેર કરવો.

અહીં તમારા આ મોકલેલા બીજા સિલેબસના પણ **તમામે તમામ ૧૧ સ્ટેજ** એક પણ ટોપિક મિસ કર્યા વગર ખૂબ જ વિગતવાર અને સરળ ગુજરાતી ભાષામાં પૂરા થાય છે. તમે હવે React.js અને Redux બંનેનો પાયો ખૂબ મજબૂત રીતે સમજી લીધો છે!

ચાલો, હવે આપણે તમારા **React.js અને Redux** ના ઓફિશિયલ સિલેબસના એક-એક સ્ટેજમાં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ. આપણે દરેક કન્સેપ્ટની વ્યાખ્યા (Definition), અંદરનું લોજિક (Logic), તેનો સાચો સિન્ટેક્સ (Syntax), અને રિયલ-વર્લ્ડ પ્રેક્ટિકલ કોડ જોઈશું જેથી એક પણ પોઇન્ટ મિસ ન થાય.

આપણે એકદમ બેઝિક **Stage One** થી શરૂઆત કરીએ.

### Stage One: Introduction to JavaScript, Browsers and DOM (ઊંડાણપૂર્વક અભ્યાસ)

#### ૧. What is Event Driven Programming?

##### 📌 વ્યાખ્યા (Definition):

**Event-Driven Programming** એ એક એવી પ્રોગ્રામિંગ પદ્ધતિ (Paradigm) છે જેમાં કોડનો ફ્લો પોતાની જાતે ક્રમબદ્ધ ચાલવાના બદલે, યુઝર અથવા સિસ્ટમ દ્વારા થતી કોઈ ઇવેન્ટ (ઘટના) ના આધારે નક્કી થાય છે.

##### ⚙️ અંદરનું લોજિક (Logic):

સાદા કમાન્ડ લાઇન પ્રોગ્રામમાં કોડ લાઇન નંબર ૧ થી શરૂ થઈને છેલ્લી લાઇન પર આપોઆપ પૂરો થઈ જાય છે. પરંતુ વેબસાઇટમાં એવું નથી હોતું. વેબસાઇટ સર્વર પર લોડ થઈને શાંતિથી યુઝરની પ્રવૃત્તિની રાહ જુએ છે.

- જેવી કોઈ ઇવેન્ટ બનશે (દા.ત. યુઝર બટન દબાવશે), એટલે બ્રાઉઝર બેકગ્રાઉન્ડમાં એ ઇવેન્ટ માટે બનેલા **Callback Function (જેને Event Handler કહેવાય છે)** ને ટ્રિગર કરશે.

##### 📄 સિન્ટેક્સ (Syntax & Code):

javascript

```
// ૧. HTML નું બટન પકડવું
```

```
const btn = document.getElementById("myBtn");
```

```
// ૨. ઇવેન્ટ લિસનર ગોઠવવો (Event: click, Handler: callback function)
```

```
btn.addEventListener("click", () => {
```

```
console.log("ઇવેન્ટ ટ્રિગર થઈ: યુઝરે બટન પર ક્લિક કર્યું!");
```

```
});
```

Use code with caution.

## ૨. What is JavaScript?

### 🔴 વ્યાખ્યા (Definition):

**JavaScript (JS)** એ એક હાઇ-લેવેલ, ડાયનેમિક, ઇન્ટરપ્રિટેડ (Interpreted), અને મલ્ટી-પેરાડાઇમ પ્રોગ્રામિંગ લેંગ્વેજ છે. તે વેબસાઇટનું મગજ છે, જે સ્થિર (Static) HTML/CSS પેજને જીવંત અને ઇન્ટરેક્ટિવ બનાવવાનું કામ કરે છે.

### ⚙️ અંદરનું લોજિક (Logic):

બ્રાઉઝરની અંદર એક ખાસ એન્જિન હોય છે (જેમ કે ક્રોમમાં V8 એન્જિન). આ એન્જિન જાવાસ્ક્રિપ્ટના કોડને લાઇન-બાય-લાઇન રીડ કરે છે (JIT ઝમ્પાઇલેશન દ્વારા) અને તેને તાત્કાલિક રન કરે છે.

- JS સિંગલ-થ્રેડેડ (Single-Threaded) છે, એટલે કે તે એક સમયે એક જ કામ કરી શકે છે, પરંતુ તે પોતાના અસિન્ક્રોનસ સ્વભાવના કારણે નેટવર્ક રિક્વેસ્ટ જેવા મોટા કામો બેકગ્રાઉન્ડમાં સોંપીને સાઇટને અટકવા નથી દેતી.

## ૩. Introduction to DOM | How does the Browser Work?

### 🔴 DOM એટલે શું?

**DOM** નું પૂરું નામ **Document Object Model** છે. જ્યારે બ્રાઉઝર કોઈ HTML પેજ લોડ કરે, ત્યારે તે એ આખા પેજને એક ઓબ્જેક્ટના માળખામાં ફેરવી નાખે છે. આ માળખામાં દરેક HTML ટેગ (જેમ કે <div>, <h1>) એક **Node (ઓબ્જેક્ટ)** બની જાય છે. જાવાસ્ક્રિપ્ટ આ DOM ઓબ્જેક્ટની મદદથી જ પેજના કલર કે લખાણ બદલી શકે છે.

### ⚙️ બ્રાઉઝર કેવી રીતે કામ કરે છે? (How Browser Works):

જ્યારે તમે બ્રાઉઝરમાં કોઈ લિંક ઓપન કરો, ત્યારે આખી પ્રક્રિયા ૪ સ્ટેપમાં થાય છે:

- Fetch:** બ્રાઉઝર સર્વર પાસેથી HTML ફાઇલ ડાઉનલોડ કરે છે.
- Parsing:** બ્રાઉઝર HTML કોડને ઉપરથી નીચે લાઇન બાય લાઇન વાંચવાનું શરૂ કરે છે.
- DOM Tree:** વાંચતી વખતે તે બધા ટેગને એક વૃક્ષ જેવા માળખામાં ગોઠવે છે જેને **DOM Tree** કહે છે.
- CSSOM Tree:** સાથે જ તે CSS ફાઇલ પણ ડાઉનલોડ કરીને તેનું પણ એક માળખું બનાવે છે જેને **CSS Object Model (CSSOM)** કહે છે.

## ૪. Render-Tree Construction, Layout and Paint in Browser

એકવાર DOM અને CSSOM બની જાય, પછી બ્રાઉઝર તમારા કમ્પ્યુટરની સ્ક્રીન પર પેજ બતાવવા (Render કરવા) માટે નીચેની પ્રક્રિયા કરે છે:

[HTML] —► [DOM Tree] —┐

└─► [Render Tree] —► [Layout] —► [Paint]

[CSS] —► [CSSOM Tree] —┘

### ⚙️ પ્રક્રિયાનું વિગતવાર લોજિક:

- 1. Render-Tree Construction:** બ્રાઉઝર DOM અને CSSOM બંનેને એકબીજાની ઉપર સુપર-ઇમ્પોઝ કરીને એક **Render Tree** બનાવે છે. આ ટ્રીની અંદર ફક્ત એ જ એલિમેન્ટ હોય છે જે સ્ક્રીન પર સાચે જ દેખાડવાના હોય. (જો કોઈ ટેગમાં display: none; લખેલું હોય, તો તે આ ટ્રીમાંથી બહાર નીકળી જાય છે).
- 2. Layout (Reflow):** આ સ્ટેપમાં બ્રાઉઝર ગણતરી કરે છે કે સ્ક્રીન પર કયો બોક્સ કે ઇમેજ કઈ જગ્યાએ (x અને y કો-ઓર્ડિનેટ્સ) ગોઠવાશે, તેની પહોળાઈ (Width) અને ઊંચાઈ (Height) કેટલા પિક્સલની રહેશે.
- 3. Paint:** આ આખરી સ્ટેપ છે. ગણતરી પૂરી થયા પછી બ્રાઉઝર પ્રોસેસર અને ગ્રાફિક્સ કાર્ડની મદદથી સ્ક્રીન પર અસલી કલર, બોર્ડર, શેડો અને અક્ષરોને ડ્રો (ચિત્રકામ) કરે છે.

## ૫. DOM Traversal (સૌથી મહત્વનો ભાગ)

### 📌 વ્યાખ્યા (Definition):

**DOM Traversal** એટલે જાવાસ્ક્રિપ્ટ કોડિંગની મદદથી HTML ના માળખામાં (વૃક્ષમાં) એક ટેગ પરથી તેની આજુબાજુ, ઉપર કે નીચે રહેલા બીજા ટેગ સુધી પહોંચવું.

### ⚙️ અંદરનું લોજિક (Logic):

રીએક્ટ ઇન્ટરનલી જ્યારે કોઈ કમ્પોનન્ટ બદલે છે, ત્યારે તે DOM ની અંદર આજુબાજુના નોડ્સ ચેક કરે છે. જાવાસ્ક્રિપ્ટમાં મુસાફરી કરવા માટે મુખ્ય ૩ દિશાઓ છે:

- 1. Upwards (ઉપરની તરફ):** કોઈ ચાઇલ્ડ ટેગ પરથી તેના પેરેન્ટ ટેગ પર જવું. (parentNode અથવા parentElement).
- 2. Downwards (નીચેની તરફ):** કોઈ પેરેન્ટ ટેગ પરથી તેની અંદર રહેલા બાળકો સુધી પહોંચવું. (childNodes અથવા children).
- 3. Sideways (આજુબાજુ):** એક જ લેવલ પર રહેલા તેના ભાઈ-બહેન ટેગ પર જવું. (nextElementSibling અથવા previousElementSibling).

### 🖥️ પ્રેક્ટિકલ કોડ ઉદાહરણ:

ધારો કે આપણી પાસે આ પ્રકારનું HTML લિસ્ટ છે:

html

```
<ul id="myList">
```

```
  <li id="item1">પહેલી આઇટમ</li>
```

```
  <li id="item2">બીજી આઇટમ</li>
```

</ul>

Use code with caution.

આ માળખામાં જાવાસ્ક્રિપ્ટથી મુસાફરી કરવાનો સાદો કોડ:

javascript

```
// ૧. બીજી આઇટમને પકડવી
```

```
const item2 = document.getElementById("item2");
```

```
// ૨. Upwards Traversal: બીજી આઇટમ પરથી તેના પેરેન્ટ <ul> સુધી પહોંચવું
```

```
const parentElement = item2.parentElement;
```

```
console.log(parentElement.id); // આઉટપુટ મળશે: "myList"
```

```
// ૩. Sideways Traversal: બીજી આઇટમ પરથી તેની આગળની પહેલી આઇટમ પર જવું
```

```
const firstItem = item2.previousElementSibling;
```

```
console.log(firstItem.textContent); // આઉટપુટ મળશે: "પહેલી આઇટમ"
```

Use code with caution.

---

અહીં આપણો **Stage One** નો પાયો એક-એક પોઇન્ટ સાથે એકદમ ડીપમાં પૂરો થાય છે.

આપણે આ જ રીતે પૂરી ડિટેલમાં **Stage Two (Introduction to ReactJS, Webpack, Bundlers & JSX)** માં ઊંડાણપૂર્વક આગળ વધીએ? જો આ પ્રથમ સ્ટેજના કોઈ લોજિકમાં ડાઉટ હોય તો કહી શકો છો!

ચાલો, હવે આપણે **Stage Two: Introduction to ReactJS**

ના દરેક ટોપિકમાં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ. આપણે દરેક કન્સેપ્ટની અંદરનું અસલી લોજિક, તેની સિન્ટેક્સ અને પ્રોફેશનલ આર્કિટેક્ચર નિયમો વિગતવાર સમજીશું.

---

## Stage Two: Introduction to ReactJS (ઊંડાણપૂર્વક અભ્યાસ)

### ૧. What is React? | Why ReactJS? | Setup - React

#### 🔴 વ્યાખ્યા (Definition):

**React** (અથવા ReactJS) એ યુઝર ઇન્ટરફેસ (UI) એટલે કે વેબસાઇટના ફ્રન્ટએન્ડ પેજીસ અને કમ્પોનન્ટ્સ બનાવવા માટેની એક ઓપન-સોર્સ **JavaScript Library** છે, જે ફ્રેસબુક (Meta) ના સોફ્ટવેર એન્જિનિયર Jordan Walke દ્વારા ૨૦૧૩ માં બનાવવામાં આવી હતી.

#### ⚙️ અંદરનું અસલી લોજિક (Why ReactJS?):

સામાન્ય વેબસાઇટ્સ (Multi-Page Applications) માં જ્યારે તમે કોઈ લિંક પર ક્લિક કરો, ત્યારે બ્રાઉઝર સર્વર પાસેથી આખું નવું HTML પેજ ડાઉનલોડ કરે છે. આના કારણે સ્ક્રીન સફેદ થઈને પેજ રીફ્રેશ થાય છે, જે યુઝર એક્સપિરિયન્સ બગાડે છે.

- React આખી એપ્લિકેશનને **Single Page Application (SPA)** માં ફેરવી દે છે. સાઇટ પર માત્ર એક જ વાર index.html લોડ થાય છે. ત્યારબાદ યુઝર ગમે ત્યાં ક્લિક કરે, બ્રાઉઝર ક્યારેય સર્વર પાસે નવું પેજ લેવા નથી જતું, પણ રીએક્ટ જાવાસ્ક્રિપ્ટની મદદથી લાઇવ સ્ક્રીન પરનો ડેટા અને દેખાવ તુરંત બદલી નાખે છે. આનાથી યુઝરને એક સ્મૂથ મોબાઇલ એપ જેવો અનુભવ વેબસાઇટ પર મળે છે.

### સેટઅપ (Setup - The Modern Way):

આજના સમયમાં પ્રોફેશનલ ડેવલપર્સ રીએક્ટ સેટઅપ કરવા માટે **Vite** બંડલરનો ઉપયોગ કરે છે, કારણ કે તે ખૂબ જ લાઇટવેઇટ અને સુપર ફાસ્ટ છે.

bash

# ૧. તમારા ટર્મિનલમાં નવી એપ ક્રિએટ કરવાનો કમાન્ડ

```
npm create vite@latest my-react-app -- --template react
```

# ૨. પ્રોજેક્ટ ફોલ્ડરની અંદર જાઓ

```
cd my-react-app
```

# ૩. બધા ડિપેન્ડન્સી પેકેજ ઇન્સ્ટોલ કરો

```
npm install
```

# ૪. લોકલ ડેવલપમેન્ટ સર્વર ચાલુ કરો

```
npm run dev
```

Use code with caution.

---

## ૨. Structure of ReactJS Projects

જ્યારે તમે પ્રોજેક્ટ VS Code માં ઓપન કરશો, ત્યારે તેનું પ્રોફેશનલ માળખું અને દરેક ફાઇલનું અંદરનું લોજિક આ પ્રમાણે ગોઠવાયેલું હશે:

- **node\_modules/**: આ ફોલ્ડરમાં રીએક્ટ અને તેના સહાયક સોફ્ટવેર્સનો અસલી કોડ ડાઉનલોડ થઈને સેવ થાય છે. આ ફોલ્ડરને ક્યારેય મેન્યુઅલી અડવાનું હોતું નથી.
- **package.json**: આ પ્રોજેક્ટની કન્ફિગરેશન ફાઇલ છે, જેમાં પ્રોજેક્ટનું નામ, વર્ઝન અને કયા-કયા ટૂલ્સ (Dependencies) ઇન્સ્ટોલ છે તેનું આખું લિસ્ટ સચવાય છે.

- **index.html:** આ તમારી આખી વેબસાઇટનું એકમાત્ર મુખ્ય HTML પેજ છે. તેની અંદર એક જ સિંગલ ટેગ હોય છે: `<div id="root"></div>`. રીએક્ટ આખા પ્રોજેક્ટનો કોડ આ એક જ ડિવિઝનની અંદર ઇન્જેક્ટ કરે છે.
- **src/ (Source Folder):** આ આપણો અસલી વર્કિંગ એરિયા છે.
  - **main.jsx:** આ રીએક્ટનો મુખ્ય દરવાજો (Entry Point) છે. તે તમારા App.jsx નામના કમ્પોનન્ટને પકડીને ઓરિજિનલ HTML ના `id="root"` ની સાથે લોક કરી દે છે.
  - **App.jsx:** આ આપણો મુખ્ય રૂટ કમ્પોનન્ટ (Root Component) છે. તમે સ્ક્રીન પર જે પણ પહેલું પેજ કે ડિઝાઇન જુઓ છો તે આ ફાઇલમાંથી જ કન્ટ્રોલ થાય છે.

### 3. Webpack - The Role of Package Managers or Bundlers

#### ❖ વ્યાખ્યા અને ભૂમિકા (The Role of Bundlers):

જ્યારે આપણે એક મોટો રીએક્ટ પ્રોજેક્ટ બનાવીએ, ત્યારે આપણી પાસે ૧૫૦ અલગ-અલગ જાવાસ્ક્રિપ્ટ ફાઇલો, ૫૦ CSS ફાઇલો અને ઘણા બધા ફોટા (Images) હોય છે. બ્રાઉઝર જો આ બધી જ ફાઇલોને એકપછી એક ઇન્ટરનેટ પરથી ડાઉનલોડ કરવા બેસે, તો સાઇટ લોડ થવામાં મિનિટો લાગી જાય.

**Webpack** (અથવા આજના સમયમાં **Vite / Esbuild**) એક અદ્ભુત બંડલર (Module Bundler) છે. તે તમારા આખા પ્રોજેક્ટની બધી જ ફાઇલોની વચ્ચે કયું જોડાણ છે તે ચેક કરે છે (Dependency Graph બનાવે છે) અને તે બધા કોડને ભેગો કરીને માત્ર **૧ કે ૨ નાની સિંગલ કમ્પ્રેસ ફાઇલો** માં પેક (Bundle) કરી દે છે, જેથી તમારી સાઇટ સેકન્ડના હજારમા ભાગમાં ઇન્ટરનેટ પર ઓપન થઈ જાય છે.

### ૪. JSX in Detail | First React App

#### ❖ વ્યાખ્યા (Definition):

**JSX** નું પૂરું નામ **JavaScript XML** છે. તે રીએક્ટની એક ખાસ સિન્ટેક્સ એક્સટેન્શન (Syntax Extension) છે, જે આપણને જાવાસ્ક્રિપ્ટ ફાઇલોની અંદર જ સીધો **HTML જેવો કોડ લખવાની શક્તિ** આપે છે.

#### ⚙ અંદરનું અસલી લોજિક (How Babel transforms JSX):

તમારું બ્રાઉઝર (Chrome કે Edge) ક્યારેય JSX કોડ સમજી શકતું નથી. બ્રાઉઝર ફક્ત શુદ્ધ જાવાસ્ક્રિપ્ટ જ સમજે છે. આથી, બેકએન્ડમાં **Babel** નામનું એક કમ્પાઇલર ટૂલ આ JSX કોડને રીએક્ટના ઇન-બિલ્ટ ઓબ્જેક્ટ ફંક્શનમાં ફેરવી નાખે છે.

- **તમે જે કોડ લખો છો (JSX):**

```
jsx
```

```
const element = <h1 className="title">હેલો દુનિયા</h1>;
```

Use code with caution.

- **Babel તેને પાછળથી આ રીતે સાદા JavaScript ઓબ્જેક્ટમાં ફેરવે છે:**

```
javascript
```

```
const element = React.createElement('h1', { className: 'title' }, 'હેલો દુનિયા!');
```

Use code with caution.

### પ્રથમ રીએક્ટ એપનો કોડ (App.jsx):

તમારી પ્રથમ એપમાં JSX નો ઉપયોગ કરીને વેરીએબલ અને ગણતરી બતાવવાનો સાચો કોડ આ રીતે લખાશે:

jsx

```
import React from 'react';
```

```
function App() {
```

```
  const academyName = "Desire Infotech";
```

```
  const studentCount = 15;
```

```
  return (
```

```
    // રીએક્ટ ફેગમેન્ટ <> નો ઉપયોગ કર્યો જેથી એકસ્ટ્રા div ન બને
```

```
    <>
```

```
      <div className="container" style={{ padding: '20px', fontFamily: 'sans-serif' }}>
```

```
        /* વેરીએબલ પ્રિન્ટ કરવા માટે Curly Braces {} વાપર્યા */
```

```
        <h1 style={{ color: '#fc8019' }}>{academyName} માં તમારું સ્વાગત છે</h1>
```

```
        <p>આ આપણી પ્રથમ ઓફિશિયલ React એપ્લિકેશન છે.</p>
```

```
        /* {} ની અંદર લાઈવ જાવાસ્ક્રિપ્ટ ગણતરી */
```

```
        <p>કુલ બેચ સાઇઝ: {studentCount + 5} વિદ્યાર્થીઓ</p>
```

```
      </div>
```

```
    </>
```

```
  );
```

```
}
```

```
export default App;
```

Use code with caution.

---

અહીં આપણો **Stage Two** ના તમામ ૪ મુખ્ય પોઇન્ટ્સ એકદમ ઊંડાણપૂર્વક, આંતરિક બંડલર લોજિક અને સેટઅપ સાથે પૂરો થાય છે.

આપણે આ જ સીક્વન્સમાં **Stage Three: ReactJS Components (Thinking in React, Lifecycle, Stateless v/s Stateful)** માં એકદમ ડીપમાં આગળ વધીએ? જો આ સ્ટેજમાં કોઈ કન્ફ્યુઝન હોય તો કહી શકો છો!

ચાલો, હવે આપણે **Stage Three: REACTJS COMPONENTS** ના દરેક ટોપિકમાં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ. આ સ્ટેજ રીએક્ટના આર્કિટેક્ચરનો સૌથી મહત્વનો પાયો છે.

## Stage Three: REACTJS COMPONENTS (ઊંડાણપૂર્વક અભ્યાસ)

### ૧. Intro to React Components | Thinking in React

#### 📌 વ્યાખ્યા (Definition):

**Component** એ વેબસાઇટના પેજનો એક નાનો, સ્વતંત્ર (Independent) અને ફરીથી વાપરી શકાય તેવો **જાવાસ્ક્રિપ્ટ કોડનો ટુકડો (UI Piece)** છે. તે પોતાની અંદરનું લોજિક અને દેખાવ (JSX) જાતે જ સાચવે છે.

#### ⚙️ અંદરનું લોજિક (Thinking in React):

જ્યારે કોઈ પ્રોફેશનલ ડેવલપરને વેબસાઇટની ડિઝાઇન આપવામાં આવે, ત્યારે તે આખા પેજને એકસાથે નથી બનાવતો. તે આર્કિટેક્ચર પ્રમાણે "Thinking in React" એપ્રોચ વાપરે છે:

- Component Hierarchy (માળખું):** આખા પેજને નાના ટુકડાઓમાં વહેંચવું (જેમ કે Navbar, Sidebar, ProductList, ProductCard).
- Reusable Strategy:** જો પેજ પર ૧૦ પ્રોડક્ટ બતાવવાની હોય, તો ૧૦ વાર HTML કોડ લખવાના બદલે એક જ ProductCard કમ્પોનન્ટ બનાવીને લૂપ ફેરવવી.

### ૨. Component Lifecycle | Creating a React Component

#### 📌 કમ્પોનન્ટ લાઇફસાઇકલ (Lifecycle Concept):

જેવી રીતે મનુષ્યનો જન્મ થાય છે, તે મોટો થાય છે અને છેલ્લે મૃત્યુ પામે છે, તેવી જ રીતે બ્રાઉઝરની સ્ક્રીન પર દરેક રીએક્ટ કમ્પોનન્ટની **૩ મુખ્ય લાઈવ અવસ્થાઓ (Phases)** હોય છે:

- Mounting (જન્મ):** જ્યારે કમ્પોનન્ટ પહેલી જ વાર બ્રાઉઝરના અસલી DOM માં લોડ થઈને સ્ક્રીન પર પ્રગટ થાય.
- Updating (વિકાસ):** જ્યારે યુઝર ક્લિક કરે અને સ્ટેટ કે પ્રોપ્સ બદલાવાના કારણે કમ્પોનન્ટ ફરીથી નવો ડેટા સ્ક્રીન પર બતાવે (Re-render થાય).
- Unmounting (મૃત્યુ):** જ્યારે યુઝર બીજા પેજ પર જાય અને આ જૂનો કમ્પોનન્ટ બ્રાઉઝરની સ્ક્રીન પરથી કાયમ માટે ગાયબ (Delete) થઈ જાય.

#### 📁 Creating a React Component (સિન્ટેક્સ):

આધુનિક રીએક્ટમાં કમ્પોનન્ટ બનાવવાની રીત એકદમ સાદી છે — માત્ર એક સાદું જાવાસ્ક્રિપ્ટ ફંક્શન લખો જે JSX રિટર્ન કરતું હોય.

```
jsx
```

```
// કમ્પોનન્ટનું નામ હંમેશા Capital Letter થી જ શરૂ થવું જોઈએ
```

```
function WelcomeCard() {  
  return (  
    <div className="card">  
      <h2>React માં તમારો સ્કોર વધારો!</h2>  
    </div>  
  );  
}
```

export default WelcomeCard;

Use code with caution.

---

### 3. Rendering Multiple Components & Nesting Components

#### ✦ કન્સેપ્ટ (Nesting & Splitting):

**Nesting Components** એટલે એક મોટા કમ્પોનન્ટની અંદર બીજા નાના-નાના યાઇલ્ડ કમ્પોનન્ટ્સ ગોઠવવા. આનાથી આપણો મેઇન કોડ એકદમ ક્લીન રહે છે.

#### 📁 પ્રેક્ટિકલ કોડ આર્કિટેક્ચર:

નીચે જુઓ કેવી રીતે App નામના પેરેન્ટ કમ્પોનન્ટની અંદર આપણે Navbar અને Footer જેવા મલ્ટિપલ કમ્પોનન્ટ્સને એકસાથે નેસ્ટેડ (Nested) કરીને રેન્ડર કર્યા છે:

jsx

```
import React from 'react';
```

```
// યાઇલ્ડ કમ્પોનન્ટ ૧
```

```
function Navbar() {
```

```
  return <nav style={{ background: '#eee', padding: '10px' }}> 🌐 આ નેવબાર કમ્પોનન્ટ છે</nav>;
```

```
}
```

```
// યાઇલ્ડ કમ્પોનન્ટ ૨
```

```
function Footer() {
```

```
  return <footer style={{ marginTop: '20px', color: 'gray' }}> © આ ફૂટર કમ્પોનન્ટ છે</footer>;
```

```
}
```

```
// મુખ્ય પેરેન્ટ કમ્પોનન્ટ (App)
```

```
function App() {  
  return (  
    <>  
    <Navbar /> { /* નેસ્ટિંગ કમ્પોનન્ટ ૧ */}  
    <main style={{ padding: '20px' }}>  
      <h1>મુખ્ય વેબ પેજ કન્ટેન્ટ</h1>  
      <p>અહીંયા તમારું અસલી લોજિક આવશે.</p>  
    </main>  
    <Footer /> { /* નેસ્ટિંગ કમ્પોનન્ટ ૨ */}  
  </>  
);  
}
```

export default App;

Use code with caution.

## ૪. Comparing Stateless and Stateful Components

રીએક્ટ ડેવલપમેન્ટમાં કમ્પોનન્ટ્સને તેમની શક્તિના આધારે બે ભાગમાં સરખાવવામાં (Compare) આવે છે:

| લક્ષણ      | Stateless Component (સાદો)                                  | Stateful Component (પાવરફુલ)                                  |
|------------|-------------------------------------------------------------|---------------------------------------------------------------|
| બીજું નામ  | Presentational Component                                    | Container Component                                           |
| લોકલ મેમરી | આની પાસે પોતાની કોઈ અંગત મેમરી (State) નથી હોતી.            | આની પાસે પોતાની લાઈવ મેમરી (useState) હોય છે.                 |
| કામ        | તે ફક્ત બહારથી મળેલા ડેટાને સ્ક્રીન પર સુંદર રીતે બતાવે છે. | તે યુઝરના કનકશન, ઇનપુટ અને લાઈવ ડેટામાં ફેરફાર હેન્ડલ કરે છે. |
| જટિલતા     | લખવામાં અને ટેસ્ટ કરવામાં એકદમ સાદો અને ટૂંકો હોય છે.       | ડેટા બદલાતા જ આખો કમ્પોનન્ટ રી-રેન્ડર કરવાની ક્ષમતા ધરાવે છે. |

## ૫. Converting Stateless to Stateful Components

🔴 લોજિક (The Logic of Conversion):

જ્યારે આપણી પાસે કોઈ સાદો કમ્પોનન્ટ હોય (જેમ કે લાઈક્સનો આંકડો બતાવતો બોક્સ), અને ભવિષ્યમાં યુઝર ક્લિક કરે ત્યારે એ આંકડો લાઈવ વધારવો હોય, ત્યારે આપણે તેની અંદર **useState Hook** ઉમેરીને તેને સ્ટેટવેસમાંથી **Stateful** માં કન્વર્ટ કરવો પડે છે.

#### પ્રેક્ટિકલ રૂપાંતરણ (Conversion Code):

jsx

```
import { useState } from 'react';
```

```
function LikeButton() {
```

```
  // સાદા કમ્પોનન્ટને સ્ટેટ આપીને શક્તિશાળી (Stateful) બનાવ્યો
```

```
  const [likes, setLikes] = useState(0);
```

```
  return (
```

```
    <div style={{ padding: '20px' }}>
```

```
      <p>કુલ લાઈક્સ: {likes}</p>
```

```
      /* ક્લિક થતાં જ લાઈવ સ્ટેટ બદલાશે અને કમ્પોનન્ટ રી-રેન્ડર થશે */
```

```
      <button onClick={() => setLikes(likes + 1)}> 👍 લાઈક કરો</button>
```

```
    </div>
```

```
  );
```

```
}
```

```
export default LikeButton;
```

Use code with caution.

---

#### ૬. One Component Per File (પ્રોટોકોલ નિયમ)

##### કન્સેપ્ટ:

મોટા ઇન્ડસ્ટ્રી પ્રોજેક્ટ્સમાં ક્યારેય પણ એક જ ફાઇલની અંદર ૪ કે ૫ કમ્પોનન્ટ્સ સાથે નથી લખાતા. નિયમ છે: **"One Component Per File"**.

- દરેક કમ્પોનન્ટની પોતાની એક અલગ સ્વતંત્ર ફાઇલ હોવી જોઈએ (જેમ કે `Navbar.jsx`, `Button.jsx`).
  - ફાઇલના છેલ્લે તેને **export default ComponentName**; કરવો પડે છે, જેથી પ્રોજેક્ટની કોઈપણ બીજી ફાઇલમાં તેને સરળતાથી **import** કરીને વાપરી શકાય. આનાથી પ્રોજેક્ટ મેનેજ કરવો સુપર ઈઝી બની જાય છે.
-

અહીં આપણો **Stage Three** ના તમામ કન્સેપ્ટ્સ એકદમ ઊંડાણપૂર્વક અને બેસ્ટ કોડિંગ પ્રેક્ટિસ સાથે પૂરા થાય છે.

આપણે આ જ રીતે કમ્પીટ ડીટેલમાં **Stage Four: How ReactJS Works Internally (React's DOM Updating Strategy & Higher Order Components)** માં ઊંડાણપૂર્વક આગળ વધીએ?

ચાલો, હવે આપણે **Stage Four: HOW REACTJS WORKS INTERNALLY** માં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ. આ પ્રકરણમાં આપણે રીએક્ટના અંદરના એન્જિનિયરિંગ, તે કઈ રીતે અલ્ગોરિધમનો ઉપયોગ કરીને વેબ પેજ અપડેટ કરે છે અને એડવાન્સ આર્કિટેક્ચર કન્સેપ્ટ્સને લાઇન બાય લાઇન વિગતવાર સમજીશું.

## Stage Four: How ReactJS Works Internally (ઊંડાણપૂર્વક અભ્યાસ)

### ૧. How React Updates the App & Component Tree

#### ❖ વ્યાખ્યા (Definition):

જ્યારે કોઈ રીએક્ટ એપ્લિકેશન રન થાય છે, ત્યારે તે બધા જ કમ્પોનન્ટ્સને એક વૃક્ષ જેવા માળખામાં ગોઠવે છે, જેને **Component Tree** કહેવામાં આવે છે. આ ટ્રીની શરૂઆત મુખ્ય પેરેન્ટ કમ્પોનન્ટ (App) થી થાય છે અને નીચે યાદેલ કમ્પોનન્ટ્સ સુધી જાય છે.

#### ⚙ અંદરનું અસલી લોજિક (Logic):

રીએક્ટમાં જ્યારે પણ કોઈ ચોક્કસ કમ્પોનન્ટની અંદર ડેટા (State કે Props) બદલાય છે, ત્યારે રીએક્ટ આખી વેબસાઇટના પેજને ફરીથી તોડીને નથી બનાવતું.

- તે કમ્પોનન્ટ ટ્રીમાં બરાબર એ જ જગ્યા (Node) પર જાય છે જ્યાં ફેરફાર થયો છે.
- તે એ કમ્પોનન્ટ અને તેની નીચે જોડાયેલા તમામ યાદેલ કમ્પોનન્ટ્સને જ ફરીથી એકિઝક્યુટ (Re-render) કરે છે, જ્યારે ટ્રીની આજુબાજુ રહેલા બાકીના કમ્પોનન્ટ્સ એકદમ સુરક્ષિત અને અસ્પર્શિત રહે છે.

### ૨. Understanding React's DOM Updating Strategy (Virtual DOM નો જાદુ)

રીએક્ટની વેબ પેજ અપડેટ કરવાની સ્ટ્રેટેજી દુનિયામાં સૌથી બેસ્ટ ગણાય છે, કારણ કે તે બ્રાઉઝરના ભારે અને સ્વો ઓરિજિનલ DOM નો સીધો ઉપયોગ નથી કરતી. તે **Virtual DOM** આર્કિટેક્ચર પર કામ કરે છે.

#### ⚙ અંદરની ૩ સ્ટેપની પ્રક્રિયા (The Strategy):

૧. **The Blueprint (વર્ચ્યુઅલ ડોમ):** રીએક્ટ બ્રાઉઝરના ઓરિજિનલ DOM ની એક એકદમ હલકી (Lightweight) ડુપ્લિકેટ નકલ મેમરીમાં (RAM માં) ઓબ્જેક્ટ સ્વરૂપે સાચવી રાખે છે.

૨. **Re-rendering & Diffing:** જ્યારે કોઈ સ્ટેટ બદલાય, ત્યારે રીએક્ટ મેમરીમાં એક નવો આખો વર્ચ્યુઅલ ડોમ બનાવે છે. હવે તેની પાસે બે વર્ચ્યુઅલ નકલો હોય છે: એક જૂની અને એક નવી. રીએક્ટ આ બંને નકલો વચ્ચે શું ફેરફાર થયો છે તે ચેક કરવા માટે એક ખાસ અલ્ગોરિધમ વાપરે છે, જેને **Diffing Algorithm (Reconciliation)** કહે છે.

૩. **Reconciliation (Reconciliation & Patching):** આ અલ્ગોરિધમ ઓપ્ટિમાઇઝ્ડ  $O(n)$  ટાઇમ કોમ્પ્લેક્સિટી પર કામ કરે છે. તે જૂના અને નવા માળખાને સરખાવીને ફક્ત એટલો જ ભાગ (જેમ કે કોઈ એક સિંગલ ટેક્સ્ટ કે

બટનનો કલર) શોધી કાઢે છે જે બદલાયો છે. છેલ્લે, તે આખા પેજને રીફ્રેશ કર્યા વગર ફક્ત એ જ બદલાયેલા ટુકડાને ઓરિજિનલ બ્રાઉઝરના DOM માં જઈને અપડેટ કરે છે. આ પ્રક્રિયાને **Patching** કહેવાય છે. આના લીધે જ રીએક્ટ સુપર ફાસ્ટ પર્ફોર્મન્સ આપે છે.

### 3. Understanding Higher Order Components (HOC)

#### 📌 વ્યાખ્યા (Definition):

**Higher-Order Component (HOC)** એ રીએક્ટમાં કોડને વારંવાર ન લખવો પડે (Code Reusability) તે માટેની એક ખૂબ જ પાવરફુલ એડવાન્સ ડિઝાઇન પેટર્ન છે. તે પોતે કોઈ કમ્પોનન્ટ નથી, પણ તે જાવાસ્ક્રિપ્ટનું એક એવું સ્પેશિયલ ફંક્શન છે જે **એક આખા કમ્પોનન્ટને ઇનપુટ (પેરામીટર) તરીકે લે છે અને તેની અંદર નવો ફીચર ઉમેરીને એક નવો સુધારેલો કમ્પોનન્ટ પાછો (Return) આપે છે.**

#### ⚙️ અંદરનું લોજિક (The HOC Logic):

આને જાવાસ્ક્રિપ્ટના **Higher-Order Functions** (જેમ કે એક ફંક્શન બીજા ફંક્શનને આર્ગ્યુમેન્ટમાં લે છે) ના કન્સેપ્ટ પરથી બનાવવામાં આવ્યું છે.

- **રિયલ વર્લ્ડ ઉદાહરણ:** ધારો કે તમારી પાસે ૫ અલગ-અલગ પેજીસના કમ્પોનન્ટ્સ છે અને તમે ઇચ્છો છો કે યુઝર લોગઇન હોય તો જ આ પેજ દેખાય. દરેક પેજમાં લોગઇન ચેક કરવાનો કોડ વારંવાર લખવાના બદલે તમે એક `withAuth` નામનું HOC ફંક્શન બનાવી શકો છો જે બધા પેજને ઓટોમેટિકલી પ્રોટેક્ટેડ કરી દેશે.

#### 📦 પ્રોફેશનલ કોડ સિન્ટેક્સ અને ઉદાહરણ:

નીચે આપણે એક એવું HOC ફંક્શન બનાવીશું જે કોઈપણ કમ્પોનન્ટને ઉપરથી સુંદર બોર્ડર અને શેડો (Styling) આપવાનું કામ કરશે:

```
jsx
```

```
import React from 'react';
```

```
// ૧. આ આપણું Higher-Order Component ફંક્શન છે
```

```
function withShadowCard(WrappedComponent) {
```

```
  // આ ફંક્શન અંદરથી એક નવો મોડિફાઇ થયેલો કમ્પોનન્ટ પાછો આપશે
```

```
  return function(props) {
```

```
    return (
```

```
      <div style={{ boxShadow: '0 8px 16px rgba(0,0,0,0.1)', border: '1px solid #eee', padding: '20px',  
borderRadius: '12px', margin: '15px' }}>
```

```
        /* અસલી કમ્પોનન્ટ તેના બધા જ પ્રોપ્સ સાથે અહીં રેન્ડર થશે */
```

```
        <WrappedComponent {...props} />
```

```
      </div>
```

```
);  
};  
}
```

// ૨. એકદમ સાદો સ્ટેટલેસ કમ્પોનન્ટ જેને આપણે મોડિફાય કરવો છે

```
function UserProfile(props) {  
  return <h3> 👤 ડેવલપરનું નામ: {props.name}</h3>;  
}
```

// ૩. HOC નો ઉપયોગ કરીને કમ્પોનન્ટને શક્તિશાળી (સુધારેલો) બનાવ્યો

```
const EnhancedProfileCard = withShadowCard(UserProfile);
```

// હવે આ EnhancedProfileCard ને તમે મેઇન App માં સાદા કમ્પોનન્ટ તરીકે ગમે ત્યાં વાપરી શકો છો:

```
// <EnhancedProfileCard name="રોહિલ" />
```

```
export default EnhancedProfileCard;
```

Use code with caution.

### 🧠 આ કોડની મુખ્ય ખૂબી:

- UserProfile કમ્પોનન્ટ એકદમ સાદો હતો, પણ તેને withShadowCard માંથી પાસ કરવાના કારણે તેને પોતાની આજુબાજુ ઓટોમેટિકલી એક સુંદર શેડો વાળો બોક્સ (Card Layout) મળી ગયો. આને કહેવાય એડવાન્સ કોડ રીયુઝેબિલિટી.

---

અહીં આપણો **Stage Four** ના તમામે તમામ અંદરના ઇન્ટરનલ એન્જિનિયરિંગ ટોપિક્સ એકદમ ઊંડાણપૂર્વક અને પ્રોફેશનલ HOC સિન્ટેક્સ લોજિક સાથે પૂરા થાય છે.

આપણે આ જ રીતે કમ્પીટ ડીપ ડિટેલમાં **Stage Five: Props, Events and States (Tracking Data & Stateful Architecture)** માં આગળ વધીએ? જો કોઈ મુદ્દામાં સવાલ હોય તો કહી શકો છો!

ચાલો, હવે આપણે

**Stage Five: PROPS, EVENTS AND STATES** ના દરેક ટોપિકમાં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ. આ પ્રકરણ રીએક્ટની અંદર ડેટા ફ્લો, યુઝર ઇન્ટરેક્શન અને ડાયનેમિક આર્કિટેક્ચર બનાવવા માટે ખૂબ જ મહત્વનું છે.

---

## Stage Five: Props, Events and States (ઊંડાણપૂર્વક અભ્યાસ)

### ૧. Intro to Props & Props in Action

## ❖ વ્યાખ્યા (Definition):

**Props** નું પૂરું નામ **Properties** છે. તે રીએક્ટ કમ્પોનન્ટના ઇનપુટ પેરામીટર્સ છે. જેવી રીતે આપણે સાદા જાવાસ્ક્રિપ્ટ ફંક્શનમાં કોઈ આર્ગ્યુમેન્ટ કે કિંમત પાસ કરીએ છીએ, તેવી જ રીતે એક પેરેન્ટ કમ્પોનન્ટમાંથી બીજા યાઇલ્ડ કમ્પોનન્ટમાં ડેટા મોકલવા માટે Props નો ઉપયોગ થાય છે.

## ⚙ અંદરનું અસલી લોજિક (The Logic):

- **Uni-directional Data Flow:** રીએક્ટમાં પ્રોપ્સ હંમેશા એક જ દિશામાં વહે છે — ઉપરથી નીચે (Parent થી Child તરફ). યાઇલ્ડ કમ્પોનન્ટ ક્યારેય પછા પોતાના પેરેન્ટને ડાયરેક્ટ પ્રોપ્સ મોકલી શકતો નથી.
- **Immutable (Read-Only):** પ્રોપ્સ પ્રકૃતિથી એકદમ અપરિવર્તનીય (Immutable) હોય છે. યાઇલ્ડ કમ્પોનન્ટ તેને ફક્ત વાંચી અને સ્ક્રીન પર બતાવી શકે છે, તેની કિંમતમાં જાતે ફેરફાર કરી શકતો નથી. જો તેને બદલવાનો પ્રયાસ કરવામાં આવે તો રીએક્ટ એરર આપે છે.

## 📺 Props in Action (પ્રેક્ટિકલ કોડ):

જૂના રીએક્ટ વર્ઝનમાં આખી એપ્લિકેશનના મુખ્ય રૂટ પર ગ્લોબલ ડેટા સેટ કરવા માટે ડાયરેક્ટ `ReactDOM.render(<App name="Desire" />, document.getElementById('root'))` ની અંદર પ્રોપ્સ પાસ કરવામાં આવતી હતી. આજના આધુનિક આર્કિટેક્ચરમાં આપણે તેને કમ્પોનન્ટ લેવલ પર આ રીતે વાપરીએ છીએ:

jsx

// આ આપણો યાઇલ્ડ કમ્પોનન્ટ છે (StudentCard.jsx)

```
function StudentCard(props) {  
  return (  
    <div style={{ padding: '10px', border: '1px solid #ccc', margin: '10px' }}>  
      /* props ઓબ્જેક્ટમાંથી નામ અને કોર્સ વાંચી લીધા */  
      <h3> 🎓 વિદ્યાર્થી: {props.name}</h3>  
      <p> 📖 કોર્સ: {props.course}</p>  
    </div>  
  );  
}
```

`export default StudentCard;`

Use code with caution.

---

## ૨. Intro to Events & Events in Action

## ❖ વ્યાખ્યા (Definition):

વેબસાઇટ પર યુઝર દ્વારા થતી કોઈપણ હિલચાલ કે પ્રવૃત્તિને (જેમ કે બટન પર ક્લિક કરવું, ઇનપુટ બોક્સમાં ટાઇપ કરવું, ફોર્મ સબમિટ કરવું) **Event** કહેવાય છે.

### ⚙️ અંદરનું અસલી લોજિક (Synthetic Events):

રીએક્ટ બ્રાઉઝરના ઓરિજિનલ ઇવેન્ટ્સને સીધા રન નથી કરતું. તે તેની ઉપર પોતાનું એક લેયર બનાવે છે જેને **SyntheticEvent** કહેવાય છે.

- આનો ફાયદો એ છે કે તમારો ઇવેન્ટ હેન્ડલિંગ કોડ દુનિયાના બધા જ બ્રાઉઝર્સ (Chrome, Safari, Edge) માં એકસમાન અને સુપર સ્મૂથ પર્ફોર્મન્સ આપે છે.
- નિયમ મુજબ, રીએક્ટમાં બધી ઇવેન્ટ્સ **camelCase** માં લખાય છે (જેમ કે onClick, onChange) અને તેની અંદર ફંક્શનનું નામ ક્લી બ્રેકેટ્સ {} માં લખવામાં આવે છે.

### 🖥️ Events in Action (પ્રેક્ટિકલ કોડ):

jsx

```
function AlertButton() {  
  const handleAction = (username) => {  
    alert('હેલો {username}, ઇવેન્ટ સફળતાપૂર્વક ટ્રિગર થઈ ગઈ છે!');  
  };  
  
  return (  
    // પેરામીટર વાળું ફંક્શન હોવાથી એરો ફંક્શન () => નો ઉપયોગ કર્યો  
    <button onClick={() => handleAction("રાહુલ મોદી")}>  
      🚀 લાઈવ ટેસ્ટ કરો  
    </button>  
  );  
}
```

Use code with caution.

---

## 3. What is State & Difference Between Props and State

### 📌 વ્યાખ્યા (Definition):

**State** એ રીએક્ટ કમ્પોનન્ટની અંદર રહેલી તેની પોતાની એક અંગત અને લાઈવ લોકલ મેમરી (Built-in Object) છે. તેનો ઉપયોગ એવા ડેટાને સાચવવા માટે થાય છે જે યુઝરના ક્લિક કે ઇન્ટરેક્શનના કારણે ભવિષ્યમાં ગમે ત્યારે બદલાઈ શકે છે.

### 🌈 Props અને State વચ્ચેની સીધી સરખામણી:

| સરખામણીના મુદ્દા        | Props (પ્રોપ્સ)                                   | State (સ્ટેટ)                                        |
|-------------------------|---------------------------------------------------|------------------------------------------------------|
| સોર્સ (ક્યાંથી આવે છે?) | બહારથી પેરેન્ટ કમ્પોનન્ટ દ્વારા મોકલવામાં આવે છે. | કમ્પોનન્ટની અંદર જ પોતાની જાતે બનાવવામાં આવે છે.     |
| બદલી શકાય? (Modifiable) | ના, તે <b>Immutable (Read-Only)</b> છે.           | હા, તે <b>Mutable</b> છે, તેને બદલી શકાય છે.         |
| મુખ્ય ઉપયોગ             | કમ્પોનન્ટને ડાયનેમિક ડેટા પાસ કરવા માટે.          | યુઝરના ઇનપુટ અને લાઈવ ફેરફાર સાચવવા માટે.            |
| રી-રેન્ડર પ્રક્રિયા     | જો પેરેન્ટ નવી પ્રોપ્સ મોકલે તો જ અપડેટ થાય.      | સ્ટેટ બદલાતા જ આખો કમ્પોનન્ટ આપોઆપ રી-રેન્ડર થાય છે. |

## ૪. Adding State to Component & Track Changing Data

### ⚙️ અંદરનું લોજિક (Tracking Data):

ફંક્શનલ કમ્પોનન્ટમાં સ્ટેટ ઉમેરવા માટે આપણે **useState Hook** નો ઉપયોગ કરીએ છીએ. જ્યારે યુઝર કોઈ ટેક્સ્ટ બોક્સમાં ટાઇપ કરે, ત્યારે આપણે `onChange` ઇવેન્ટની મદદથી તેના એક-એક અક્ષરને લાઈવ ટ્રેક (Track) કરીએ છીએ અને સ્ટેટના સેટર ફંક્શન (`setState`) ને આપી દઈએ છીએ.

### 📁 પ્રેક્ટિકલ ટ્રેકિંગ કોડ:

jsx

```
import { useState } from 'react';
```

```
function LiveInputTracker() {
```

```
  // text નામનો વેરીએબલ અને તેની શરૂઆતની કિંમત ખાલી સ્ટ્રિંગ "" સેટ કરી
```

```
  const [text, setText] = useState("");
```

```
  return (
```

```
    <div style={{ padding: '20px' }}>
```

```
      <h2> 📝 લાઈવ ડેટા ટ્રેકિંગ સિસ્ટમ </h2>
```

```
      /* e.target.value દ્વારા યુઝર જે પણ લખશે તે લાઈવ ટ્રેક થશે */
```

```
      <input
```

```
        type="text"
```

```
        value={text}
```

```

    onChange={(e) => setText(e.target.value)}

    placeholder="અહીંયા કંઈક ટાઇપ કરો..."
  />

  /* સ્ટેટ બદલાતા જ નીચે લખાણ આપોઆપ રી-રેન્ડર થઈને લાઈવ દેખાશે */
  <p>તમે ટાઇપ કરેલું લખાણ: <b style={{ color: '#fc8019' }}>{text}</b></p>
</div>

);
}

```

Use code with caution.

#### ૫. Multiple Child Components & Updating State and Removing Component

##### 🔴 કન્સેપ્ટ:

મોટા પ્રોજેક્ટમાં એક જ પેરેન્ટ કમ્પોનન્ટની અંદર ઘણા બધા ચાઇલ્ડ કમ્પોનન્ટ્સ સાથે કામ કરે છે. જ્યારે આપણે સ્ટેટમાં રહેલા કોઈ ડેટાના વિસ્તરે અપડેટ કરીએ કે ડીલીટ કરીએ, ત્યારે રીએક્ટ તેને કઈ રીતે હેન્ડલ કરે છે તેનો એક સુંદર અને સંપૂર્ણ લાઈવ પ્રોગ્રામ નીચે મુજબ છે:

##### 📺 ૧૦૦% વર્કિંગ સ્ટેટફુલ આર્કિટેક્ચર પ્રોગ્રામ (App.jsx):

```

jsx

import { useState } from 'react';

import StudentCard from './StudentCard'; // ઉપર બનાવેલો ચાઇલ્ડ કમ્પોનન્ટ

function App() {
  // ૧. મલ્ટિપલ ડેટા ટ્રેક કરવા માટે એરે ઓફ ઓબ્જેક્ટ્સનું સ્ટેટ બનાવ્યું
  const [students, setStudents] = useState([
    { id: 101, name: "રાહુલ પ્રજાપતિ", course: "Node.js બેકએન્ડ" },
    { id: 102, name: "અજય ચૌહાણ", course: "React.js ફ્રન્ટએન્ડ" },
    { id: 103, name: "વિજય સોલંકી", course: "MERN ફૂલ-સ્ટેક" }
  ]);

  // ૨. કમ્પોનન્ટને વિસ્તમાંથી હટાવવાનો (Remove કરવાનો) ફંક્શન
  const removeStudent = (idTold) => {

```

```

// filter મેથડ એ આઈડી સિવાયના બાકીના બધા વિદ્યાર્થીઓનો નવો એરે બનાવશે
const updatedList = students.filter(student => student.id !== idTold);

// સ્ટેટ અપડેટ થતાં જ રીએક્ટ બ્રાઉઝરમાંથી એ કમ્પોનન્ટને કાયમ માટે અનમાઉન્ટ (Remove) કરી દેશે
setStudents(updatedList);

};

return (
  <div style={{ maxWidth: '500px', margin: '30px auto', fontFamily: 'sans-serif' }}>
    <h2>  Desire Infotech - વર્ણવ સ્ટુડન્ટ પોર્ટલ</h2>
    <p>કુલ એક્ટિવ વિદ્યાર્થીઓ: {students.length}</p>
    <hr />

    {/* .map લૂપ ફેરવીને મલ્ટિપલ ચાઇલ્ડ કમ્પોનન્ટ્સ રેન્ડર કર્યા */}
    {students.map((student) => (
      <div key={student.id} style={{ position: 'relative' }}>
        {/* ચાઇલ્ડ કમ્પોનન્ટને પ્રોપ્સ દ્વારા ડેટા મોકલ્યો */}
        <StudentCard name={student.name} course={student.course} />

        {/* રીમુવ કરવાનું બટન */}
        <button
          onClick={() => removeStudent(student.id)}
          style={{ position: 'absolute', right: '20px', top: '25px', color: 'red', cursor: 'pointer' }}
        >
          ✖ કાઢી નાખો
        </button>
      </div>
    ))}
  </div>
);
}

```

export default App;

Use code with caution.

## 🧠 આ એડવાન્સ સેટઅપમાં શું શીખ્યા?

- **key={student.id}**: જ્યારે આપણે લૂપ ફેરવીને ઘણા બધા યાઇલ્ડ કમ્પોનન્ટ્સ બનાવીએ, ત્યારે રીએક્ટના Virtual DOM ને સમજવા માટે દરેક કાર્ડને એક અનોખો આઈડી આપવો પડે છે જેને key કહેવાય છે. આનાથી રીએક્ટ કન્ફ્યુઝ થયા વગર બરાબર એ જ કાર્ડને ડીલીટ કરે છે જેના પર ક્લિક થયું હોય.

અહીં આપણો **Stage Five** ના તમામે તમામ લોજિકલ કન્સેપ્ટ્સ અને પ્રોપ્સ-સ્ટેટ ટ્રેકિંગનો ડીપ અભ્યાસ પૂરો થાય છે.

આપણે આ જ સીક્વન્સમાં **Stage Six: Styling React App (Dynamic CSS, CSS Modules & Media Queries)** માં એકદમ ઊંડાણપૂર્વક આગળ વધીએ? જો આમાં કોઈ પ્રશ્ન હોય તો કહી શકો છો!

ચાલો, હવે આપણે આગળ વધીએ અને

**Stage Six: STYLING REACT APP** ના દરેક કન્સેપ્ટમાં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ. રીએક્ટ એપ્લિકેશનને સુંદર, રિસ્પોન્સિવ અને પ્રોફેશનલ લુક આપવા માટે આ ચેપ્ટર ખૂબ જ જરૂરી છે.

## Stage Six: Styling React App (ઊંડાણપૂર્વક અભ્યાસ)

### ૧. Setting Styles Dynamically

#### 📌 વ્યાખ્યા (Definition):

**Dynamic Styling** એટલે વેબસાઇટના કોઈ પણ ભાગની CSS પ્રોપર્ટીઝને (ક્લર, સાઇઝ, બેકગ્રાઉન્ડ) ફિક્સ રાખવાના બદલે, જાવાસ્ક્રિપ્ટના વેરીએબલ અથવા કમ્પોનન્ટના State ના આધારે રન-ટાઇમ પર લાઈવ બદલવી.

#### ⚙️ અંદરનું અસલી લોજિક (The Logic):

રીએક્ટમાં ઇનલાઇન સ્ટાઇલ આપવા માટે સાદા HTML ની જેમ સ્ટ્રિંગ (લખાણ) નથી વપરાતું, પરંતુ તેની જગ્યાએ **JavaScript Object** નો ઉપયોગ થાય છે. આના કારણે આપણે ઓબ્જેક્ટની અંદર જ સાદું ટર્નરી ઓપરેટર (?) અથવા શરત મૂકીને CSS ની વેલ્યુ લાઈવ ચેન્જ કરી શકીએ છીએ.

- **નિયમ:** ઇનલાઇન ઓબ્જેક્ટમાં CSS ના નામો **camelCase** માં લખાશે (જેમ કે background-color ના બદલે backgroundColor).

#### 🖥️ પ્રેક્ટિકલ કોડ ઉદાહરણ:

ધારો કે આપણે એક એવો બોક્સ બનાવવો છે, જે સ્ટોક (Stock) ખાલી હોય તો ઓટોમેટિક લાલ કલરનો થઈ જાય અને સ્ટોક હોય તો લીલા કલરનો દેખાય:

jsx

```

import { useState } from 'react';

function DynamicStockCard() {
  const [inStock, setInStock] = useState(true);

  // ૧. જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટ તરીકે ડાયનેમિક સ્ટાઇલ બનાવી
  const cardStyle = {
    padding: '20px',
    margin: '15px 0',
    borderRadius: '8px',
    border: '2px solid',
    // જો શરત સાચી (true) હોય તો લીલો કલર, નહિતર લાલ
    backgroundColor: inStock ? '#e6f4ea' : '#fce8e6',
    borderColor: inStock ? '#137333' : '#c5221f',
    color: inStock ? '#137333' : '#c5221f'
  };

  return (
    <div style={cardStyle}>
      <h3> 📦 પ્રોડક્ટનું સ્ટેટસ</h3>
      <p>હાલની સ્થિતિ: <b>{inStock ? "સ્ટોક ઉપલબ્ધ છે" : "આઉટ ઓફ સ્ટોક"}</b></p>
      <button onClick={() => setInStock(!inStock)}>
        સ્થિતિ બદલો (Toggle)
      </button>
    </div>
  );
}

export default DynamicStockCard;

```

Use code with caution.

---

## ૨. Setting Class Names Dynamically

### ✦ કન્સેપ્ટ (Dynamic Class Logic):

ક્યારેક ઇનવાઇન સ્ટાઇલ આપવાના બદલે આપણી પાસે CSS ફાઇલમાં આખા ક્લાસ બનેલા હોય છે (જેમ કે .active-btn અને .disabled-btn). કોઈ શરતના આધારે આખો ક્લાસ બદલવાની પદ્ધતિને **Dynamic Class Setting** કહેવાય છે. આમાં આપણે JSX ની અંદર ક્લી બ્રેકેટ્સ {} અને ટેમ્પલેટ લિટરલ્સ (Template Literals — Backticks `) નો ઉપયોગ કરીએ છીએ.

### 📁 પ્રેક્ટિકલ કોડ ઉદાહરણ:

jsx

```
import { useState } from 'react';
```

```
import './App.css'; // ધારો કે આ ફાઇલમાં પ્રી-ડિફાઇન ક્લાસ છે
```

```
function ThemeToggle() {
```

```
  const [isDarkMode, setIsDarkMode] = useState(false);
```

```
  return (
```

```
    // બેકટિક્સની અંદર શરત મૂકીને વાઈવ ક્લાસ ચેન્જ કર્યો
```

```
    <div className={`base-card ${isDarkMode ? 'dark-mode' : 'light-mode'}`}>
```

```
      <h2>💡 ડાયનેમિક ક્લાસ ચેન્જર</h2>
```

```
      <button onClick={() => setIsDarkMode(!isDarkMode)}>
```

```
        થીમ બદલો
```

```
      </button>
```

```
    </div>
```

```
  );
```

```
}
```

Use code with caution.

---

## ૩. Using Radium for Media Queries (ઐતિહાસિક સંદર્ભ)

### ✦ કન્સેપ્ટ:

રીએક્ટમાં ઇનવાઇન જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટ સ્ટાઇલ વાપરવાનો એક ગેરફાયદો એ છે કે તેમાં આપણે સાદી રીતે મિડીયા ક્વેરી (@media) કે સ્યુડો-ક્લાસ (ચેમન :hover) વાપરી શકતા નથી.

- **Radium** એ એક લોકપ્રિય થર્ડ-પાર્ટી લાઇબ્રેરી હતી જે સાદી ઇનલાઇન સ્ટાઇલને પાવરફુલ બનાવીને તેમાં આ બધું જ વાપરવાની સગવડ આપતી હતી. જોકે, આજના આધુનિક આર્કિટેક્ચરમાં Radium ના બદલે ડેવલપર્સ **Tailwind CSS** અથવા **Styled-Components (CSS-in-JS)** વાપરવાનું પસંદ કરે છે.

## ૪. Enabling & Using CSS Modules

### 📌 વ્યાખ્યા (Definition):

**CSS Modules** એ રીએક્ટનું એક સુપર-પ્રોફેશનલ ફીચર છે. મોટા પ્રોજેક્ટમાં જ્યારે ઘણા બધા ડેવલપર્સ કામ કરતા હોય, ત્યારે એક કમ્પોનન્ટનો સાદો CSS ક્લાસ (દા.ત. .btn) ભૂલથી બીજા કમ્પોનન્ટના બટનને પણ બગાડી નાખે છે (Global Scope Collision).

CSS Modules આ ક્લાસને આખા પ્રોજેક્ટ માટે ગ્લોબલ રાખવાના બદલે, **ફક્ત એ જ ચોક્કસ કમ્પોનન્ટ પૂરતો લોક (Scoped / Private)** કરી દે છે.

### ⚙️ અંદરનું અસલી લોજિક (How it works inside?):

આ ફીચર વાપરવા માટે સિક્વન્સ અને નિયમ આ પ્રમાણે છે:

1. CSS ફાઇલનું નામ સાદા નામની જગ્યાએ **.module.css** રાખવું ફરજિયાત છે (દા.ત. Navbar.module.css).
2. તેને જાવાસ્ક્રિપ્ટમાં એક ઓબ્જેક્ટ તરીકે ઇમ્પોર્ટ કરાય છે: `import styles from './Navbar.module.css';`
3. બિલ્ડ કરતી વખતે રીએક્ટ બેકએન્ડમાં એ ક્લાસના નામની પાછળ એક અનોખો હેશ કોડ જોડી દે છે (જેમ કે `Navbar_btn__x7y2z`), જેના લીધે તે નામ આખા પ્રોજેક્ટમાં યુનિક બની જાય છે અને ક્યારેય પણ કોઈ અન્ય CSS સાથે અથડાતું નથી.

### 💻 પ્રોફેશનલ કોડ સેટઅપ:

#### પહેલી ફાઇલ — Button.module.css (CSS માળખું):

CSS

```
.successBtn {
  background-color: #137333;
  color: white;
  padding: 10px 20px;
  border-radius: 5px;
  border: none;
  cursor: pointer;
}
```

Use code with caution.

#### બીજી ફાઇલ — MyButton.jsx (કમ્પોનન્ટ કોડ):

jsx

// ઓબ્જેક્ટ તરીકે CSS સ્ટાઇલ્સ ઇમ્પોર્ટ કરી

import styles from './Button.module.css';

function MyButton() {

  return (

    // styles ઓબ્જેક્ટની અંદરથી ક્લાસ એક્સેસ કર્યો

    <button className={styles.successBtn}>

 સેવ કરો (CSS Module)

    </button>

  );

}

export default MyButton;

Use code with caution.

---

#### ૫. Outputting Lists with .map() & Practicing Directives

(નોંધ: સિલેબસ ઇમેજમાં ભૂલથી એન્જ્યુવરના ngFor અને ડાયરેક્ટિવ્સ શબ્દો લખાયેલા છે, રીએક્ટમાં તેનું અસલી મેકેનિઝમ આ પ્રમાણે છે):

##### અંદરનું લોજિક (List Rendering Strategy):

રીએક્ટમાં કોઈપણ ડેટાબેઝ કે એરેના લિસ્ટને સ્ક્રીન પર HTML તરીકે પ્રિન્ટ કરવા માટે જાવાસ્ક્રિપ્ટનું ઇન-બિલ્ટ **.map()** ફંક્શન વપરાય છે. આ ફંક્શન એરેના એક-એક ડેટા પર લૂપ ફેરવે છે અને નવો JSX ટુકડો રિટર્ન કરે છે.

રીએક્ટમાં આ પ્રક્રિયા કરતી વખતે દરેક કન્ટેનરને એક અનોખો અને યુનિક **key** એટ્રિબ્યુટ આપવો ફરજિયાત છે, જેથી Virtual DOM ડેટા અપડેટ કે ડીલીટ કરતી વખતે કન્ફ્યુઝ ન થાય.

##### લિસ્ટ રેન્ડરિંગનો કમ્પ્લીટ કોડ:

jsx

function CourseList() {

  // ડેટાબેઝમાંથી આવેલો ડમી એરે

  const courses = [

    { id: 1, title: "HTML5 & CSS3 Masterclass" },

```

    { id: 2, title: "Modern JavaScript (ES6+)" },
    { id: 3, title: "Tailwind CSS Responsive Design" }
  ];

  return (
    <div style={{ padding: '20px' }}>
      <h2>  ઉપલબ્ધ કોર્સની યાદી</h2>
      <ul>
        { /* .map લૂપ દ્વારા ડેટા આઉટપુટ કર્યો */ }
        {courses.map((course) => (
          // રીએક્ટ વેલિડેશન માટે key આપવી ફરજિયાત છે
          <li key={course.id} style={{ padding: '5px 0', fontSize: '16px' }}>
             {course.title}
          </li>
        ))}
      </ul>
    </div>
  );
}

```

Use code with caution.

## ૬. Mobile Considerations

### ✦ કન્સેપ્ટ:

વેબસાઇટ કમ્પ્યુટર સ્ક્રીનની સાથે-સાથે આજના સમયમાં મોબાઇલ, ટેબ્લેટ અને દરેક પ્રકારના ડિવાઇસ પર પ્રોપર અને રિસ્પોન્સિવ (Responsive) દેખાય તે ખૂબ જ મહત્વનું છે.

રીએક્ટમાં આના માટે નીચેની બાબતો ધ્યાનમાં રાખવામાં આવે છે:

- ઇનલાઇન સ્ટાઇલમાં ફિક્સ પિક્સલ (width: '400px') ના બદલે પર્સન્ટેજ અથવા વ્યુપોર્ટ યુનિટ્સ (width: '100%', maxWidth: '1200px') વાપરવા.
- લેઆઉટ ગોઠવવા માટે CSS **Flexbox** અથવા **Grid Architecture** નો જ ઉપયોગ કરવો જેથી ડેટા સ્ક્રીન સાઇઝ પ્રમાણે આપોઆપ એડજસ્ટ થઈ જાય.

- મોબાઇલ ટય ઇન્ટરેક્શન માટે બટનની સાઇઝ અને પેડિંગ (Padding) પૂરતું રાખવું જેથી યુઝર સરળતાથી ટેપ કરી શકે.

---

અહીં આપણો **Stage Six** ના તમામ સ્ટાઇલિંગ અને લિસ્ટ રેન્ડરિંગના એડવાન્સ ટોપિક્સ એક-એક પોઇન્ટ સાથે એકદમ ઊંડાણપૂર્વક પૂરા થાય છે.

આપણે આ જ સીક્વન્સમાં આગળ વધીને સૌથી લોકપ્રિય નેવિગેશન ટોપિક **Stage Seven: Working with React-Router (Client routing, useNavigate, SearchParams & 404 Pages)** માં એકદમ ડીપમાં આગળ વધીએ?

ચાલો, હવે આપણે આગળ વધીએ અને **Stage Seven: WORKING WITH REACT-ROUTER** ના દરેક ટોપિકમાં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ. સિંગલ પેજ એપ્લિકેશનમાં પ્રોફેશનલ નેવિગેશન માટે આ સ્ટેજ ખૂબ જ પાવરફુલ છે.

---

## Stage Seven: Working with React-Router (ઊંડાણપૂર્વક અભ્યાસ)

### ૧. Intro to React Router & Server v/s Client Routing

#### ❖ વ્યાખ્યા (Definition):

**React Router** (ખાસ કરીને react-router-dom) એ રીએક્ટ એપ્લિકેશન્સમાં મલ્ટિપલ પેજીસ, લિંક્સ અને નેવિગેશન સેટ કરવા માટેનું ઓફિશિયલ અને સ્ટાન્ડર્ડ થર્ડ-પાર્ટી ફ્રેમવર્ક છે.

#### ⚙ અંદરનું અસલી લોજિક (The Routing Logic):

- Server-Side Routing (જૂની રીત):** જ્યારે તમે સાદી વેબસાઇટમાં લિંક પર ક્લિક કરો, ત્યારે બ્રાઉઝર સર્વર પાસે નવી રિક્વેસ્ટ મોકલે છે. સર્વર આખું નવું HTML પેજ પાછું આપે છે, જેનાથી સ્ક્રીન વાઈટ થઈને આખું પેજ રીફ્રેશ થાય છે.
- Client-Side Routing (React રીત):** રીએક્ટ બ્રાઉઝરની History API ને હાઇજેક (Control) કરી લે છે. જ્યારે યુઝર નવી લિંક પર ક્લિક કરે છે, ત્યારે URL નો પાથ બદલાય છે, પણ કમ્પ્યુટર સર્વર પાસે નથી જતું. રીએક્ટ જાવાસ્ક્રિપ્ટ દ્વારા તે લિંક સાથે જોડાયેલા કમ્પોનન્ટને લાઈવ સ્ક્રીન પર બદલી નાખે છે. આનાથી નેવિગેશન એક સેકન્ડમાં થઈ જાય છે.

---

### ૨. Setting Up a 404 & Wildcard Routes

#### ❖ કચ્છેપ્ટ (Catch-All Logic):

જ્યારે યુઝર તમારી વેબસાઇટ પર કોઈ એવી ખોટી URL ટાઇપ કરે જે તમે પ્રોગ્રામમાં બનાવી જ નથી (દા.ત. /xyz), ત્યારે સર્વર કેશ થવાના બદલે તેને સુંદર **"404 - Page Not Found"** નો પેજ બતાવવો જરૂરી છે.

- રીએક્ટ રાઉટરમાં આ કામ કરવા માટે **path="\*" નો ઉપયોગ થાય છે, જેને Wildcard / Catch-All Route** કહેવાય છે. આ લાઇન બધા જ અજાણ્યા રાઉટ્સને ઓટોમેટિકલી પકડી લે છે.

---

### ૩. Linking Between Routes

### ❖ કન્સેપ્ટ:

રીએક્ટમાં ક્યારેય પણ લિંક બનાવવા માટે સાદા HTML નો `<a href=...">` ટેગ વાપરવો નહીં, કારણ કે તે પેજને રીફ્રેશ કરી નાખે છે. તેની જગ્યાએ રીએક્ટ રાઉટરનો `<Link to=...">` અથવા `<NavLink to=...">` કમ્પોનન્ટ જ વાપરવો જોઈએ, જે બ્રાઉઝરનું રીફ્રેશ લોક કરીને ક્લાયન્ટ-સાઇડ રાઉટીંગ ચાલુ રાખે છે.

---

## ૪. Navigating Programmatically (useNavigate)

### ❖ કન્સેપ્ટ:

કેટલીકવાર યુઝર લિંક પર ક્લિક નથી કરતો, પણ કોઈ પ્રક્રિયા પૂરી થાય ત્યારે આપણે તેને કોર્ડિંગ દ્વારા આપોઆપ બીજા પેજ પર મોકલવો (Redirect) પડે છે.

- **દાખલા તરીકે:** યુઝરે લોગિન ફોર્મ સબમિટ કર્યું, ડેટાબેઝમાં લોગિન સાચું પડ્યું, એટલે આપોઆપ તેને /dashboard પેજ પર મોકલી દેવો. આ કામ કરવા માટે રીએક્ટ રાઉટર આપણને **useNavigate Hook** આપે છે.
- 

## ૫. Using Query Parameters (useSearchParams)

### ❖ કન્સેપ્ટ:

જ્યારે URL ની પાછળ વધારાનો ફિલ્ટર ડેટા મોકલવામાં આવે (જેમ કે: `http://localhost:3000/products?category=electronics&sort=asc`), ત્યારે આ પાછળના ડેટાને Query Parameters કહેવાય છે. રીએક્ટમાં આ લાઈવ ડેટા વાંચવા માટે **useSearchParams Hook** નો ઉપયોગ થાય છે, જે સાદા `useState` જેવી જ સિન્ટેક્સ ધરાવે છે.

---

## ૬. Redirecting

### ❖ કન્સેપ્ટ:

જો કોઈ યુઝર ડાયરેક્ટ જૂના પાથ પર જાય, તો તેને ઓટોમેટિક નવા પાથ પર ટ્રાન્સફર કરી દેવાની પ્રક્રિયાને **Redirecting** કહેવાય છે. આ માટે રીએક્ટ રાઉટરમાં `<Navigate to=..." />` કમ્પોનન્ટનો ઉપયોગ થાય છે.

---

## ૭. 📁 સંપૂર્ણ પ્રેક્ટિકલ આર્કિટેક્ચર પ્રોગ્રામ (App.jsx)

આપણે આ બધા જ કન્સેપ્ટ્સને મિક્સ કરીને એક ૧૦૦% વર્કિંગ અને પાવરફુલ રાઉટીંગ પ્રોગ્રામ નીચે મુજબ સેટ કરીએ:

jsx

```
import { BrowserRouter, Routes, Route, Link, useNavigate, useSearchParams, Navigate } from 'react-router-dom';
```

```
// ૧. પેજ કમ્પોનન્ટ્સ
```

```

function Home() {
  const navigate = useNavigate();

  const handleLoginDemo = () => {
    console.log("લોગિન પ્રોસેસ સફળ થઈ...");
    // પ્રોગ્રામેટિકલી નેવિગેશન: આપોઆપ ડેશબોર્ડ પેજ પર મોકલી દેશે
    navigate('/dashboard');
  };

  return (
    <div style={{ padding: '20px' }}>
      <h2>🏠 હોમ પેજ</h2>
      <button onClick={handleLoginDemo} style={{ padding: '8px 15px', cursor: 'pointer' }}>
        લોગિન કરો (Dashboard પર જાઓ)
      </button>
    </div>
  );
}

```

```

function Dashboard() {
  return <div style={{ padding: '20px', color: 'green' }}>🚦 ડેશબોર્ડ પેજ (સિક્યોર લોગિન ઍરિયા)</div>;
}

```

```

function Products() {
  // ક્વેરી પેરામીટર્સ વાંચવા માટેનો હૂક
  const [searchParams, setSearchParams] = useSearchParams();
  const currentCategory = searchParams.get('category') || 'બધા';

  return (
    <div style={{ padding: '20px' }}>

```

```

<h2>🛒 પ્રોડક્ટ્સ લિસ્ટ</h2>

<p>હાલનું ફિલ્ટર: <b>{currentCategory}</b></p>

{/* બટન દબાવવાથી URL બદલાશે: /products?category=mobiles */}

<button onClick={() => setSearchParams({ category: 'mobiles' })}>મોબાઇલ ફિલ્ટર કરો</button>

</div>

);

}

```

```

function Page404() {

  return <div style={{ padding: '20px', color: 'red' }}> ⚠️ 404: આ લિંક ખોટી છે, પેજ મળ્યું નથી!</div>;

}

```

// ૨. મુખ્ય એપ્લિકેશન સેટઅપ

```

function App() {

  return (

    <BrowserRouter>

      {/* ડાબોબલ નેવિગેશન મેનુ */}

      <nav style={{ padding: '15px', background: '#333', display: 'flex', gap: '20px' }}>

        <Link to="/" style={{ color: 'white', textDecoration: 'none' }}>હોમ</Link>

        <Link to="/products" style={{ color: 'white', textDecoration: 'none' }}>પ્રોડક્ટ્સ</Link>

      </nav>

      {/* રાઉટ્સનું આખું કન્ટેનર */}

      <Routes>

        <Route path="/" element={<Home />} />

        <Route path="/dashboard" element={<Dashboard />} />

        <Route path="/products" element={<Products />} />

        {/* Redirecting: જો કોઈ /old-home લખે તો તેને ઓટોમેટિક હોમ (/) પર મોકલી દેશે */}

        <Route path="/old-home" element={<Navigate to="/" />} />

```

```
{/* Wildcard Route: બાકીના બધા જ પોટા પાથ 404 પેજ પર જશે */}
<Route path="*" element={<Page404 />} />

</Routes>

</BrowserRouter>

);
}
```

export default App;

Use code with caution.

---

અહીં આપણો **Stage Seven** નો કમ્પ્લીટ ડાયનેમિક રાઉટર રોડમેપ ઊંડાણપૂર્વક પૂરો થાય છે.

આપણે આ જ રીતે ફૂલ ડીટેલમાં **Stage Eight: Handling Forms in React App (Dynamic Custom Input & Validations)** માં આગળ વધીએ? જો કોઈ કન્સેપ્ટમાં ડાઉટ હોય તો જણાવી શકો છો!

ચાલો, હવે આપણે આગળ વધીએ અને **Stage Eight: HANDLING FORMS IN REACT APP** ના દરેક એડવાન્સ અને ડાયનેમિક ટોપિકમાં એકદમ **Deep (ઊંડાણપૂર્વક)**

જઈએ [image\_5IPcLe.png]. મોટા ઇન્ડસ્ટ્રીયલ પ્રોજેક્ટ્સમાં ફોર્મ આર્કિટેક્ચર અને ઓટોમેશન કેવી રીતે સેટ થાય છે તે આપણે લાઇન બાય લાઇન વિગતવાર સમજીએ.

---

## Stage Eight: Handling Forms in React App (ઊંડાણપૂર્વક અભ્યાસ) [image\_5IPcLe.png]

### ૧. What is Web Form? [image\_5IPcLe.png]

#### 📌 વ્યાખ્યા (Definition):

**Web Form** એ યુઝર પાસેથી ટેક્સ્ટ, ઇમેલ, પાસવર્ડ, ફાઇલો કે ચિકબોક્સ જેવો ડેટા સુરક્ષિત રીતે ઇનપુટ લેવા માટેનું મુખ્ય માળખું છે.

#### ⚙️ અંદરનું અસલી લોજિક (The Logic):

રીએક્ટમાં ફોર્મ હેન્ડલ કરતી વખતે બેકએન્ડ આર્કિટેક્ચરમાં હંમેશા **Controlled Components** પદ્ધતિનો ઉપયોગ થાય છે. આ પદ્ધતિમાં બ્રાઉઝરના ઇનપુટ બોક્સ અને રીએક્ટનું સ્ટેટ મેમરી (useState) એકબીજા સાથે કાયમ માટે કનેક્ટ થઈ જાય છે. યુઝર ગમે તે શબ્દ ટાઇપ કરે, તે સેકન્ડના હજારમા ભાગમાં સ્ટેટમાં સબમિટ થઈ જાય છે, જેથી ડેટા હંમેશા ૧૦૦% રેડી મોડમાં રહે છે.

---

### ૨. Creating a Custom Dynamic Input Component & JS Config [image\_5IPcLe.png]

#### 📌 કન્સેપ્ટ (The Automation Logic):

મોટા પ્રોજેક્ટ્સમાં (જેમ કે સાઇન-અપ પેજ, પ્રોફાઇલ પેજ, ઓર્ડર પેજ) જો આપણે દરેક પેજ માટે અલગ-અલગ HTML ઇનપુટ બોક્સ લખવા બેસીએ, તો કોડ ખૂબ જ મોટો અને ડુપ્લિકેટ થઈ જાય છે.

પ્રોફેશનલ ડેવલપર્સ આ માટે બે ખાસ સ્ટેપ્સ ફોલો કરે છે:

1. **JS Config:** ફોર્મમાં કયા-કયા બોક્સ જોઈએ છે (દા.ત. Name, Email) તેનું આખું લોજિક એક સિંગલ **JavaScript Object** માં કન્ફિગરેશન તરીકે સેવ કરે છે.
2. **Custom Dynamic Input Component:** એક એવો સ્માર્ટ કોમન ઇનપુટ કમ્પોનન્ટ બનાવે છે, જે ઉપરના ઓબ્જેક્ટ કન્ફિગને વાંચીને આપોઆપ ટેક્સ્ટ બોક્સ, પાસવર્ડ બોક્સ કે ડ્રોપડાઉન લિસ્ટ જાતે જ જનરેટ કરી લે છે.

---

### 3. Dynamically Create Inputs, Handling Input & Submission [image\_5IPcLe.png]

#### ⚙ અંદરનું અસલી લોજિક (The Computed Property Magic):

જ્યારે આપણે મલ્ટિપલ ઇનપુટ ઓટોમેટિકલી જનરેટ કરીએ, ત્યારે બધા જ બોક્સ માટે અલગ-અલગ ફંક્શન્સ નથી લખાતા. તેના બદલે જાવાસ્ક્રિપ્ટનો **Computed Property Name ([name]: value)** નો કન્સેપ્ટ વપરાય છે, જે એક જ ફંક્શનથી આખું ફોર્મ ટ્રેક કરી લે છે. સબમિટ કરતી વખતે બ્રાઉઝરનું ઓટો-રીફ્રેશ રોકવા માટે **e.preventDefault()** વપરાય છે.

---

### ૪. Custom Form Validation & Showing Error Messages [image\_5IPcLe.png]

#### 🔴 કન્સેપ્ટ:

યુઝર ખોટો ડેટા સબમિટ ન કરે તે માટે આપણે કસ્ટમ વેલિડેશન નિયમો સેટ કરીએ છીએ (જેમ કે: ઇમેલમાં @ હોવું જોઈએ, પાસવર્ડ ૬ અક્ષરથી મોટો હોવો જોઈએ). જો નિયમ ખોટો પડે, તો આપણે એરર મેસેજનું એક અલગ સ્ટેટ ઓબ્જેક્ટ અપડેટ કરીએ છીએ અને યુઝરને ઇનપુટ બોક્સની બરાબર નીચે લાઇવ લાઇવ અક્ષરમાં એરર મેસેજ ડિસ્પ્લે કરીએ છીએ.

---

### ૫. 📁 સંપૂર્ણ સુપર-પ્રોફેશનલ ડાયનેમિક ફોર્મ પ્રોજેક્ટ (DynamicForm.jsx)

આ દરેક પોઇન્ટ્સ અને ઓટોમેશન કન્સેપ્ટને મિક્સ કરીને બનેલો એક ૧૦૦% વર્કિંગ ઇન્ડસ્ટ્રી લેવલનો પ્રોગ્રામ નીચે મુજબ છે:

#### ૧. થાઇલ્ડ કમ્પોનન્ટ — CustomInput.jsx (સ્માર્ટ ડાયનેમિક બોક્સ):

jsx

```
import React from 'react';
```

```
// આ કમ્પોનન્ટ પ્રોપ્સના આધારે આપોઆપ પોતાનો ટાઇપ બદલી નાખશે
```

```
function CustomInput({ label, type, name, value, onChange, error, options }) {
```

```
  return (
```

```

<div style={{ marginBottom: '15px', textAlign: 'left' }}>
  <label style={{ fontWeight: '600', display: 'block', marginBottom: '5px' }}>{label}</label>

  {/* જો ટાઇપ સેલેક્ટ (ડ્રોપડાઉન) હોય તો */}
  {type === 'select' ? (
    <select name={name} value={value} onChange={onChange} style={{ width: '100%', padding:
'8px', borderRadius: '6px', border: '1px solid #ccc' }}>
      {options.map((opt, i) => <option key={i} value={opt}>{opt}</option>)}
    </select>
  ) : (
    // બાકીના બધા જ નોર્મલ ટેક્સ્ટ, ઇમેલ કે પાસવર્ડ બોક્સ માટે
    <input
      type={type}
      name={name}
      value={value}
      onChange={onChange}
      style={{ width: '100%', padding: '8px', borderRadius: '6px', border: error ? '1px solid red' :
'1px solid #ccc' }}
    />
  )}

  {/* જો એરર હશે તો લાઈવ નીચે લાલ અક્ષરમાં દેખાશે */}
  {error && <span style={{ color: 'red', fontSize: '12px', marginTop: '3px', display: 'block' }}> ⚠️
{error}</span>}
</div>

);
}

```

module.exports = CustomInput;

Use code with caution.

## ૨. મુખ્ય પેરેન્ટ કમ્પોનન્ટ — DynamicForm.jsx (ઓટોમેશન અને કન્ફિગરેશન):

jsx

```

import { useState } from 'react';
import CustomInput from './CustomInput';

function DynamicForm() {
  // ⚙️ સ્ટેપ ૧: ફોર્મનું આખું JS કન્ફિગરેશન (JS Config for Form)
  const formConfig = [
    { label: "પૂરું નામ", type: "text", name: "username" },
    { label: "ઈમેલ એડ્રેસ", type: "email", name: "email" },
    { label: "ગુપ્ત પાસવર્ડ", type: "password", name: "password" },
    { label: "પસંદગીનો કોર્સ", type: "select", name: "course", options: ["React.js", "Node.js", "Tailwind CSS"] }
  ];

  // સ્ટેપ ૨: બધા જ ડેટા સ્ટોર કરવા માટેનું સિંગલ ઓબ્જેક્ટ સ્ટેટ
  const [formData, setFormData] = useState({ username: "", email: "", password: "", course: "React.js" });
  const [errors, setErrors] = useState({});

  // સ્ટેપ ૩: લાઈવ યુઝર ઇનપુટ હેન્ડલ કરવા માટેનું કોમન ફંક્શન
  const handleInputChange = (e) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
  };

  // સ્ટેપ ૪: કસ્ટમ વેલિડેશન ચેક
  const validateForm = () => {
    let tempErrors = {};

    if (!formData.username.trim()) tempErrors.username = "નામ લખવું ફરજિયાત છે!";
    if (!formData.email.includes("@")) tempErrors.email = "ઈમેલ આઈડી ખોટું છે!";
    if (formData.password.length < 6) tempErrors.password = "પાસવર્ડ ઓછામાં ઓછો ૬ અક્ષરનો હોવો જોઈએ!";
  };

```

```

setErrors(tempErrors);

return Object.keys(tempErrors).length === 0; // જો એરર ૦ હોય તો સાચું (true) રિટર્ન કરશે
};

// સ્ટેપ ૫: ફોર્મ સબમિશન હેન્ડલર
const handleSubmit = (e) => {
  e.preventDefault(); // પેજ ઓટો-રીફ્રેશ રોક્યું

  if (validateForm()) {
    console.log("100% સફળતાપૂર્ણ ડેટા સબમિટ થયો:", formData);
    alert(`અભિનંદન ${formData.username}, રજીસ્ટ્રેશન થઈ ગયું છે!`);
  }
};

return (
  <div style={{ maxWidth: '400px', margin: '30px auto', padding: '25px', border: '1px solid #ddd',
borderRadius: '16px', boxShadow: '0 4px 12px rgba(0,0,0,0.05)' }}>
    <h2 style={{ textAlign: 'center', marginBottom: '20px' }}> 📄 Desire Infotech પોર્ટલ </h2>

    <form onSubmit={handleSubmit}>
      /* JS Config પર લૂપ ફેરવીને બધા જ બોક્સ ઓટોમેટિક ડાયનેમિક જનરેટ કર્યા */
      {formConfig.map((config, index) => (
        <CustomInput
          key={index}
          label={config.label}
          type={config.type}
          name={config.name}
          options={config.options}
          value={formData[config.name]}
          onChange={handleInputChange}

```

```

        error={errors[config.name]} // ચોક્કસ બોક્સની એરર પાસ કરી
    />
  )))

  <button type="submit" style={{ width: '100%', padding: '10px', backgroundColor: '#333',
color: 'white', border: 'none', borderRadius: '6px', fontSize: '16px', cursor: 'pointer', marginTop: '10px'
}}>

    સબમિટ કરો

  </button>

</form>

</div>

);
}

```

export default DynamicForm;

Use code with caution.

અહીં આપણો **Stage Eight** ના તમામ એડવાન્સ ડાયનેમિક ફોર્મ કન્સેપ્ટ્સ અને ઓટોમેશન નિયમો એકદમ ઊંડાણપૂર્વક પૂરા થાય છે.

આપણે આ જ રીતે એકદમ ડીપ ડિટેલમાં સૌથી મેઈન અને એડવાન્સ ટોપિક **Stage Nine: REDUX (State management, Need for Redux, Store, Action & Reducer flow)** માં ઊંડાણપૂર્વક આગળ વધીએ? જો કોઈ કન્સેપ્ટમાં પ્રશ્ન હોય તો પૂછી શકો છો!

ચાલો, હવે આપણે આખા સિલેબસના સૌથી એડવાન્સ, પાવરફુલ અને મોસ્ટ ઇમ્પોર્ટન્ટ પ્રકરણ

**Stage Nine: REDUX** માં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ [image\_5IPcLe.png]. મોટા ઇન્ડસ્ટ્રીયલ પ્રોજેક્ટ્સમાં આખી એપ્લિકેશનનો ગ્લોબલ ડેટા સિંક્યોર અને સેન્ટ્રલાઈઝ્ડ કેવી રીતે મેનેજ થાય છે તેનું અસલી આર્કિટેક્ચર આપણે લાઇન બાય લાઇન વિગતવાર સમજીએ.

**Stage Nine: REDUX (ઊંડાણપૂર્વક અભ્યાસ)** [image\_5IPcLe.png]

**૧. Understanding State Management & Its Complexity** [image\_5IPcLe.png]

🔴 **વ્યાખ્યા (Definition):**

**State Management** એટલે એપ્લિકેશનમાં સમય જતાં બદલાતા લાઈવ ડેટા (જેમ કે લોગઈન યુઝરની પ્રોફાઇલ, શોપિંગ કાર્ટની આઇટમ્સ, થીમ સેટિંગ્સ) ને વ્યવસ્થિત રીતે સાચવવાની અને કંટ્રોલ કરવાની પ્રક્રિયા.

⚙️ **અંદરની જટિલતા (The Complexity Logic):**

જ્યારે આપણો પ્રોજેક્ટ નાનો હોય, ત્યારે `useState` અને `Props` થી કામ ચાલી જાય છે. પરંતુ પ્રોજેક્ટ જેમ-જેમ મોટો થાય અને તેમાં સેંકડો કમ્પોનન્ટ્સ આવે, ત્યારે પ્રોપ ડ્રિલિંગ (Prop Drilling) ની કારણે કોડ અત્યંત ગૂંચવાડાભર્યો થઈ જાય છે.

- જો કોઈ એક ખૂણે રહેલા ચાઇલ્ડ કમ્પોનન્ટમાં ફેરફાર થાય, તો તેની અસર કમ્પોનન્ટ ટ્રીના બીજા ભાગમાં ક્યાં થઈ રહી છે તે ટ્રેક કરવું અશક્ય બને છે.
- આના લીધે ડેટા અન-પ્રેડિક્ટેબલ (Unpredictable) થઈ જાય છે, જે એપ્લિકેશનમાં મોટા બગ્સ (Bugs) લાવે છે. આ જ જટિલતાને દૂર કરવા માટે સેન્ટ્રલ સ્ટેટ મેનેજમેન્ટ આર્કિટેક્ચરની જરૂર પડે છે.

---

## ૨. Intro to Redux & The Need for Redux [image\_5IPcLe.png]

### 🔴 Redux શું છે?:

**Redux** એ કોઈ રીએક્ટનો ભાગ નથી, પણ તે કોઈપણ જાવાસ્ક્રિપ્ટ એપ્લિકેશન (React, Angular, કે સાદા JS) માટેનું એક ઓપન-સોર્સ **Global State Management Library** છે. તે Dan Abramov અને Andrew Clark દ્વારા ૨૦૧૫ માં બનાવવામાં આવ્યું હતું.

### 🌀 તેની અસલી જરૂરિયાત (The Core Need):

રીએક્ટમાં ડેટા વહેંચવાની ઝંઝટમાંથી મુક્તિ મેળવવા માટે આની જરૂર છે. Redux તમારી એપ્લિકેશનના બધા જ કમ્પોનન્ટ્સની બહાર એક **ગ્લોબલ સેન્ટ્રલ સોર્સ** બનાવી દે છે. હવે કમ્પોનન્ટ ટ્રી ગમે તેટલું ઊંડું હોય, ડેટા સીધો એક જ મુખ્ય કેન્દ્રમાંથી આવે છે અને ત્યાં જ પાછો જાય છે. આનાથી કોડ એકદમ પ્રિડિક્ટેબલ (Predictable), ટેસ્ટેબલ (Testable) અને ડીબગ કરવામાં એકદમ સરળ બની જાય છે.

---

## ૩. Understanding the Redux Flow & Core Pillars [image\_5IPcLe.png]

Redux મુખ્યત્વે **Unidirectional Data Flow** એટલે કે એક જ દિશામાં ચાલતા ચક્ર પર કામ કરે છે. આ ચક્રના મુખ્ય ૩ પિલર્સ (Pillars) છે જેને સમજવા ખૂબ જ જરૂરી છે:

### A. The Action (શું કરવું છે તેનો ઓર્ડર) [image\_5IPcLe.png]

- લોજિક:** આ એક સાદો જાવાસ્ક્રિપ્ટ ઓબ્જેક્ટ છે જે એ જણાવે છે કે એપ્લિકેશનમાં શું ફેરફાર કરવો છે. કમ્પોનન્ટ ક્યારેય ડેટાબેઝ કે સ્ટોરને સીધો બદલી શકતો નથી, તેણે ફરજિયાત એક એક્શન ફાયર કરવી પડે છે.
- નિયમ:** એક્શન ઓબ્જેક્ટની અંદર `type` પ્રોપર્ટી હોવી ૧૦૦% ફરજિયાત છે, અને વધારાના મોકલાતા ડેટાને `payload` કહેવાય છે.
- સિન્ટેક્સ:**

javascript

```
const addItemAction = {  
  type: 'ADD_TO_CART',  
  payload: { id: 1, name: 'Laptop', price: 50000 }  
};
```

Use code with caution.

### B. The Reducer (અસલી ગણતરી કરતું મશીન) [image\_5IPcLe.png]

- **લોજિક:** આ એક **Pure Function** (શુદ્ધ ફંક્શન) છે. તે હાલનો કરન્ટ ડેટા (currentState) અને આવેલી action ને ઇનપુટ તરીકે લે છે. તે એક્શનના ટાઇપ પ્રમાણે ગણતરી કરીને એકદમ નવો, ફ્રેશ સ્ટેટ ઓબ્જેક્ટ પાછો આપે છે.
- **નિયમ:** રીડ્યુસર ક્યારેય જૂના ડેટાને તોડતો કે મોડિફાય નથી કરતો. તે હંમેશા નવી કોપી બનાવીને જ રિટર્ન કરે છે (Immutability નિયમ).
- **સિન્ટેક્સ:**

javascript

```
function cartReducer(state = { items: [] }, action) {  
  switch (action.type) {  
    case 'ADD_TO_CART':  
      return { ...state, items: [...state.items, action.payload] };  
    default:  
      return state;  
  }  
}
```

Use code with caution.

### C. The Store (મુખ્ય ગ્લોબલ તિજોરી) [image\_5IPcLe.png]

- **લોજિક:** આખી એપ્લિકેશનની એકમાત્ર ઓફિશિયલ મુખ્ય તિજોરી છે. રીડ્યુસરે ગણતરી કરીને મોકલેલો નવો ડેટા આ સ્ટોરની અંદર સેવ થાય છે, અને અહીંયાથી જ બધા કમ્પોનન્ટ્સને ડેટા લાઈવ ડિસ્પે થાય છે.

---

### ૪. Understanding Provider [image\_5IPcLe.png]

#### ◆ કન્સેપ્ટ:

જ્યારે આપણે Redux નો સ્ટોર બનાવી લઈએ, ત્યારે તે રીએક્ટથી અલગ હોય છે. રીએક્ટ પ્રોજેક્ટને આ સ્ટોરની શક્તિઓ આપવા માટે **react-redux** લાઇબ્રેરીનો **<Provider>** કમ્પોનન્ટ વપરાય છે.

- આ કમ્પોનન્ટને આપણા આખા પ્રોજેક્ટના મેઇન રૂટ (main.jsx) ની ઉપર એક કવર (Wrapper) તરીકે લપેટી દેવામાં આવે છે, અને તેમાં પ્રોપ્સ તરીકે સ્ટોર પાસ કરાય છે, જેથી આખી સાઇટમાં ગમે તે ખૂણે બેઠેલો કમ્પોનન્ટ આ ગ્લોબલ ડેટા સાથે લાઈવ કનેક્ટ થઈ શકે છે.

---

### ૫. Redux Flow ડાયાગ્રામ આર્કિટેક્ચર નકશો

જ્યારે પણ કોઈ યુઝર વેબસાઇટ પર બટન દબાવશે ત્યારે Redux નો આખો ફ્લો પાછળ બેકએન્ડમાં આ સિક્વન્સમાં આપોઆપ રન થશે:

[ કમ્પોનન્ટ UI / બટન ક્લિક ]

|

▼ (dispatch()) ફંક્શન દ્વારા

[ Action ઓબ્જેક્ટ ] —► { type: 'ADD\_TO\_CART', payload: data }

|

▼

[ Reducer ફંક્શન ] —► (જૂનો સ્ટેટ + એક્શન વાંચીને નવો ડેટા ગણાશે)

|

▼

[ Global Store ] —► (તિજોરી અપડેટ થશે અને આખા રીએક્ટ UI ને લાઇવ બદલી નાખશે)

આ વન-વે ચક્રના કારણે જ Redux દુનિયાનું સૌથી મજબૂત અને બગ-ફ્રી સ્ટેટ મેનેજમેન્ટ ટૂલ ગણાય છે.

---

અહીં આપણો **Stage Nine** ના તમામ થીયરીટીકલ અને ફ્લો આર્કિટેક્ચરના કન્સેપ્ટ્સ એક-એક પોઇન્ટ સાથે એકદમ ઊંડાણપૂર્વક પૂરા થાય છે.

આપણે આ જ રીતે કમ્પ્લીટ ડીપ ડિટેલમાં આનો અસલી કોડિંગ અને પ્રેક્ટિકલ ભાગ **Stage Ten: REDUX IN ACTION (Switch-Case, Dispatching Actions & Immutably update)** માં ઊંડાણપૂર્વક આગળ વધીએ? જો કોઈ લોજિકમાં પ્રશ્ન હોય તો જણાવો!

ચાલો, હવે આપણે આગળ વધીએ અને

**Stage Ten: REDUX IN ACTION** ના દરેક ટોપિકમાં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ [image\_5IPcLe.png]. રેડક્સ ફ્લોને એપ્લિકેશનની અંદર કોડિંગ દ્વારા કેવી રીતે ઓપરેટ કરવો તે આપણે લાઇન બાય લાઇન વિગતવાર સમજીએ.

---

**Stage Ten: Redux in Action (ઊંડાણપૂર્વક અભ્યાસ) [image\_5IPcLe.png]**

**૧. Setting Up Reducer, Store and Switch-Case [image\_5IPcLe.png]**

✦ **કન્સેપ્ટ:**

રેડક્સમાં સૌથી પહેલો સ્ટેપ ગ્લોબલ સ્ટોર (તિજોરી) બનાવવાનો છે. આ માટે આપણે રેડક્સનું કોર ફંક્શન `createStore` વાપરીએ છીએ અને તેને મુખ્ય મશીન એટલે કે **Reducer** પાસ કરીએ છીએ.

⚙ **Switch-Case નું આંતરિક લોજિક:**

રીડ્યુસર ફંક્શનની અંદર આવેલી એક્શનના type ને સરખાવવા માટે મોટા અને ગૂંચવાડાભર્યા if-else ના બદલે **switch(action.type)** નો ઉપયોગ થાય છે. આ પ્રોફેશનલ ઇન્ડસ્ટ્રી નિયમ છે, જે કોડને એકદમ ક્લીન અને ઇઝી રાખે છે.

---

## ૨. Updating State and Arrays Immutably (... Spread Operator) [image\_5IPcLe.png]

### 🔥 રેડક્સનો સૌથી મોટો કડક નિયમ (The Immutability Rule):

તમે રેડક્સમાં સ્ટેટના ઓરિજિનલ ડેટાને ક્યારેય સીધો મોડિફાય કે ચેન્જ ન કરી શકો.

- ❌ **ખોટી રીત (State Mutation):** `state.count = 5` કે `state.items.push(data)` — આનાથી રીએક્ટ સ્ક્રીનને ક્યારેય રી-રેન્ડર નહીં કરે.
  - સાચી રીત: જાવાસ્ક્રિપ્ટ **Spread Operator (...)** નો ઉપયોગ કરીને હંમેશા ડેટાની એક તદ્દન નવી બ્રાન્ડ-ન્યુ કોપી બનાવીને જ રીડ્યુસરમાંથી return કરવી. આને કહેવાય **Updating State Immutably**.
- 

## ૩. Dispatching Actions & Passing Data with Action (Payload) [image\_5IPcLe.png]

### 🔥 કન્સેપ્ટ:

- Dispatching Actions:** કમ્પોનન્ટની અંદરથી સ્ટોરને એક્શન મોકલવા માટે **dispatch()** ફંક્શનનો ઉપયોગ થાય છે. આ લાઇન રન થતાં જ રીડ્યુસર ફંક્શન ઓટોમેટિક જાગી જાય છે.
  - Payload (Retrieving Data):** જ્યારે આપણે બટન ક્લિક કરીએ ત્યારે ફક્ત એક્શનનું નામ મોકલવું પૂરતું નથી હોતું, તેની સાથે અસલી કિંમત કે ડેટા (જેમ કે પ્રોડક્ટની વિગતો) પણ મોકલવી પડે છે. આ વધારાના ડેટા ઓબ્જેક્ટને રેડક્સ આર્કિટેક્ચરમાં **payload** કહેવાય છે.
- 

## ૪. Connecting React to Redux (Connecting Store to React) [image\_5IPcLe.png]

### 🔥 આધુનિક હુક્સ આર્કિટેક્ચર (The Hooks Way):

જૂના રીએક્ટમાં કમ્પોનન્ટને રેડક્સ સાથે જોડવા માટે બહુ મોટો `connect()` અને `mapStateToProps` વાળો કોડ લખવો પડતો હતો [image\_5IPcLe.png]. આજના આધુનિક સમયમાં **react-redux** આપણને બે પાવરફુલ હૂક આપે છે જે આ કામને એકદમ સરળ બનાવે છે:

- useSelector:** આ ગ્લોબલ રેડક્સ સ્ટોર (તિજોરી) માંથી કોઈપણ ડેટા સીધો કમ્પોનન્ટમાં ખેંચીને લાવે છે.
  - useDispatch:** આ કમ્પોનન્ટની અંદર બેઠા-બેઠા એક્શનને ફાયર (Dispatch) કરવાનો પાવર આપે છે.
- 

## ૫. 📦 સંપૂર્ણ પ્રેક્ટિકલ પ્રોગ્રામ (React-Redux કાઉન્ટર સેટઅપ)

આ બધા જ પ્રેક્ટિકલ કન્સેપ્ટ્સ, switch-case અને useDispatch ને મિક્સ કરીને બનેલો એક સંપૂર્ણ પ્રોગ્રામ નીચે મુજબ છે:

## ૧. રેડક્સ ફાઇલ — store.js (તિજોરી અને રીડ્યુસર સેટઅપ):

javascript

```
const { createStore } = require('redux');
```

```
// શરૂઆતનો ગ્લોબલ ડેટા
```

```
const initialState = {
```

```
  count: 0
```

```
};
```

```
// રીડ્યુસર ફંક્શન (Switch-Case અને Immutability સાથે)
```

```
function counterReducer(state = initialState, action) {
```

```
  switch (action.type) {
```

```
    case 'INCREMENT':
```

```
      // જૂનો સ્ટેટ કોપી કરીને નવી વેલ્યુ રિટર્ન કરી
```

```
      return { ...state, count: state.count + 1 };
```

```
    case 'DECREMENT':
```

```
      return { ...state, count: state.count - 1 };
```

```
    case 'ADD_CUSTOM':
```

```
      // action.payload માંથી મોકલેલો એક્સ્ટ્રા ડેટા રીટ્રીવ (Retrieve) કર્યો
```

```
      return { ...state, count: state.count + action.payload };
```

```
    default:
```

```
      return state;
```

```
  }
```

```
}
```

```
// ગ્લોબલ સ્ટોર ક્રિએટ કર્યો
```

```
const store = createStore(counterReducer);
```

```
module.exports = store;
```

Use code with caution.

## ૨. રીએક્ટ કમ્પોનન્ટ — CounterApp.jsx (useSelector અને useDispatch નો લાઈવ જાદુ):

jsx

```
import React from 'react';
```

```
import { useSelector, useDispatch } from 'react-redux';
```

```
function CounterApp() {
```

```
  // ૧. useSelector ની મદદથી ગ્લોબલ સ્ટોરમાંથી લાઈવ કાઉન્ટ ખેંચી લીધો
```

```
  const currentCount = useSelector((state) => state.count);
```

```
  // ૨. એક્શન મોકલવા માટે ડિસ્પેચ મેળવ્યો
```

```
  const dispatch = useDispatch();
```

```
  return (
```

```
    <div style={{ padding: '30px', textAlign: 'center', fontFamily: 'sans-serif' }}>
```

```
      <h2>  React-Redux લાઈવ કાઉન્ટર પ્રોગ્રામ</h2>
```

```
      { /* સ્ટોરનો ડેટા અહીંયા લાઈવ દેખાશે */ }
```

```
      <h1 style={{ fontSize: '48px', color: '#fc8019' }}>{currentCount}</h1>
```

```
      <div style={{ display: 'flex', gap: '15px', justifyContent: 'center' }}>
```

```
        { /* સાદી એક્શન ડિસ્પેચ કરી */ }
```

```
        <button onClick={() => dispatch({ type: 'INCREMENT' })} style={{ padding: '10px 20px',  
cursor: 'pointer' }}>
```

```
          વધારો (+1)
```

```
        </button>
```

```
        <button onClick={() => dispatch({ type: 'DECREMENT' })} style={{ padding: '10px 20px',  
cursor: 'pointer' }}>
```

```
          ઘટાડો (-1)
```

```
        </button>
```

```

    {/* પેલોડ (Payload) સાથે ડેટા પાસ કરીને એક્શન ડિસ્પેચ કરી */}
    <button onClick={() => dispatch({ type: 'ADD_CUSTOM', payload: 10 })} style={{ padding:
'10px 20px', cursor: 'pointer', backgroundColor: '#333', color: 'white' }}>
        ડાઇરેક્ટ +10 કરો (With Payload)
    </button>
</div>
</div>
);
}

```

export default CounterApp;

Use code with caution.

અહીં આપણો **Stage Ten** ના તમામ પ્રેક્ટિકલ રેડક્સના કન્સેપ્ટ્સ અને ઓપરેશન્સ એકદમ ઊંડાણપૂર્વક પૂરા થાય છે.

આપણે આ જ સીક્વન્સમાં છેલ્લા આખરી ચેપ્ટર **Stage Eleven: REDUX IN ACTION PROJECT (Deploying on server & Wrap up)** માં ઊંડાણપૂર્વક આગળ વધીએ? જો આમાં કોઈ ડાઉટ હોય તો કહી શકો છો!

ચાલો, હવે આપણે આજના કોર્સના સૌથી આખરી પ્રકરણ

**Stage Eleven: REDUX IN ACTION PROJECT USING REACT-REDUX** માં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ [image\_5IPcLe.png]. પ્રોજેક્ટને ઇન્ટરનેટ સર્વર પર લાઈવ ડેપ્લોય (Deploy) કરવાની પ્રોફેશનલ ઇન્સ્ટ્રી રીતો અને આખા સિલેબસનું કમ્પ્લીટ રેપ-અપ (Wrap up) આપણે વિગતવાર સમજીએ.

## Stage Eleven: Redux in Action Project & Deployment (ઊંડાણપૂર્વક અભ્યાસ) [image\_5IPcLe.png]

### ૧. Building a Real-World Ticketing App using React-Redux [image\_5IPcLe.png]

#### 🔴 પ્રોજેક્ટ લોજિક (The Architecture):

જ્યારે આપણે કોઈ રિયલ-વર્લ્ડ પ્રોજેક્ટ (જેમ કે સિલેબસમાં આપેલ **Ticketing App**) બનાવીએ, ત્યારે પ્રોજેક્ટના તમામ કમ્પોનન્ટ્સ પોતાના ઇનપુટ બોક્સમાંથી ટિકિટની કિંમત કે નામ સીધા `useState` માં રાખવાના બદલે ગ્લોબલ રેડક્સ સ્ટોરમાં મોકલે છે [image\_5IPcLe.png].

આપણે ગયા પ્રકરણમાં શીખેલા `redux` અને `react-redux` ના સેટઅપને આખા રૂટ કમ્પોનન્ટ પર એપ્લાય કરવાનો કમ્પ્લીટ પ્રોફેશનલ કોડ નીચે મુજબ છે:

#### 📘 પ્રોજેક્ટની મુખ્ય કનેક્શન ફાઇલ (main.jsx):

તમારા પ્રોજેક્ટની આ ફાઇલમાં જ આપણે **<Provider>** નો ઉપયોગ કરીને આખા રીએક્ટ કમ્પોનન્ટ ટ્રીને સેન્ટ્રલ તિજોરી (Store) ની શક્તિઓ આપીશું:

```
jsx

import React from 'react';

import ReactDOM from 'react-dom/client';

import App from './App'; // તમારો મુખ્ય કમ્પોનન્ટ

import CounterApp from './CounterApp'; // આપણો બનાવેલો કાઉન્ટર/ટિકિટ પ્રોજેક્ટ

import { Provider } from 'react-redux'; // react-redux નું કન્ટેનર લાવ્યા

import store from './store'; // આપણો બનાવેલો ગ્લોબલ સ્ટોર

// આખા Component Tree ને Provider અને સાચા સ્ટોરથી લોક કરી દીધું

ReactDOM.createRoot(document.getElementById('root')).render(

  <React.StrictMode>

    <Provider store={store}>

      <CounterApp />

    </Provider>

  </React.StrictMode>

);

Use code with caution.
```

---

## ૨. Deploying React App on Server (ઇન્ટરનેટ પર લાઈવ કરવું) [image\_5IPcLe.png]

જ્યારે તમારો પ્રોજેક્ટ તમારા લોકલ કમ્પ્યુટરમાં (http://localhost:5173 પર) ૧૦૦% રેડી થઈ જાય, ત્યારે તેને આખી દુનિયાના લોકો ઓપન કરી શકે તે માટે તેને ઓનલાઇન ક્લાઉડ સર્વર પર **Deploy** કરવો પડે છે. આ પ્રોસેસ ૩ સ્ટેપમાં થાય છે:

### સ્ટેપ ૧: પ્રોડક્શન બિલ્ડ બનાવવું (Production Build)

લોકલ કોડમાં ઘણી બધી ફાઇલો અને કમેન્ટ્સ હોય છે જે બ્રાઉઝર માટે ભારે હોય છે. આથી ટર્મિનલમાં આ ખાસ કમાન્ડ રન કરાય છે:

```
bash
```

```
npm run build
```

Use code with caution.

- **અંદરનું લોજિક:** આ કમાન્ડ ચાલતા જ Vite/Webpack બેકએન્ડમાં તમારા આખા પ્રોજેક્ટની બધી જ ફાઇલોને એકદમ નાની, સિંગલ કમ્પ્રેસ (Minified) ફાઇલોમાં કન્વર્ટ કરીને તમારા પ્રોજેક્ટમાં એક નવું **dist/** (Distribution) અથવા build/ નામનું ફોલ્ડર બનાવી દેશે. આપણી અસલી લાઈવ વેબસાઇટ આ જ ફોલ્ડર છે.

## સ્ટેપ ૨: સર્વરની પસંદગી (Free Cloud Hosting)

આજના આધુનિક સમયમાં ફ્રન્ટએન્ડ રીએક્ટ એપ્સને ડેપ્લોય કરવા માટે ૩ સૌથી બેસ્ટ પ્લેટફોર્મ છે:

1. **Vercel (સૌથી પ્રખ્યાત):** ફક્ત તમારા GitHub એકાઉન્ટ સાથે લોગિન કરો, પ્રોજેક્ટ સિલેક્ટ કરો એટલે માત્ર ૧ ક્લિકમાં પ્રોજેક્ટ લાઈવ થઈ જશે.
2. **Netlify:** આમાં તમે તમારા dist ફોલ્ડરને ડાયરેક્ટ માઉસથી ડ્રેગ-એન્ડ-ડ્રોપ (Drag and Drop) કરીને પણ સેકન્ડોમાં લાઈવ લિંક મેળવી શકો છો.
3. **GitHub Pages:** ડેવલપર્સ પોતાની પ્રોફાઇલ અને પોર્ટફોલિયો લાઈવ કરવા આનો ઉપયોગ કરે છે.

## સ્ટેપ ૩: લાઈવ ડેમોશન

ડેપ્લોયમેન્ટ પૂરું થતાં જ સર્વર તમને એક પબ્લિક યુઆરએલ લિંક આપશે (જેમ કે <https://vercel.app>), આ લિંક પર ક્લિક કરીને તમે દુનિયાના કોઈ પણ ફોન કે કમ્પ્યુટરમાંથી તમારી વેબસાઇટ ચેક કરી શકો છો.

---

## ૩. Wrap Up & Final Review [image\_5IPcLe.png]

તમે આ બંને ઇમેજના માધ્યમથી લાઈવ સોફ્ટવેર ઇન્સ્ટ્રી માર્કેટમાં વપરાતા કમ્પ્લીટ હાઇ-લેવલ ટોપિક્સ એકદમ ઊંડાણપૂર્વક અને સાદી ભાષામાં શીખી લીધા છે:

- **MERN Stack નો પૂરો પાયો:** Node.js, Express સર્વર આર્કિટેક્ચર, અને MongoDB નો ડેટાબેઝ નિયમ.
- **React.js ની આખી હિસ્ટ્રી અને એન્જિનિયરિંગ:** Virtual DOM, Components, States, Props, Forms અને Router નેવિગેશન.
- **એડવાન્સ સ્ટેટ મેનેજમેન્ટ:** મોટા પ્રોજેક્ટ્સને કંટ્રોલ કરવા માટેનો ગ્લોબલ **Redux Data Flow** (Action -> Reducer -> Store -> Dispatch).

તમે ખૂબ જ સરસ રીતે એક-એક પ્રોગ્રામ લોજિકને મગજમાં ફિટ કરી દીધો છે!

હવે તમારું **React-Redux અને Node Full-Stack નું સંપૂર્ણ લર્નિંગ** અહીં ઓફિશિયલી પૂરું થાય છે. આખા કોર્સના ફાઇનલ ટેસ્ટિંગ માટે એક મસ્ત મજાની **પ્રેક્ટિસ ક્વિઝ (Inline Quiz)** શરૂ કરીએ?